C

CHEAT SHEETS

A Complete Reference for Modern C

Types & Strings · Operators · Control Flow · Functions · Pointers · Memory Management · Data Structures · Structs & Unions · File I/O · Preprocessor · Builds & Tools · Threads & Atomics

98 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–05 Introduction & Setup
06–08 Output & Comments
09–16 Variables & Data Types
17–21 Strings
22–26 Operators
27–32 Control Flow

CORE LANGUAGE

33–35 Error Handling
36–43 Functions
44–49 Pointers
50–54 Memory Management
55–62 Arrays & Data Structures
63–69 Structs, Unions & Enums

SYSTEMS & TOOLS

70–75 Files & I/O
76–82 Preprocessor, Headers & Builds
83–85 Standard Library
86–89 Tools & External Libraries
90–96 Advanced Topics
97–98 Quick Reference

C

Table of Contents

98 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & History
- 02 Compilers & Toolchain
- 03 Compiling & Running
- 04 Compiler Flags & Warn…
- 05 Program Structure

OUTPUT & COMMENTS 06 – 08

- 06 printf & Basic Output
- 07 printf Format Specifiers
- 08 Comments

VARIABLES & DATA TYPES 09 – 16

- 09 Variables & Declaratio…
- 10 Integer Types
- 11 Fixed-Width Integers
- 12 Floating-Point Types
- 13 The char Type
- 14 Booleans & Truthiness
- 15 Type Conversion & Ca…
- 16 const, volatile & sizeof

STRINGS 17 – 21

- 17 String Basics
- 18 Copying & Concatenat…
- 19 Comparing & Searching
- 20 String ↔ Number Conv…
- 21 Character Classes & U…

OPERATORS 22 – 26

- 22 Arithmetic Operators
- 23 Comparison & Logical …
- 24 Bitwise Operators
- 25 Assignment, Comma &…
- 26 Precedence & Evaluati…

CONTROL FLOW 27 – 32

- 27 if / else if / else
- 28 Ternary Operator
- 29 switch
- 30 for Loops
- 31 while & do-while
- 32 break, continue & goto

ERROR HANDLING 33 – 35

- 33 Return Codes & errno
- 34 assert, static_assert &…
- 35 Error Handling Patterns

FUNCTIONS 36 – 43

- 36 Defining & Calling Func…
- 37 Parameters & Pass-by…
- 38 Variadic Functions
- 39 Scope & Linkage
- 40 Storage Duration & Lif…
- 41 Function Pointers
- 42 Recursion
- 43 inline & Attributes

POINTERS 44 – 49

- 44 Pointer Basics
- 45 Pointer Arithmetic
- 46 Pointers & Arrays
- 47 Pointers to Pointers
- 48 const & void Pointers
- 49 Pointer Pitfalls

MEMORY MANAGEMENT 50 – 54

- 50 malloc & free
- 51 calloc, realloc & Aligne…
- 52 Memory Functions
- 53 Memory Layout
- 54 Memory Debugging

ARRAYS & DATA STRUCTURES 55 – 62

- 55 Array Basics
- 56 Multidimensional Arrays
- 57 Dynamic Arrays
- 58 Linked Lists
- 59 Stacks, Queues & Ring …
- 60 Hash Tables
- 61 Trees & Heaps
- 62 Sorting & Searching

STRUCTS, UNIONS & ENUMS 63 – 69

- 63 Struct Basics
- 64 Struct Pointers & type…
- 65 Struct Layout & Paddi…
- 66 Unions
- 67 Enums
- 68 Bit-Fields
- 69 Opaque Types & OOP …

FILES & I/O 70 – 75

- 70 Console Input
- 71 Opening & Closing Files
- 72 Reading Text Files
- 73 Writing Text Files
- 74 Binary Files
- 75 Files & Directories (PO…

PREPROCESSOR, HEADERS & BUILDS 76 – 82

- 76 #include & Header Files
- 77 Macros
- 78 Conditional Compilation
- 79 Advanced Preprocessor
- 80 Multi-File Projects & Li…
- 81 Makefiles
- 82 CMake

STANDARD LIBRARY 83 – 85

- 83 stdlib.h Utilities
- 84 Math
- 85 Time & Dates

TOOLS & EXTERNAL LIBRARIES 86 – 89

- 86 Debugging with gdb & …
- 87 Unit Testing
- 88 Static Analysis & Form…
- 89 Popular C Libraries

ADVANCED TOPICS 90 – 96

- 90 Undefined Behavior
- 91 C11 Threads
- 92 POSIX Threads
- 93 Atomics
- 94 Signals & Processes
- 95 Generic Programming
- 96 C23 at a Glance

QUICK REFERENCE 97 – 98

- 97 Quick Reference — Lan…
- 98 Quick Reference — Lib…

Introduction & History

WHAT C IS

C is a small, compiled, statically typed systems language created by **Dennis Ritchie** at Bell Labs (1972) to write Unix. It gives you direct control over memory through **pointers**, compiles to fast native code, and runs almost everywhere: operating systems, embedded firmware, databases, language runtimes. The trade-off is that the compiler trusts you: there are no bounds checks, no garbage collector, and many mistakes are **undefined behavior** rather than errors.

KEY FACTS

- Source files end in `.c`; headers end in `.h`
- ISO/IEC 9899; newest edition is **C23** (2024)
- This set targets **C17** and tags newer features `(C23)`
- C++ is a separate language, not a superset

HELLO, WORLD

```c
/* hello.c */
#include <stdio.h>              // declares printf

int main(void)                  // program starts here
{
    printf("Hello, World!\n");  // Hello, World!
    return 0;                   // 0 = success
}

/* Check which standard the compiler is using */
#if __STDC_VERSION__ >= 201710L
    /* C17 or newer */
#endif
```

CODE EXPLAINED

- → `#include <stdio.h>` — pulls in the standard I/O declarations
- → `int main(void)` — entry point; returns an exit status to the OS
- → `\n` — newline; `printf` doesn't add one for you
- → `__STDC_VERSION__` — predefined macro holding the standard as a `long` date
- → Build & run: `cc hello.c -o hello && ./hello`

STANDARDS TIMELINE

STANDARD	YEAR	HIGHLIGHTS
K&R C	1978	the book *The C Programming Language*; no official standard
C89 / C90	1989	ANSI then ISO; function prototypes, `void`, `const`
C99	1999	`//` comments, `stdint.h`, `bool`, `inline`, VLAs, designated initializers
C11	2011	`_Generic`, `_Static_assert`, threads, atomics, anonymous structs
C17	2018	bug-fix release of C11; `__STDC_VERSION__` = `201710L`
C23	2024	`nullptr`, `bool` keyword, `constexpr`, `typeof`, `#embed`, `[[attributes]]`

KEY TERMS

TERM	MEANING
translation unit	one `.c` file after the preprocessor runs
undefined behavior	the standard sets no rules; anything can happen
implementation-defined	compiler chooses and must document it
hosted / freestanding	full standard library vs. bare-metal subset
libc	the C standard library (glibc, musl, Apple libSystem, MSVC CRT)

⚠ COMMON MISTAKES

- Treating C as "C++ without classes": casts, `bool`, empty `()` and string literals all behave differently
- Assuming the compiler will catch memory bugs. It won't; out-of-bounds writes compile fine and corrupt memory silently
- Mixing up the standard (C17) with a compiler version (GCC 14). Pick the standard with `-std=`

✓ TIPS & BEST PRACTICES

- Always compile with warnings on: `-Wall -Wextra` catches a large share of beginner bugs
- Bookmark **cppreference.com/w/c**: it has a C-only section with per-standard notes
- Read *K&R* for style, but check modern references; it predates C99

QUICK SUMMARY

- `C` — compiled, low-level, portable systems language
- `C17` — baseline here; `C23` is newest
- `main` — entry point, returns exit status
- `UB` — compiles, but results are unpredictable

Compilers & Toolchain

gcc · clang · MSVC · xcode-select --install · cc --version · -std=c17 / -std=c23 · editor + LSP (clangd)

GETTING A COMPILER

C code needs a **compiler** that turns source into machine code, plus a linker and the C library. On macOS the Xcode Command Line Tools install **Clang** (and alias `gcc` to it). Linux distributions ship **GCC** and usually offer Clang. On Windows use **MSVC** (Visual Studio) or GCC/Clang through MSYS2 or WSL. Pair the compiler with an editor that uses **clangd** for completion and inline errors.

GOOD TO KNOW

- On macOS, `gcc` is really **Apple clang**
- `cc` is the system's default C compiler on Unix-like systems
- Older compilers spell C23 as `-std=c2x`
- MSVC flags use `/` : `/std:c17`, `/W4`

INSTALL & VERIFY

```
# macOS: Apple Clang + make + lldb
xcode-select --install
clang --version          # Apple clang version 17.x ...

# macOS: real GCC via Homebrew (installs as gcc-15)
brew install gcc

# Debian / Ubuntu
sudo apt install build-essential clang gdb

cc --version             # which compiler is "cc"?

# Pick a standard explicitly
cc -std=c17 hello.c -o hello
cc -std=c23 hello.c -o hello
```

CODE EXPLAINED

→ `xcode-select --install` — Clang, `make`, `lldb`, headers & SDK

→ `build-essential` — GCC, `make`, libc headers on Debian/Ubuntu

→ `brew install gcc` — versioned binary like `gcc-15`; plain `gcc` stays Clang

→ `-std=c17` — choose the language standard; default varies by compiler

→ `-std=gnu17` — C17 plus GNU extensions (GCC's default family)

COMPILERS

COMPILER	PLATFORMS	NOTES
`gcc`	Linux, macOS (brew), Windows (MSYS2)	GNU; best `-fanalyzer`, wide target support
`clang`	macOS, Linux, Windows	LLVM; clear errors, sanitizers, clangd/clang-tidy
`cl (MSVC)`	Windows	Visual Studio; C11/C17 since VS 2019 16.8
`zig cc`	all	Clang wrapper with easy cross-compiling

EDITOR & TOOLING

TOOL	PURPOSE
`clangd`	language server: completion, go-to-def, errors
`compile_commands.json`	tells clangd your flags (CMake/Bear generate it)
`CLion / Xcode`	full IDEs with CMake / project support
`lldb / gdb`	debuggers (Sheet 86)

⚠ COMMON MISTAKES

- Thinking `gcc` on a Mac is GNU GCC. It's Clang unless you call `gcc-15` (or similar) from Homebrew
- Not passing `-std=`: you get the compiler's default (often `gnu17`), so code may rely on extensions without you knowing
- Running MSVC with C++ settings on a `.c` file; use `/TC` or keep the `.c` extension

✓ TIPS & BEST PRACTICES

- Use both GCC and Clang in CI; each catches warnings the other misses
- Generate `compile_commands.json` with `cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON` or `bear -- make`
- Add `-std=c17 -Wall -Wextra -pedantic` to every build from day one

QUICK SUMMARY

- `clang` / `gcc` — the two main open compilers
- `cc --version` — see what you're really running
- `-std=c17` — pin the language standard
- `clangd` — editor intelligence for C

Compiling & Running

FROM SOURCE TO EXECUTABLE

Building a C program runs four stages: the **preprocessor** expands `#include` and macros, the **compiler** turns C into assembly, the **assembler** produces an object file (`.o`), and the **linker** combines object files and libraries into an executable. One `cc` command normally runs all four; flags let you stop after any stage, which is useful for debugging macros or inspecting generated code.

NOTES

- Each `.c` file compiles **independently** into its own `.o`
- Link errors (`undefined reference`) come from the linker, not the compiler
- Without `-o` , the output is named `a.out`
- Order matters on the command line: libraries go **after** the files that use them

BUILD COMMANDS

```
cc main.c -o app            # all stages → ./app
./app                       # run it (needs ./ on Unix)
./app; echo $?              # print exit status

cc -E main.c > main.i       # stop after preprocessing
cc -S main.c                # stop after compiling → main.s
cc -c main.c                # stop after assembling → main.o

# Multi-file build: compile separately, then link
cc -c main.c util.c         # main.o util.o
cc main.o util.o -o app     # link

# Headers, library paths, libraries
cc main.c -Iinclude -Llib -lmylib -lm -o app
```

CODE EXPLAINED

- → `-o app` — name the output file
- → `-E` — preprocessed source only (see what macros became)
- → `-S` — assembly listing; add `-O2` to see optimized code
- → `-c` — object file, no linking
- → `-I` / `-L` / `-l` — header dir / library dir / link `libNAME`

BUILD STAGES

STAGE	INPUT → OUTPUT	FLAG
Preprocess	`.c` → `.i` (text)	–E
Compile	`.i` → `.s` (assembly)	–S
Assemble	`.s` → `.o` (object)	–c
Link	`.o` + libs → executable	(default)

FILE TYPES

EXTENSION	WHAT IT IS
`.c` / `.h`	source / header
`.o` (`.obj`)	object file (machine code + symbols)
`.a` (`.lib`)	static library: archive of `.o` files
`.so` / `.dylib` / `.dll`	shared library
`a.out`	default executable name

WHO REPORTS WHAT

ERROR	STAGE
`x.h: No such file`	preprocessor
`expected ';'`	compiler
`undefined reference`	linker (GNU)
`Undefined symbols`	linker (macOS)
`Segmentation fault`	runtime

⚠ COMMON MISTAKES

- Putting `-lm` before the source file with GNU ld: `cc -lm main.c` can fail; write `cc main.c -lm`
- Compiling only `main.c` when the program also needs `util.c` , then getting `undefined reference`
- Running `app` instead of `./app` ; the current directory isn't on `PATH`

✓ TIPS & BEST PRACTICES

- Use `cc -E file.c | less` to debug a confusing macro
- Use `-save-temps` to keep the `.i` , `.s` and `.o` files from one build
- Let `make` or CMake track multi-file builds (Sheets 81–82)

QUICK SUMMARY

- `cc x.c -o x` — compile and link
- `-E` / `-S` / `-c` — stop after a stage
- `.o` — object file per `.c`
- `-lNAME` — link a library, after the sources

Sheet 04

Compiler Flags & Warnings

-Wall -Wextra -Wpedantic · -Werror · -Wconversion / -Wshadow · -O0…-O3 / -Os · -g · -D NAME=val · -fsanitize=address,undefined

FLAGS WORTH USING EVERY TIME

C compilers are quiet by default and will build code with obvious bugs. Warning flags turn on hundreds of checks for things like unused variables, wrong `printf` formats and implicit conversions. Optimization flags change speed and size, debug flags add symbols for the debugger, and **sanitizers** add runtime checks that catch memory errors and undefined behavior while the program runs.

RECOMMENDED SETS

- Dev: `-std=c17 -Wall -Wextra -g -O0`
- Bug hunting: add `-fsanitize=address,undefined`
- Release: `-O2 -DNDEBUG` (keep warnings on)
- `-Wall` is **not** all warnings

TYPICAL COMMAND LINES

```
# Strict development build
cc -std=c17 -Wall -Wextra -Wpedantic -Wshadow \
   -Wconversion -g -O0 main.c -o app

# Treat warnings as errors (CI)
cc -Wall -Wextra -Werror main.c -o app

# Sanitizer build: crashes with a report on bugs
cc -g -O1 -fsanitize=address,undefined \
   -fno-omit-frame-pointer main.c -o app

# Release build, with a macro defined
cc -O2 -DNDEBUG -DVERSION=\"1.2\" main.c -o app
```

CODE EXPLAINED

- → `-Wpedantic` — warn on non-ISO extensions for the chosen `-std`
- → `-Wconversion` — implicit conversions that may change a value
- → `-Werror` — any warning stops the build
- → `-fsanitize=address` — out-of-bounds, use-after-free, leaks
- → `-DNAME=val` — like `#define NAME val` in every file

WARNING FLAGS

FLAG	CATCHES
`-Wall`	unused vars, bad `printf` args, missing `return`
`-Wextra`	unused params, signed/unsigned compares
`-Wshadow`	inner variable hides an outer one
`-Wconversion`	`int` → `char`, `double` → `int` truncation
`-Wformat=2`	stricter `printf` / `scanf` checks

OPTIMIZATION

FLAG	EFFECT
`-O0`	no optimization; best for debugging
`-Og`	light; keeps debugging usable
`-O2`	standard release level
`-O3`	aggressive (vectorizing, inlining)
`-Os` / `-Oz`	optimize for size
`-g`	debug symbols (combine with any –O)

SANITIZERS

FLAG	DETECTS
`address`	overflows, use-after-free, double free
`undefined`	signed overflow, bad shifts, null deref
`thread`	data races (not with ASan)
`memory`	uninitialized reads (Clang, Linux)

⚠ COMMON MISTAKES

- Shipping a binary built with `-O0 -g` and wondering why it's slow
- Ignoring warnings because "it runs". A `printf` format mismatch or uninitialized variable is a real bug
- Combining `-fsanitize=address` with `-fsanitize=thread`; they can't run together

✓ TIPS & BEST PRACTICES

- Put flags in a Makefile/CMake variable so every file gets the same set
- Run tests under ASan + UBSan regularly; it's the cheapest way to find memory bugs
- Use `-Werror` in CI, not necessarily on your laptop, so new compiler versions don't block you

QUICK SUMMARY

- `-Wall -Wextra -Wpedantic` — baseline warnings
- `-g -O0` — debug; `-O2` — release
- `-fsanitize=address,undefined` — runtime bug detection
- `-DNAME=val` — define a macro at build time

Sheet 05

Program Structure

int main(void) · int main(int argc, char *argv[]) · return 0 / EXIT_SUCCESS · #include · statements & semicolons · blocks { } · declare before use

ANATOMY OF A C PROGRAM

A C program is a set of **declarations** and **function definitions** spread over one or more source files. Execution starts in `main`. Statements end with a semicolon, blocks are wrapped in braces, and whitespace and indentation don't matter to the compiler. Everything must be **declared before use**, which is why headers with function prototypes are included at the top of the file.

RULES

- Two valid `main` signatures: `int main(void)` and `int main(int argc, char *argv[])`
- Reaching the end of `main` without `return` returns `0` (C99+); only `main` does this
- `argv[0]` is the program name; `argv[argc]` is `NULL`
- C is case-sensitive: `Main` is not `main`

ANNOTATED PROGRAM

```c
#include <stdio.h>        // printf
#include <stdlib.h>       // EXIT_SUCCESS, EXIT_FAILURE

static int square(int n); // prototype: declare before use

int main(int argc, char *argv[])
{
    if (argc < 2) {                         // no argument given
        fprintf(stderr, "usage: %s N\n", argv[0]);
        return EXIT_FAILURE;                // non-zero = failure
    }
    int n = atoi(argv[1]);                  // "7" -> 7
    printf("%d squared is %d\n", n, square(n));
    return EXIT_SUCCESS;                    // 0
}

static int square(int n)  // definition (file-local)
{
    return n * n;
}
```

CODE EXPLAINED

- → `#include` / `#define` — preprocessor lines, no semicolon
- → `static int square(int n);` — prototype so `main` can call it before its definition
- → `argc` / `argv` — count and array of command-line strings
- → `fprintf(stderr, …)` — errors go to standard error
- → `EXIT_SUCCESS` / `EXIT_FAILURE` — portable exit codes

BUILDING BLOCKS

ELEMENT	EXAMPLE	ENDS WITH
preprocessor directive	`#include <stdio.h>`	newline
declaration	`int count;`	`;`
statement	`count = count + 1;`	`;`
block	`{ ... }`	`}` (no `;`)
function definition	`int f(void) { ... }`	`}`

RUNNING ./APP 7 X

EXPRESSION	VALUE
argc	3
argv[0]	"./app"
argv[1]	"7" (a string, not an int)
argv[2]	"x"

⚠ COMMON MISTAKES

- Calling a function before it's declared. Pre-C99 compilers guessed `int f()`; modern ones error out. Add a prototype or include the header
- Forgetting the `;` after a `struct` definition; the error often appears on the next line
- Treating `argv[1]` as a number; convert it with `strtol` (Sheet 20)

✓ TIPS & BEST PRACTICES

- Keep `main` short: parse arguments, call functions, return a status
- Mark helper functions `static` so they're private to the file
- Print usage messages and errors to `stderr`, results to `stdout`

QUICK SUMMARY

- `int main(void)` — program entry point
- `argc` / `argv` — command-line arguments
- `;` ends statements; `{}` groups them
- Declare (prototype) before use

printf & Basic Output

PRINTING WITH PRINTF

`printf` writes formatted text to standard output. Its first argument is a **format string**; each % **conversion specifier** in it consumes the next argument and must match that argument's type exactly. `puts` and `putchar` are simpler, faster options for plain strings and single characters, and `fprintf(stderr, ...)` sends output to the error stream.

NOTES

- `printf` returns the number of characters written (negative on error)
- `puts` adds a newline; `fputs` does not
- `stdout` is line-buffered on a terminal
- `float` args become `double`; `%f` fits both

OUTPUT FUNCTIONS

```c
int n = 42; unsigned u = 7u; long l = 123456789L;
long long ll = 9000000000LL; size_t sz = sizeof(int);
double d = 3.14159; char c = 'A'; const char *s = "text";

printf("%d %i\n", n, n);        // 42 42
printf("%u\n", u);              // 7
printf("%ld %lld\n", l, ll);    // 123456789 9000000000
printf("%zu\n", sz);            // 4
printf("%f\n", d);              // 3.141590
printf("%e\n", d);              // 3.141590e+00
printf("%g\n", d);              // 3.14159
printf("%c %s\n", c, s);        // A text
printf("%p\n", (void *)&n);     // 0x7ffd5c3a8b4c
printf("100%%\n");              // 100%

puts("line");                   // line + newline
putchar('x');                   // x (no newline)
```

CODE EXPLAINED

- → `%d` / `%i` — `int`; same in `printf`, different in `scanf`
- → `l`, `ll`, `z` — length modifiers for `long`, `long long`, `size_t`
- → `%f` — fixed, 6 decimals by default; `%e` scientific; `%g` shortest
- → `%p` — pointer; cast to `void *` to be strictly correct
- → `%%` — a literal percent sign

CONVERSION SPECIFIERS

SPEC	TYPE	EXAMPLE → OUTPUT
`%d %i`	`int`	`-5` → `-5`
`%u`	`unsigned int`	`7u` → `7`
`%ld %lld`	`long` / `long long`	`5L` → `5`
`%zu`	`size_t`	`sizeof x` → `8`
`%f %e %g`	`double`	`0.5` → `0.500000`
`%c`	`int` (char)	`'A'` → `A`
`%s`	`char *`	`"hi"` → `hi`
`%p`	`void *`	`0x7ff...`

OUTPUT FUNCTIONS

FUNCTION	DOES
`printf(fmt, ...)`	formatted → `stdout`
`fprintf(f, fmt, ...)`	formatted → any `FILE *`
`puts(s)`	string + `\n` → `stdout`
`putchar(c)`	one character
`snprintf(buf, n, fmt, ...)`	formatted → string (Sheet 18)

⚠ COMMON MISTAKES

- `printf("%d", 3.5)` or `printf("%f", 3)`: a mismatched type is undefined behavior and prints garbage
- `printf(user_input)`: if the text contains `%`, it's read as a format. Use `printf("%s", user_input)`
- Using `%d` for `size_t` or `long`; use `%zu` / `%ld`. `-Wformat` warns

✓ TIPS & BEST PRACTICES

- Use `puts` for fixed lines; it's simpler and can't misread `%`
- Call `fflush(stdout)` before a long operation if a prompt has no newline
- Keep `-Wall` on: GCC and Clang check `printf` arguments against the format

QUICK SUMMARY

- `printf(fmt, ...)` — formatted output
- `%d %u %ld %zu %f %c %s %p` — core specifiers
- `puts` / `putchar` — plain output
- `fprintf(stderr, ...)` — errors

Sheet 07

printf Format Specifiers

width %5d · precision %.2f / %.3s · flags - + 0 # space · %*d · %x %X %o · PRId64 / PRIu32 · mismatched specifier = UB

CONTROLLING THE FORMAT

A full conversion specification is `%[flags][width][.precision][length]type`. **Width** sets a minimum field size, **precision** sets decimals for floats or a maximum length for strings, and **flags** control alignment, signs and padding. Fixed-width types like `int64_t` need the macros from `<inttypes.h>`, because their underlying type differs between platforms.

NOTES

- A value wider than the width is never truncated
- `*` takes the width or precision from an argument (`int`)
- `%x` / `%o` expect **unsigned** values
- Wrong specifier for the argument type is **undefined behavior**

WIDTH, PRECISION & FLAGS

```c
#include <inttypes.h>

printf("[%5d]\n", 42);        // [   42]
printf("[%-5d]\n", 42);       // [42   ]
printf("[%05d]\n", 42);       // [00042]
printf("[%+d]\n", 42);        // [+42]
printf("[% d]\n", 42);        // [ 42]
printf("[%.2f]\n", 3.14159);  // [3.14]
printf("[%8.3f]\n", 3.14159); // [   3.142]
printf("[%.3s]\n", "abcdef"); // [abc]
printf("[%*d]\n", 6, 42);     // [    42]
printf("[%.*f]\n", 1, 2.25);  // [2.2] (round-half-even here)
printf("%x %X %o\n", 255u, 255u, 8u);  // ff FF 10
printf("%#x %#o\n", 255u, 8u);         // 0xff 010
printf("%08.3f\n", -3.5);              // -003.500

int64_t big = 9007199254740993;
uint32_t id = 42;
printf("%" PRId64 " %" PRIu32 "\n", big, id);
```

CODE EXPLAINED

- → `%5d` — right-align in 5 columns; `%-5d` left-align
- → `%05d` — pad with zeros instead of spaces
- → `%.2f` — 2 digits after the point; `%.3s` — at most 3 chars
- → `%#x` — add `0x` prefix (`0` for octal)
- → `PRId64` — string macro, e.g. `"lld"` or `"ld"` ; joined by adjacent-literal concatenation

FLAGS

FLAG	EFFECT	EXAMPLE
-	left-justify	%-6s → ab
+	always show sign	%+d → +5
space	space before positives	% d → 5
0	zero-pad numbers	%04d → 0005
#	alternate form (0x, 0, keep .)	%#x → 0xff

LENGTH MODIFIERS

MODIFIER	TYPE
hh / h	char / short
l / ll	long / long long
z	size_t
t	ptrdiff_t
j	intmax_t
L	long double (%Lf)

<INTTYPES.H> MACROS

MACRO	FOR
PRId32 / PRId64	int32_t / int64_t
PRIu8 … PRIu64	unsigned fixed widths
PRIx32 / PRIX64	hex
PRIdPTR	intptr_t
SCNd32	scanf versions

⚠ COMMON MISTAKES

- `printf("%ld", int64_value)` : `int64_t` is `long` on 64-bit Linux but `long long` on Windows and macOS. Use `PRId64`
- Expecting `%.3s` to pad. It only truncates; combine with width: `%-10.3s`
- Passing a negative `int` to `%x` ; cast to `unsigned` first

✓ TIPS & BEST PRACTICES

- Align table output with fixed widths: `printf("%-12s %8.2f\n", name, price)`
- Use `%*s` with a computed width for indentation: `printf("%*s", depth * 2, "")`
- In C23, `%b` prints binary on newer libcs (glibc 2.35+); don't rely on it for portable code

QUICK SUMMARY

- `%[flags][width][.prec][len]type` — full spec
- `%-10s` / `%08.2f` — align and pad
- `%*d` — width from an argument
- `PRId64` — portable fixed-width printing

Comments

COMMENTS IN C

C has two comment forms: **block comments** `/* ... */`, which can span lines, and **line comments** `//`, added in C99. Comments are replaced by a single space before preprocessing, so they can appear almost anywhere. Block comments don't nest, so the reliable way to disable a chunk of code that already contains comments is `#if 0 ... #endif`.

NOTES

- `//` is C99+; strict `-std=c89` rejects it
- Comments inside string literals are just text: `"/* no */"`
- A `\` at the end of a `//` line continues the comment onto the next line
- Doxygen reads `/** */` and `///` comments to generate docs

COMMENT STYLES

```c
/*
 * vec.c — small dynamic array
 * Author: Ana Ruiz   License: MIT
 */

#include <stdlib.h>   // line comment: runs to end of line

/**
 * Append a value, growing the buffer if needed.
 *
 * @param v      vector to modify (not NULL)
 * @param value  value to append
 * @return 0 on success, -1 if allocation failed
 */
int vec_push(struct vec *v, int value);

int x = 10; /* inline block comment */

#if 0
    old_code();   /* comments inside are fine here */
#endif
```

CODE EXPLAINED

→ File header block — purpose, author, license

→ `/** ... */` — Doxygen doc comment; `@param`, `@return` tags

→ `/* */` mid-line — works because it becomes a space

→ `#if 0` — the preprocessor drops everything until `#endif`

COMMENT FORMS

SYNTAX	SINCE	USE FOR
`/* ... */`	C89	headers, multi-line notes
`// ...`	C99	short end-of-line notes
`/** ... */`	Doxygen	function / type docs
`///`	Doxygen	one-line docs
`#if 0 ... #endif`	C89	disabling code

COMMON DOXYGEN TAGS

TAG	MEANING
`@brief`	one-line summary
`@param name`	describe a parameter (`[in]`, `[out]`)
`@return`	return value
`@note / @warning`	extra info / caution

⚠ COMMON MISTAKES

- Nesting block comments: `/* a /* b */ c */` ends at the first `*/`, leaving `c */` as code
- Commenting out code with `/* */` when it already contains a block comment; use `#if 0`
- Comments that repeat the code (`i++; // add 1 to i`) instead of explaining why

✓ TIPS & BEST PRACTICES

- Document every function in its header: what it does, ownership of pointers, error returns
- Say who frees returned memory: "caller must `free()` the result"
- Prefer `#if 0` to deleting code only briefly; version control keeps the history

QUICK SUMMARY

- `/* */` — block comment, doesn't nest
- `//` — line comment (C99)
- `/** @param */` — Doxygen docs
- `#if 0` — disable a code block

Sheet 09

Variables & Declarations

declaration vs definition · initialization · uninitialized = indeterminate · int *a, b gotcha · identifier rules · reserved _X / __x names · auto (C23)

DECLARING VARIABLES

A **declaration** tells the compiler a name and its type; a **definition** also reserves storage. Local variables that aren't initialized hold an **indeterminate** value, and reading one is undefined behavior. Globals and `static` variables start at zero. C99 lets you declare variables anywhere in a block, not just at the top.

NAMING RULES

- Letters, digits, `_`; can't start with a digit
- `_Upper` and `__x` names are reserved
- Case-sensitive: `count` ≠ `Count`
- Common style: `snake_case` for variables

DECLARATIONS & INITIALIZATION

```c
int count;              // local: indeterminate!
int total = 0;          // initialized
double rate = 0.075, tax = 0.0;
char grade = 'A';

int *a, b;              // a is int*, b is plain int
int *c, *d;             // both pointers

extern int shared;      // declaration only (defined elsewhere)
int shared = 5;         // definition

static int calls;       // zero-initialized (static storage)
auto n = 10;            // (C23) n is int
const int MAX = 100;    // read-only after init
```

CODE EXPLAINED

- → `int count;` — inside a function, reading it before assignment is UB
- → `int *a, b;` — the `*` binds to `a` only
- → `extern` — says the variable exists in another file
- → `auto` (C23) — type inferred from initializer; before C23 `auto` meant automatic storage

DEFAULT VALUES

WHERE	STARTS AS
`local int x;`	indeterminate (UB to read)
`global / file scope`	`0`
`static local`	`0`
`malloc memory`	indeterminate
`calloc memory`	all bits zero

DECLARATION VS DEFINITION

CODE	KIND
`extern int x;`	declaration
`int x; (file scope)`	tentative definition
`int x = 1;`	definition
`int f(void);`	declaration (prototype)
`int f(void) { }`	definition

STORAGE KEYWORDS

KEYWORD	EFFECT
`static`	lives for whole program; file-private at file scope
`extern`	defined in another file
`thread_local`	one copy per thread (C23 keyword)
`register`	hint only; can't take address
`auto`	(C23) infer type from initializer

⚠ COMMON MISTAKES

- `int total; total += x;` reads garbage. Initialize: `int total = 0;`
- `int* a, b;` makes only `a` a pointer; spacing doesn't change that
- Defining a global (`int x = 1;`) in a header: every file that includes it defines `x` again

✓ TIPS & BEST PRACTICES

- Declare variables where first used, initialized, in the smallest scope
- One declarator per line avoids the `int *a, b` trap
- Turn on `-Wuninitialized` (in `-Wall`) and run with `-O1` or higher so it can see more

QUICK SUMMARY

- `int x = 0;` — declare + initialize
- Locals start indeterminate; globals start at 0
- `extern` — declare, define elsewhere
- `auto` (C23) — inferred type

Integer Types

INTEGER TYPES

C has five standard signed integer types, each with an `unsigned` partner. The standard only sets **minimum** sizes, so the exact width depends on the platform: `int` is 32 bits almost everywhere, but `long` is 64 bits on Linux/macOS and 32 bits on Windows. `<limits.h>` gives the real ranges. Literals can be written in decimal, hex, octal and (C23) binary, with suffixes that set their type.

NOTES

- `sizeof(char)` is always `1`
- Signed overflow is **undefined behavior**
- Unsigned arithmetic wraps modulo 2^n
- Use `<stdint.h>` when exact width matters

INTEGER LITERALS & LIMITS

```c
#include <limits.h>

int dec = 42;           // decimal
int hex = 0x2A;         // 42
int oct = 052;          // 42 (leading 0 = octal!)
int bin = 0b101010;     // 42 (C23)
long big = 1'000'000L;  // (C23) digit separator

unsigned u   = 42u;
long long ll = 42LL;
unsigned long long ull = 42ULL;

printf("%d\n", INT_MAX);    // 2147483647
printf("%d\n", INT_MIN);    // -2147483648
printf("%u\n", UINT_MAX);   // 4294967295
printf("%zu\n", sizeof(long)); // 8 (Linux/macOS), 4 (Windows)
```

CODE EXPLAINED

- → `0x` hex, `0` octal, `0b` binary (C23)
- → `'` digit separator (C23) — ignored by the compiler
- → `u`, `l`, `ll` suffixes — combine freely: `42ull`
- → `INT_MAX` — from `<limits.h>`; also `LONG_MAX`, `CHAR_BIT`

TYPICAL SIZES (64-BIT)

TYPE	MIN	UNIX	WIN
char	8	8	8
short	16	16	16
int	16	32	32
long	32	64	32
long long	64	64	64

<LIMITS.H>

MACRO	TYPICAL VALUE
CHAR_BIT	8
INT_MAX	2147483647
UINT_MAX	4294967295
LONG_MAX	9223372036854775807 (LP64)
LLONG_MIN	−9223372036854775808

LITERAL SUFFIXES

LITERAL	TYPE
42	int
42u	unsigned int
42L	long
42ULL	unsigned long long
0xFFFFFFFF	unsigned int (too big for int)

⚠ COMMON MISTAKES

- Writing `010` and expecting ten. A leading zero means octal: `010 == 8`
- Assuming `long` is 64-bit; it's 32-bit on Windows. Use `int64_t` or `long long`
- `INT_MAX + 1` is undefined behavior, not `INT_MIN`. The optimizer may assume it never happens

✓ TIPS & BEST PRACTICES

- Use plain `int` for everyday counters; reach for `<stdint.h>` types for file formats and protocols
- Use `unsigned` for bit masks, not for "values that can't be negative"
- Check `sizeof` at compile time with `static_assert(sizeof(long) == 8, "...")`

QUICK SUMMARY

- `char short int long long long` — min sizes only
- `<limits.h>` — real ranges
- `0x 0 0b` — hex, octal, binary (C23)
- Signed overflow = UB; unsigned wraps

Fixed-Width Integers

EXACT-SIZE INTEGERS

`<stdint.h>` (C99) defines integer types with known widths, so code means the same thing on every platform. `int32_t` is **exactly** 32 bits; `int_least32_t` is the smallest type with at least 32; `int_fast32_t` is the fastest with at least 32. It also defines `intptr_t` for holding pointers as integers. `<inttypes.h>` adds the `printf/scanf` macros for these types.

NOTES

- Exact types are optional, but exist on all mainstream platforms
- `size_t` (unsigned) — sizes and indexes; from `<stddef.h>`
- `ptrdiff_t` (signed) — difference of two pointers
- Literal macros: `INT64_C(1)`, `UINT32_C(5)`

USING <STDINT.H>

```c
#include <stdint.h>
#include <inttypes.h>   // PRId64 etc. (includes stdint.h)

uint8_t  byte  = 255;           // 0..255
int16_t  temp  = -40;
uint32_t crc   = 0xDEADBEEF;
int64_t  nanos = INT64_C(1700000000000000000);

size_t   n     = sizeof(int64_t);   // 8
intptr_t addr  = (intptr_t)&temp;   // pointer as integer

byte++;                             // wraps to 0
printf("%" PRIu8 " %" PRId64 "\n", byte, nanos);
printf("%zu\n", n);                 // 8
printf("%" PRIu32 "\n", UINT32_MAX);  // 4294967295
```

CODE EXPLAINED

→ `uint8_t` — exactly 8 bits, unsigned; good for bytes

→ `INT64_C(x)` — makes a literal of the right type

→ `intptr_t` — integer wide enough to round-trip a pointer

→ `PRIu8` / `PRId64` — format strings for these types

TYPE FAMILIES

TYPE	MEANING
`int8_t … int64_t`	exactly N bits, signed
`uint8_t … uint64_t`	exactly N bits, unsigned
`int_least16_t`	smallest with ≥ 16 bits
`int_fast16_t`	fastest with ≥ 16 bits
`intmax_t`	widest signed integer

SPECIAL-PURPOSE

TYPE	HEADER	USE
`size_t`	`stddef.h`	sizes, counts, indexes
`ptrdiff_t`	`stddef.h`	`p - q`
`intptr_t`	`stdint.h`	pointer ↔ integer
`ssize_t`	POSIX	`read()` results (-1 = error)
`SIZE_MAX`	`stdint.h`	largest `size_t`

⚠ COMMON MISTAKES

- Printing `int64_t` with `%ld`: wrong on Windows and macOS. Use `PRId64`
- Looping `for (size_t i = n - 1; i >= 0; i--)`: `size_t` is never negative, so it never stops
- Doing math on `uint8_t` and expecting 8-bit results; operands are promoted to `int` first

✓ TIPS & BEST PRACTICES

- Use `size_t` for anything holding `sizeof` or an array index
- Use exact-width types for binary file formats and network protocols
- Count down safely with `for (size_t i = n; i-- > 0; )`

QUICK SUMMARY

- `int32_t` / `uint64_t` — exact widths
- `size_t` — unsigned size/index type
- `intptr_t` — integer that holds a pointer
- `PRId64` — matching `printf` macro

Floating-Point Types

FLOATING-POINT TYPES

`float`, `double` and `long double` store real numbers in binary floating point (IEEE 754 on almost every platform). `double` gives about 15–16 significant digits and is the default type of literals like `3.14`. Most decimal fractions such as `0.1` can't be stored exactly, so results carry tiny **rounding errors**. Special values `NAN` and `INFINITY` come from `<math.h>`.

NOTES

- `float` ≈ 7 digits; `double` ≈ 15–16
- `3.14` is `double`; `3.14f` is `float`
- Integer division happens first: `1 / 2` is `0`
- Dividing a float by `0.0` gives `inf` or `nan`

FLOATS IN PRACTICE

```c
#include <math.h>
#include <float.h>

float  f = 3.14f;           // f suffix = float
double d = 1e-3;            // 0.001
long double ld = 2.5L;

printf("%.17f\n", 0.1);           // 0.10000000000000001
printf("%d\n", 0.1 + 0.2 == 0.3); // 0  (not equal!)
printf("%f\n", 1.0 / 0.0);        // inf
printf("%d\n", !!isnan(NAN));     // 1
printf("%d\n", NAN == NAN);       // 0
printf("%g\n", DBL_EPSILON);      // 2.22045e-16

double a = 0.1 + 0.2, b = 0.3;
if (fabs(a - b) < 1e-9) puts("close enough");
double avg = (double)7 / 2;       // 3.5, not 3
```

CODE EXPLAINED

- → `%.17f` — shows the stored value isn't exactly 0.1
- → `isnan(x)` — the only reliable NaN test; `NaN != NaN`
- → `DBL_EPSILON` — gap between 1.0 and the next `double`
- → `fabs(a - b) < tol` — compare with a tolerance
- → `(double)7 / 2` — cast first to get real division

TYPES

TYPE	BYTES	DIGITS	LITERAL
`float`	4	~7	`1.5f`
`double`	8	~15	`1.5`
`long double`	8–16	18+	`1.5L`

<FLOAT.H> / <MATH.H>

NAME	MEANING
`DBL_MAX`	≈ `1.8e308`
`DBL_MIN`	smallest normal ≈ `2.2e-308`
`FLT_EPSILON`	≈ `1.19e-7`
`INFINITY / NAN`	special values
`isinf / isfinite`	classify a value

⚠ COMMON MISTAKES

- Comparing with `==` after arithmetic: `0.1 + 0.2 == 0.3` is false. Compare against a tolerance
- `double avg = sum / count;` with two `int`s truncates before converting. Cast one operand
- Using `float` for money. Store cents in an integer (`int64_t`) instead

✓ TIPS & BEST PRACTICES

- Default to `double`; use `float` only for large arrays or GPU/graphics data
- Scale your tolerance: `fabs(a - b) <= 1e-9 * fmax(fabs(a), fabs(b))`
- Link `<math.h>` functions with `-lm` on Linux

QUICK SUMMARY

- `double` — default floating type
- `3.0f` / `3.0` / `3.0L` — literal types
- Never `==` computed floats
- `isnan` / `isinf` — special values

The char Type

THE CHAR TYPE

`char` is a 1-byte integer type that usually holds a character code. A character literal like `'A'` is just a number (65 in ASCII) and in C its type is actually `int`. Whether plain `char` is **signed or unsigned** is implementation-defined: signed on x86, unsigned on ARM Linux. Use `unsigned char` when you mean raw bytes.

NOTES

- `char`, `signed char`, `unsigned char` are 3 distinct types
- `sizeof('A')` is `sizeof(int)` in C (1 in C++)
- `'0'` is 48, not 0; `'7' - '0'` is 7
- `EOF` is an `int` (-1), not a `char`

CHARACTERS AS NUMBERS

```c
char c = 'A';
printf("%c %d\n", c, c);        // A 65
printf("%c\n", c + 1);          // B
printf("%c\n", 'a' - 32);       // A

char digit = '7';
int  value = digit - '0';       // 7

char tab = '\t', nl = '\n', nul = '\0';
char hexA = '\x41';             // 'A'
char quote = '\'';              // single quote

unsigned char byte = 0xFF;      // 255
signed char   sc   = -1;        // always signed
printf("%zu\n", sizeof('A'));   // 4 (it's an int)
```

CODE EXPLAINED

- → `%c` prints the character; `%d` prints its code
- → `digit - '0'` — convert a digit character to its value
- → `'\x41'` — hex escape; `'\101'` octal does the same
- → `unsigned char` — 0..255, safe for byte buffers

ESCAPE SEQUENCES

ESCAPE	MEANING
`\n \t`	newline, tab
`\0`	null character (value 0)
`\\ \' \"`	backslash, quotes
`\r \a \b`	carriage return, bell, backspace
`\xHH \ooo`	hex / octal code

HANDY ASCII CODES

CHAR	DEC	HEX
`'0' - '9'`	48–57	0x30–0x39
`'A' - 'Z'`	65–90	0x41–0x5A
`'a' - 'z'`	97–122	0x61–0x7A
`' ' (space)`	32	0x20
`'\n'`	10	0x0A

IS CHAR SIGNED?

PLATFORM	CHAR
`x86 / x86-64`	signed
`ARM Linux / Android`	unsigned
`Apple arm64`	signed
`MSVC (all)`	signed
`RISC-V, PowerPC Linux`	unsigned

⚠ COMMON MISTAKES

- Storing `getchar()`'s result in a `char`: you can't tell `EOF` from byte 0xFF. Use `int c`
- Assuming `char` is signed: `char c = 200; if (c > 127)` behaves differently on ARM
- Confusing `'a'` (char) with `"a"` (2-byte string `{'a', '\0'}`)

✓ TIPS & BEST PRACTICES

- Use `unsigned char` (or `uint8_t`) for binary data
- Use `<ctype.h>` (`toupper`, `isdigit`) instead of ASCII arithmetic (Sheet 21)
- GCC/Clang: `-fsigned-char` / `-funsigned-char` show how code depends on it

QUICK SUMMARY

- `char` — 1-byte integer for characters
- `'A'` is an `int` (65) in C
- `c - '0'` — digit char → value
- `unsigned char` — raw bytes

Booleans & Truthiness

BOOLEANS & TRUTHINESS

C treats any **zero** value (0, 0.0, NULL) as false and **anything else** as true. C99 added the `_Bool` type and the `<stdbool.h>` header with `bool`, `true` and `false`. In **C23**, `bool`, `true` and `false` became real keywords, so the header is no longer needed. Comparison and logical operators produce an `int` that is `0` or `1`.

NOTES

- Converting any nonzero value to `bool` gives `1`
- `if (ptr)` means `if (ptr != NULL)`
- `!!x` normalizes any value to 0 or 1
- In C23, `true` and `false` have type `bool`

BOOL IN PRACTICE

```c
#include <stdbool.h>        // not needed in C23

bool done = false;
bool ok = 42;               // 1 (any nonzero → true)
int  flag = (3 > 2);        // 1
int  none = (3 < 2);        // 0

if (done) { /* not run */ }
if (!done) puts("working"); // working

int count = 5;
if (count) puts("has items");   // has items

printf("%d\n", !!count);    // 1
printf("%d\n", ok);         // 1  (print bool as int)
printf("%s\n", ok ? "true" : "false");  // true
```

CODE EXPLAINED

- → `bool` — alias for `_Bool` (C99), keyword in C23
- → `bool ok = 42;` — stores `1`, not 42
- → `(3 > 2)` — comparisons evaluate to `int` `1` or `0`
- → `%d` — `bool` has no format; print as int or with `?:`

WHAT COUNTS AS FALSE

VALUE	TRUTH
`0, 0L, 0u`	false
`0.0`	false
`NULL / nullptr`	false
`'\0'`	false
`-1, 42, 0.001, ""`	true ("" is a non-null pointer)

BOOLEAN SUPPORT BY STANDARD

STANDARD	HOW
C89	`int` + `#define TRUE 1`
C99 – C17	`_Bool` + `<stdbool.h>`
C23	`bool` / `true` / `false` keywords

LOGICAL RESULTS

EXPRESSION	VALUE
`!0`	1
`!5`	0
`3 && 0`	0
`2 || 0`	1
`(bool)0.5`	1

⚠ COMMON MISTAKES

- Comparing with `true`: `if (x == true)` fails when `x` is 2. Write `if (x)`
- Assuming `""` or `"0"` is false. They're non-null pointers, so they're true
- Writing your own `#define bool int` in C23 code; `bool` is now a keyword

✓ TIPS & BEST PRACTICES

- Name booleans as questions: `is_empty`, `has_error`
- Return `bool` from predicate functions instead of `int` 0/1
- Keep `#include <stdbool.h>` for code that must build as C99–C17; it's harmless in C23

QUICK SUMMARY

- `0` is false, nonzero is true
- `bool` / `true` / `false` — `<stdbool.h>` or C23
- Comparisons yield `int` 0 or 1
- `!!x` — normalize to 0/1

Sheet 15

Type Conversion & Casting

implicit conversion · integer promotion · usual arithmetic conversions · (int)x · signed vs unsigned comparison trap · truncation · -Wconversion

CONVERSIONS

C converts between arithmetic types automatically. In expressions, small types (`char`, `short`) are **promoted** to `int`, and when two operands differ the **usual arithmetic conversions** pick a common type, preferring floating over integer and wider over narrower. Assignments convert to the target type, which can silently **truncate**. A **cast** `(type)expr` makes a conversion explicit.

NOTES

- Float → int truncates toward zero
- Out-of-range float → int is **UB**
- Out-of-range → unsigned wraps (well-defined)
- Out-of-range → signed is implementation-defined

IMPLICIT & EXPLICIT CONVERSION

```c
int    i = 3.99;              // 3 (truncated)
double d = 7 / 2;             // 3.0 (int division first)
double e = (double)7 / 2;     // 3.5
int    r = (int)(2.7 + 0.5);  // 3 (manual rounding)

unsigned char uc = 300;       // 44 (300 mod 256)
char c = 'a';
printf("%zu\n", sizeof(c + 1));  // 4: char promoted to int

int neg = -1;
unsigned int u = 1;
if (neg < u) puts("less");    // NOT printed!
// -1 converts to unsigned → 4294967295

long long big = 1000000 * 1000000;    // overflow (int math)
long long ok  = 1000000LL * 1000000;  // 1000000000000
```

CODE EXPLAINED

- → `int i = 3.99;` — fraction dropped, no rounding
- → `(double)7 / 2` — cast one operand before dividing
- → `sizeof(c + 1)` — arithmetic on `char` happens in `int`
- → `neg < u` — signed converted to unsigned; `-Wsign-compare` warns
- → `1000000LL *` — make the math wide before it overflows

USUAL ARITHMETIC CONVERSIONS

OPERANDS	RESULT TYPE
`int + double`	double
`int + long`	long
`int + unsigned`	unsigned
`float + double`	double
`char + char`	int (promotion)

COMMON CASTS

CAST	RESULT
`(int)3.99`	3
`(int)-3.99`	−3
`(unsigned)-1`	UINT_MAX
`(double)5 / 2`	2.5
`(uint8_t)257`	1

FLOAT → INTEGER

CALL	-2.5 →
`(int)x`	−2 (truncate)
`lround(x)`	−3 (half away from 0)
`floor(x)`	`-3.0`
`ceil(x)`	`-2.0`
`rint(x)`	`-2.0` (half to even)

⚠ COMMON MISTAKES

- `double avg = total / n;` with `int`s gives a truncated result. Write `(double)total / n`
- Comparing `int` with `size_t` (`if (i < strlen(s))` with negative `i`); the `int` becomes huge
- `long long x = a * b;` with `int a, b`: the multiply overflows before widening

✓ TIPS & BEST PRACTICES

- Enable `-Wconversion -Wsign-conversion` to see every silent narrowing
- Round with `lround(x)` from `<math.h>` instead of adding 0.5
- Cast at the operand, not the result: `(int64_t)a * b`

QUICK SUMMARY

- Small types promote to `int`
- Mixed types convert to the wider/floating one
- `(T)x` — explicit cast
- Signed vs unsigned compares are a trap

const, volatile & sizeof

QUALIFIERS & SIZEOF

Type qualifiers add rules to a type: `const` makes an object read-only, `volatile` tells the compiler every access matters (hardware registers, signal flags), and `restrict` promises a pointer is the only way to reach its data. `sizeof` is an **operator** that gives an object's size in bytes at compile time. C23 adds `typeof` and `constexpr` for type inference and true compile-time constants.

NOTES

- `sizeof` returns `size_t`; print with `%zu`
- `sizeof x` needs no parentheses for objects
- `sizeof` doesn't evaluate its operand (except VLAs)
- `alignof(T)` — required alignment (C11 `_Alignof`)

CONST, VOLATILE, SIZEOF, TYPEOF

```c
const int max = 10;
// max = 11;                    // error: read-only

volatile int ready = 0;         // re-read on every access
const volatile uint32_t *status_reg = (uint32_t *)0x40021000;

int arr[8];
printf("%zu\n", sizeof arr);            // 32
printf("%zu\n", sizeof arr / sizeof arr[0]);  // 8
printf("%zu\n", sizeof(double));        // 8
int i = 0;
size_t s = sizeof(i++);                 // i is still 0

typeof(max) copy = max;         // (C23) const int
constexpr int N = 16;           // (C23) compile-time constant
int buf[N];
```

CODE EXPLAINED

- → `volatile` — no caching in registers; not a thread-safety tool
- → `sizeof arr / sizeof arr[0]` — element count (only for real arrays)
- → `sizeof(i++)` — operand not evaluated
- → `typeof` (C23) — the type of an expression
- → `constexpr` (C23) — usable where a constant expression is required

QUALIFIERS

KEYWORD	MEANING
`const`	can't modify through this name
`volatile`	every read/write really happens
`restrict`	no other pointer aliases this data
`_Atomic`	atomic access (Sheet 93)

CONST VS #DEFINE VS CONSTEXPR

FORM	NOTES
`#define N 16`	text substitution, no type, no scope
`const int N = 16;`	typed; **not** a constant expression in C
`enum { N = 16 };`	typed `int` constant; works pre-C23
`constexpr int N = 16;`	(C23) typed, true constant

SIZEOF ON 64-BIT

EXPRESSION	BYTES
`sizeof(char)`	1 (always)
`sizeof(int)`	4
`sizeof(void *)`	8
`sizeof(int[10])`	40
`sizeof "hi"`	3 (includes `\0`)

⚠ COMMON MISTAKES

- Using `const int N = 10; int a[N];` at file scope. In C, `const` variables aren't constant expressions; use `enum` or `constexpr`
- Using `volatile` for shared data between threads; use `_Atomic` or a mutex
- `sizeof(ptr)` when you meant the array size: inside a function, an array parameter is a pointer

✓ TIPS & BEST PRACTICES

- Mark pointer parameters `const` when the function only reads: `size_t len(const char *s)`
- Allocate with `malloc(n * sizeof *p)` so the size follows `p`'s type
- Use `enum { BUF_SIZE = 256 };` for typed constants in pre-C23 code

QUICK SUMMARY

- `const` — read-only
- `volatile` — don't optimize away accesses
- `sizeof x` — size in bytes, compile time
- `typeof` / `constexpr` — C23 additions

String Basics

char arrays · '\0' terminator · char s[] = "hi" vs char *s = "hi" · literals are read-only · strlen vs sizeof · arrays of strings

C STRINGS

C has no string type. A **string** is an array of `char` ending in a **null terminator** `'\0'`, and most code passes it around as a `char *` pointing at the first character. Every library function finds the end by scanning for `'\0'`, so a missing terminator or a buffer one byte too small breaks everything. String literals live in read-only memory.

NOTES

- `"hi"` occupies 3 bytes: `h`, `i`, `\0`
- `strlen` counts chars before `\0`; O(n)
- Arrays can't be assigned: use `strcpy` / `snprintf`
- Adjacent literals join: `"ab" "cd"` → `"abcd"`

ARRAYS VS POINTERS

```c
#include <string.h>

char a[] = "hello";       // array copy: 6 bytes, writable
char *p  = "hello";       // pointer to read-only literal
const char *q = "hello";  // better: say it's read-only

a[0] = 'H';               // OK → "Hello"
// p[0] = 'H';            // UB: writing to a literal

printf("%zu %zu\n", strlen(a), sizeof a);   // 5 6
printf("%zu\n", sizeof p);                  // 8 (pointer size)

char buf[16] = "hi";      // rest filled with '\0'
char name[4] = {'B', 'o', 'b', '\0'};

const char *days[] = {"Mon", "Tue", "Wed"};  // array of strings
printf("%s %c\n", days[1], days[1][0]);    // Tue T
```

CODE EXPLAINED

- → `char a[]` — array sized by the initializer, contents copied
- → `char *p = "..."` — points at a literal; modifying it is UB
- → `strlen` vs `sizeof` — length vs storage (incl. `\0`)
- → `const char *days[]` — array of pointers to literals

DECLARATION FORMS

CODE	STORAGE	WRITABLE?
`char s[] = "hi";`	3-byte array	yes
`char s[10] = "hi";`	10 bytes, zero-filled	yes
`const char *s = "hi";`	pointer → literal	no
`char *s = malloc(n);`	heap	yes (free it)

BASIC OPERATIONS

TASK	USE
length	`strlen(s)`
copy	`snprintf(dst, sizeof dst, "%s", s)`
compare	`strcmp(a, b) == 0`
empty?	`s[0] == '\0'`
print	`printf("%s", s)`

⚠ COMMON MISTAKES

- `char buf[5] = "hello";` compiles in C but leaves no room for `\0`; later calls read past the end
- Comparing strings with `==`: it compares addresses. Use `strcmp(a, b) == 0`
- Writing through `char *p = "text"`; declare it `const char *` so the compiler stops you

✓ TIPS & BEST PRACTICES

- Size buffers as "max characters + 1"
- Cache `strlen(s)` outside loops: `for (size_t i = 0, n = strlen(s); i < n; i++)`
- Compile with `-Wwrite-strings` (GCC) to make literals `const char[]`

QUICK SUMMARY

- String = `char` array + `'\0'`
- `strlen` — chars before `\0`
- `sizeof` — bytes of the array
- Literals are read-only: `const char *`

Copying & Concatenating

COPYING & JOINING

Copying and concatenating strings means moving bytes into a buffer **you** sized. The classic functions `strcpy` and `strcat` don't know the destination size and will overflow it. The bounded versions have traps too: `strncpy` may leave the result unterminated. For most code, `snprintf` is the safest single tool: it always terminates and tells you whether the output was truncated.

NOTES

- `snprintf` returns the length it **wanted** to write
- `strdup` / `strndup` allocate; caller must `free`
- `strdup` is standard in **C23** (POSIX before)
- `strlcpy` / `strlcat` — BSD, macOS, glibc 2.38+

SAFE COPY & CONCATENATE

```c
#include <string.h>
#include <stdlib.h>
char dst[16];

strcpy(dst, "short");                   // fine if it fits
snprintf(dst, sizeof dst, "%s", src);   // always terminated

int n = snprintf(dst, sizeof dst, "%s-%d", name, id);
if (n < 0 || (size_t)n >= sizeof dst) { /* truncated */ }

size_t len = strlen(dst);               // append safely
snprintf(dst + len, sizeof dst - len, "%s", ".txt");

strncpy(dst, src, sizeof dst - 1);      // may not terminate,
dst[sizeof dst - 1] = '\0';             // so do it yourself

char *copy = strdup(src);               // malloc'd copy
free(copy);
```

CODE EXPLAINED

- → `snprintf(dst, size, ...)` — writes at most `size-1` chars + `\0`
- → `n >= sizeof dst` — output was truncated
- → `dst + len` — append by printing at the current end
- → `strncpy` — pads with zeros, no `\0` if `src` is too long
- → `strdup` — returns heap memory you must `free`

COPY FUNCTIONS

FUNCTION	BOUNDED?	TERMINATES?
`strcpy(d, s)`	no	yes
`strncpy(d, s, n)`	yes	**not if** `strlen(s) >= n`
`strlcpy(d, s, n)`	yes	yes (non-standard)
`snprintf(d, n, "%s", s)`	yes	yes
`memcpy(d, s, n)`	yes	copies bytes only

CONCATENATE FUNCTIONS

FUNCTION	NOTES
`strcat(d, s)`	no bound; overflow risk
`strncat(d, s, n)`	n = max chars **from s**, not buffer size
`strlcat(d, s, n)`	n = full buffer size
`snprintf(d + len, ...)`	portable bounded append

⚠ COMMON MISTAKES

- `strncat(dst, src, sizeof dst)` : the limit is characters to append, not buffer size, so it can still overflow
- Assuming `strncpy` terminates: it doesn't when the source fills the buffer
- Overlapping source and destination with `strcpy` / `memcpy` ; that's UB. Use `memmove`

✓ TIPS & BEST PRACTICES

- Prefer `snprintf` for building strings and check its return value
- Track lengths yourself (`len` variable) to avoid repeated `strlen` scans
- Turn on `-D_FORTIFY_SOURCE=2 -O2` to add runtime overflow checks to these calls

QUICK SUMMARY

- `snprintf` — bounded, always terminated
- `strcpy` / `strcat` — only when size is proven
- `strncpy` — may not terminate
- `strdup` — heap copy; `free` it

Comparing & Searching

COMPARING & SEARCHING

`<string.h>` compares and searches null-terminated strings. `strcmp` returns **negative, zero or positive** (not just -1/0/1) depending on byte order. The search functions return a **pointer** into the original string, or `NULL` when nothing is found, so the position is `p - s`. `strtok` splits a string by modifying it in place, which makes it unsafe for literals and for nested or threaded use.

NOTES

- `strcmp` compares bytes; not locale-aware
- `strcasecmp` (POSIX) ignores case; `_stricmp` on MSVC
- `strtok_r` (POSIX) / `strtok_s` are re-entrant
- Always check search results for `NULL`

COMPARE, FIND, SPLIT

```c
#include <string.h>

strcmp("apple", "apple");    // 0
strcmp("apple", "banana");   // < 0
strncmp("prefix_x", "prefix", 6);  // 0

const char *s = "key=value;x=1";
char *eq  = strchr(s, '=');  // first '=' → "=value;x=1"
char *lst = strrchr(s, '='); // last  '=' → "=1"
char *sub = strstr(s, "val");// "value;x=1"
size_t pos = (size_t)(eq - s);       // 3
size_t n = strcspn(s, ";=");         // 3 (chars before ; or =)

char line[] = "a,b,,c";              // must be writable
for (char *t = strtok(line, ","); t; t = strtok(NULL, ","))
    printf("[%s]", t);               // [a][b][c]
```

CODE EXPLAINED

- → `strcmp(a, b) == 0` — equality test
- → `strchr` / `strrchr` — first / last occurrence of a char
- → `eq - s` — pointer difference gives the index
- → `strcspn` — length of prefix with none of the given chars
- → `strtok` — skips empty fields and writes `\0` into `line`

COMPARE

FUNCTION	RETURNS
`strcmp(a, b)`	<0, 0, >0
`strncmp(a, b, n)`	first n chars only
`strcasecmp(a, b)`	case-insensitive (POSIX)
`memcmp(a, b, n)`	raw bytes, ignores `\0`

SEARCH

FUNCTION	FINDS
`strchr(s, c)`	first char c
`strrchr(s, c)`	last char c
`strstr(s, t)`	substring t
`strpbrk(s, set)`	first char from set
`strspn(s, set)`	length of prefix **in** set
`strcspn(s, set)`	length of prefix **not in** set

⚠ COMMON MISTAKES

- `if (strcmp(a, b))` reads as "if equal" but means "if different"; write `== 0` or `!= 0`
- Calling `strtok` on a string literal: it writes into it, which is UB
- Using the result of `strchr` without checking for `NULL`

✓ TIPS & BEST PRACTICES

- Strip a trailing newline: `s[strcspn(s, "\n")] = '\0';`
- Check a prefix: `strncmp(s, "http", 4) == 0`
- Use `strsep` (BSD/glibc) or manual `strchr` loops when you need empty fields

QUICK SUMMARY

- `strcmp(a, b) == 0` — equal
- `strchr` / `strstr` — return pointer or `NULL`
- `p - s` — index of a match
- `strtok` — modifies input; not re-entrant

String ↔ Number Conversion

atoi (and why to avoid it) · strtol / strtoul / strtoll · strtod · endptr + errno checks · snprintf · sscanf

CONVERTING TEXT AND NUMBERS

To parse numbers from text, use the `strto*` family: they report **where parsing stopped** through an end pointer and signal overflow through `errno`. The older `atoi` and `atof` can't report errors at all: `atoi("abc")` and `atoi("0")` both return 0. To go the other way, format with `snprintf`.

NOTES

- Set `errno = 0` **before** calling `strtol`
- `base` 0 auto-detects `0x` and leading `0`
- `strtol` skips leading whitespace
- `sscanf` returns how many fields matched

ROBUST PARSING

```c
#include <stdlib.h>
#include <errno.h>

bool parse_int(const char *s, int *out) {
    char *end;
    errno = 0;
    long v = strtol(s, &end, 10);
    if (end == s || *end != '\0') return false;   // no digits / junk
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX) return false;
    *out = (int)v;
    return true;
}

strtol("0x1F", NULL, 16);      // 31
strtol("0x1F", NULL, 0);       // 31 (auto base)
strtod("3.5e2", NULL);         // 350.0
char buf[32]; snprintf(buf, sizeof buf, "%d", 42);  // "42"
```

CODE EXPLAINED

→ `end == s` — no digits were read

→ `*end != '\0'` — trailing characters after the number

→ `errno == ERANGE` — value didn't fit in `long`

→ range check — `long` may be wider than `int`

→ `snprintf` — number → string

STRTO* FAMILY

FUNCTION	RETURNS
`strtol / strtoll`	`long / long long`
`strtoul / strtoull`	unsigned (accepts –!)
`strtod / strtof`	`double / float`
`strtoimax`	`intmax_t (<inttypes.h>)`

STRTOL(S, &END, 10)

INPUT	VALUE	*END
`"42"`	42	`'\0'`
`" 42abc"`	42	`'a'`
`"abc"`	0	`'a'` (end == s)
`"99999999999999999999"`	LONG_MAX	ERANGE

⚠ COMMON MISTAKES

- Using `atoi` on user input: no error detection, and overflow is UB
- Forgetting `errno = 0` first; an old `ERANGE` from earlier code looks like a new error
- `sscanf(s, "%d", &n)` without checking it returned 1

✓ TIPS & BEST PRACTICES

- Wrap `strtol` in a helper like `parse_int` and use it everywhere
- Use `strtoul` with care: `"-1"` parses to `ULONG_MAX`; reject a leading `-` yourself
- For a whole line, read with `fgets`, then parse with `strtol` or `sscanf`

QUICK SUMMARY

- `strtol(s, &end, base)` — parse with error info
- Check `end` and `errno`
- Avoid `atoi` / `atof` for input
- `snprintf` — number to text

Character Classes & Unicode

CHARACTERS & UNICODE

<ctype.h> classifies and converts single characters (`isdigit`, `toupper`, …). Their argument must be an `unsigned char` value or EOF; passing a negative `char` is UB. C strings are just bytes, so **UTF-8** text works with most string functions, but one character may take 1–4 bytes. Wide (`wchar_t`) and `char16_t`/`char32_t` types exist for other encodings.

NOTES

- `ctype` functions honor the current locale
- `wchar_t` is 32-bit on Unix, 16-bit on Windows
- `strlen("é")` is 2 in UTF-8
- `u8""` is `char8_t[]` in C23

CTYPE & UNICODE LITERALS

```c
#include <ctype.h>
#include <locale.h>

char s[] = "Hello, World 42";
int letters = 0;
for (char *p = s; *p; p++) {
    unsigned char c = (unsigned char)*p;   // required cast
    if (isalpha(c)) letters++;
    *p = (char)toupper(c);                 // "HELLO, WORLD 42"
}

setlocale(LC_ALL, "");          // use the user's locale
wchar_t  *w  = L"wide";         // wchar_t[]
char     *u8 = u8"héllo";       // UTF-8 (char8_t[] in C23)
char16_t *u  = u"utf16";        // <uchar.h>
char32_t *U  = U"utf32";
printf("%zu\n", strlen("héllo"));  // 6 bytes, 5 characters
```

CODE EXPLAINED

- → `(unsigned char)*p` — avoids UB for bytes ≥ 0x80
- → `toupper` — returns the converted `int`; cast back to `char`
- → `setlocale` — locale-dependent functions start in `"C"`
- → `L` / `u8` / `u` / `U` — literal prefixes
- → UTF-8 `é` = 2 bytes

<CTYPE.H> TESTS

FUNCTION	TRUE FOR
`isalpha / isdigit`	letters / `0–9`
`isalnum`	letter or digit
`isspace`	space `\t \n \v \f \r`
`isupper / islower`	case
`isxdigit`	`0–9a–fA–F`
`ispunct / isprint`	punctuation / printable

LITERAL PREFIXES

PREFIX	TYPE (C23)
`"..."`	`char[]`
`L"..."`	`wchar_t[]`
`u8"..."`	`char8_t[]` (was `char[]`)
`u"..."`	`char16_t[]`
`U"..."`	`char32_t[]`

⚠ COMMON MISTAKES

- `isdigit(c)` with a plain `char` holding `é` (negative): UB. Cast to `unsigned char` first
- Uppercasing UTF-8 byte by byte; `toupper` only handles single-byte characters
- Assuming `strlen` counts characters; it counts bytes

✓ TIPS & BEST PRACTICES

- Keep text in UTF-8 internally; convert only at OS boundaries (e.g. Windows APIs)
- Use a library like ICU or utf8proc for case mapping and normalization
- Count UTF-8 code points by skipping continuation bytes: `(c & 0xC0) != 0x80`

QUICK SUMMARY

- `isalpha(c)`, `toupper(c)` — pass `unsigned char`
- UTF-8 works as bytes; length ≠ characters
- `L u8 u U` — literal prefixes
- `setlocale(LC_ALL, "")` — user locale

Arithmetic Operators

ARITHMETIC OPERATORS

C has the usual `+ - * / %` plus increment and decrement. With integers, `/` **truncates toward zero** and `%` takes the sign of the left operand. **Signed overflow is undefined behavior**, so the compiler may assume it never happens; **unsigned** arithmetic wraps around. C23's `<stdckdint.h>` adds `ckd_add`, `ckd_sub` and `ckd_mul`, which report overflow safely.

NOTES

- `(a / b) * b + a % b == a` always holds
- Dividing by zero is UB for integers
- `%` works only on integers; use `fmod` for floats
- `INT_MIN / -1` overflows (UB)

INTEGER ARITHMETIC

```
printf("%d\n", 7 / 2);       // 3
printf("%d\n", -7 / 2);      // -3 (toward zero)
printf("%d\n", 7 % 3);       // 1
printf("%d\n", -7 % 3);      // -1 (sign of left)
printf("%f\n", 7 / 2.0);     // 3.500000

int i = 5;
int a = i++;                 // a = 5, i = 6
int b = ++i;                 // b = 7, i = 7

unsigned u = 0;
u--;                         // UINT_MAX (wraps, defined)
int big = INT_MAX;
// big + 1                   // UB!

#include <stdckdint.h>       // (C23)
int sum;
if (ckd_add(&sum, big, 1)) puts("overflow");  // overflow
```

CODE EXPLAINED

- → `7 / 2` — integer division drops the fraction
- → `-7 % 3` — `-1`, not `2`
- → `i++` / `++i` — value before / after increment
- → `u--` from 0 — unsigned wrap is well-defined
- → `ckd_add(&r, a, b)` — returns `true` if the result overflowed

OPERATORS

OP	NAME	EXAMPLE
`+ -`	add, subtract	5 − 8 → −3
`*`	multiply	6 * 7 → 42
`/`	divide	9 / 4 → 2
`%`	remainder	9 % 4 → 1
`++ --`	increment, decrement	i++
`- (unary)`	negate	−x

OVERFLOW BEHAVIOR

CASE	RESULT
`signed + - * overflow`	**UB**
`unsigned overflow`	wraps mod 2^n
`x / 0, x % 0`	**UB**
`1.0 / 0.0`	`inf`
`ckd_add (C23)`	reports via return value

⚠ COMMON MISTAKES

- Checking overflow after it happens: `if (a + b < a)` is UB for `int`. Check first or use `ckd_add` / `__builtin_add_overflow`
- Using `%` to test oddness with negatives: `-3 % 2 == -1`, so `x % 2 == 1` misses it; use `x % 2 != 0`
- `average = (a + b) / 2` can overflow; use `a + (b - a) / 2`

✓ TIPS & BEST PRACTICES

- Pre-C23, GCC/Clang offer `__builtin_add_overflow(a, b, &r)`
- Build tests with `-fsanitize=undefined` to catch signed overflow at runtime
- Use `-fwrapv` only if you truly need wrapping signed math; it hides bugs

QUICK SUMMARY

- `/` truncates toward zero
- `%` takes the left operand's sign
- Signed overflow = UB; unsigned wraps
- `ckd_add` (C23) — checked arithmetic

Sheet 23

Comparison & Logical Operators

== != < > <= >= · && || ! · short-circuit evaluation · results are 0 / 1 · = vs == bug · pointer comparisons

COMPARING & COMBINING

Relational operators (< > <= >=) and equality operators (== !=) produce an `int`: `1` for true, `0` for false. The logical operators &&, || and ! treat any nonzero operand as true and **short-circuit**: the right side runs only if the left side doesn't already decide the result. That guarantee makes guards like `p && p->next` safe.

NOTES

- `&&` and `||` are sequence points
- `&&` binds tighter than `||`
- Chained `a < b < c` is not a range check
- Pointers compare with `==` or to `NULL`

COMPARISONS & SHORT-CIRCUIT

```c
int x = 5, y = 10;
printf("%d %d\n", x < y, x == y);   // 1 0
printf("%d\n", !x);                 // 0
printf("%d\n", x && 0);             // 0
printf("%d\n", 0 || y);             // 1

if (p != NULL && p->count > 0)      // safe: right side skipped if p is NULL
    use(p);

if (1 < x < 3) puts("??");          // always true! (1<x) is 0 or 1, < 3
if (1 < x && x < 3) puts("in range");

if (x = 0) { }                      // assignment, always false (bug)
if (x == 0) { }                     // comparison

int found = count > 0 || load();    // load() runs only if count <= 0
```

CODE EXPLAINED

→ `x < y` — evaluates to `int` 1 or 0

→ `p != NULL && ...` — `&&` stops at the first false operand

→ `1 < x < 3` — parsed as `(1 < x) < 3`

→ `x = 0` — assigns; `-Wall` warns "suggest parentheses"

→ `||` — stops at the first true operand

COMPARISON

OP	MEANING	5 ? 3
==	equal	0
!=	not equal	1
< <=	less (or equal)	0
> >=	greater (or equal)	1

LOGICAL

OP	RESULT	SHORT-CIRCUITS
a && b	1 if both nonzero	skips b if a is 0
a || b	1 if either nonzero	skips b if a is nonzero
!a	1 if a is 0	—

LOOK-ALIKES

WRITE	NOT
a == b	a = b
a && b	a & b (bitwise)
a || b	a | b (bitwise)
strcmp(a,b) == 0	a == b for strings

⚠ COMMON MISTAKES

- `if (x = 5)` assigns and is always true. Compile with `-Wall`; some people write `5 == x`
- Using `&` instead of `&&`: `if (n & valid())` doesn't short-circuit and does bit math
- Comparing floats or strings with `==` (Sheets 12, 19)

✓ TIPS & BEST PRACTICES

- Put cheap or guarding tests first in `&&` chains
- Wrap an intentional assignment in extra parens: `if ((n = read()) > 0)`
- Compare pointers to `NULL` explicitly when it helps readability

QUICK SUMMARY

- `== != < > <= >=` → `int` 0/1
- `&& || !` — logical, short-circuit
- `a < b < c` is not a range test
- `=` assigns, `==` compares

Bitwise Operators

BITWISE OPERATORS

Bitwise operators work on the individual bits of integers: AND &, OR |, XOR ^, NOT ~, and shifts << >>. They're used for flags, masks, hardware registers and compact encodings. Use **unsigned** types: shifting a negative value left, or shifting by the type's width or more, is undefined behavior. C23's `<stdbit.h>` adds bit-counting helpers.

NOTES

- `x << n` = `x * 2ⁿ` (if no overflow)
- `>>` on negative signed is implementation-defined
- `~` flips **all** bits of the promoted type
- Bitwise ops bind looser than `==`

MASKS & SHIFTS

```c
unsigned a = 0xC;   // 1100
unsigned b = 0xA;   // 1010
a & b;              // 1000  (8)
a | b;              // 1110  (14)
a ^ b;              // 0110  (6)
~a;                 // 0xFFFFFFF3
1u << 4;            // 16
0x80u >> 3;         // 16

unsigned flags = 0;
flags |=  (1u << 3);          // set bit 3
flags &= ~(1u << 3);          // clear bit 3
flags ^=  (1u << 3);          // toggle bit 3
if (flags & (1u << 3)) { }    // test bit 3

if ((flags & MASK) == MASK) { }   // parentheses required!
#include <stdbit.h>                 // (C23)
stdc_count_ones(0xF0u);             // 4
```

CODE EXPLAINED

- → `|=` with a shifted 1 — set a bit
- → `&= ~mask` — clear the bits in `mask`
- → `^=` — toggle
- → `(flags & MASK) == MASK` — `==` binds tighter than `&`
- → `stdc_count_ones` (C23) — population count

OPERATORS

OP	NAME	5 OP 3 (101, 011)
&	AND	1 (001)
|	OR	7 (111)
^	XOR	6 (110)
~	NOT	~5u = 0xFFFFFFFA
<<	left shift	5 << 1 = 10
>>	right shift	5 >> 1 = 2

IDIOMS

EXPRESSION	DOES
`x & (x - 1)`	clear lowest set bit
`x & -x`	isolate lowest set bit
`(x & (x-1)) == 0`	power of two (x > 0)
`(x >> n) & 1u`	read bit n
`x & 0xFFu`	low byte

<STDBIT.H> (C23)

FUNCTION	RETURNS
`stdc_count_ones`	number of 1 bits
`stdc_leading_zeros`	leading 0 bits
`stdc_trailing_zeros`	trailing 0 bits
`stdc_bit_width`	bits needed
`stdc_has_single_bit`	is power of 2

⚠ COMMON MISTAKES

- `1 << 31` on a 32-bit `int` is UB (shifts into the sign bit). Write `1u << 31`
- `if (flags & MASK == MASK)` parses as `flags & (MASK == MASK)`; add parentheses
- `~` on a `uint8_t` gives an `int` with high bits set: `(uint8_t)~x` to keep 8 bits

✓ TIPS & BEST PRACTICES

- Name bits with an `enum`: `enum { F_READ = 1u << 0, F_WRITE = 1u << 1 };`
- Use `uint32_t` / `uint64_t` for bit fields in protocols so widths are fixed
- See the separate **Bitwise** set for deeper tricks

QUICK SUMMARY

- `& | ^ ~ << >>` — bit operations
- `x |= bit`, `x &= ~bit`, `x ^= bit`
- Use unsigned operands
- Parenthesize `&` in comparisons

Assignment, Comma & Other Operators

ASSIGNMENT & FRIENDS

In C, assignment is an **expression**: `x = 5` has the value 5, which is why `a = b = 0` and `while ((c = getchar()) != EOF)` work. Compound operators like += combine an operation with assignment. The **comma operator** evaluates left to right and yields the right value. Casts, member access (`. ->`), address-of & and dereference ∗ round out the operator set.

NOTES

- Assignment is right-associative
- `x op= y` evaluates `x` only once
- Commas in calls/declarations are separators, not operators
- `&` and `*` are covered in Sheet 44

ASSIGNMENT EXPRESSIONS

```
int a, b, c;
a = b = c = 0;              // right to left: c=0, b=0, a=0

int x = 10;
x += 5;   // 15      x -= 3;   // 12
x *= 2;   // 24      x /= 5;   // 4
x %= 3;   // 1       x <<= 4;  // 16
x |= 1;   // 17      x &= 0xF; // 1

int ch;
while ((ch = getchar()) != EOF)   // assign, then compare
    putchar(ch);

for (int i = 0, j = 9; i < j; i++, j--) { }  // comma operator
int r = (x++, x * 2);       // r = 4, x = 2

struct point pt = {1, 2}, *pp = &pt;
pt.x = 5;   pp->y = 7;      // member access
double d = (double)x / 3;   // cast
```

CODE EXPLAINED

- → `a = b = c = 0` — value of each assignment feeds the next
- → `x op= y` — same as `x = x op y`
- → `(ch = getchar()) != EOF` — parentheses required
- → `i++, j--` — comma operator in the loop step
- → `pp->y` — shorthand for `(*pp).y`

COMPOUND ASSIGNMENT

OP	SAME AS
`x += y`	`x = x + y`
`x -= y x *= y`	subtract / multiply
`x /= y x %= y`	divide / remainder
`x <<= n x >>= n`	shift
`x &= m x |= m x ^= m`	bitwise

OTHER OPERATORS

OP	MEANING
`(type)x`	cast
`s.m p->m`	member / via pointer
`&x *p`	address-of / dereference
`sizeof x`	size in bytes
`a, b`	evaluate a, then b; value of b
`c ? a : b`	conditional (Sheet 28)

⚠ COMMON MISTAKES

- `while (ch = getchar() != EOF)` : `!=` binds first, so `ch` becomes 0 or 1
- `x =+ 5` is `x = +5`, not `x += 5`
- `int a, b = 0;` initializes only `b`

✓ TIPS & BEST PRACTICES

- Keep side effects to one per statement; chained assignments are fine, `a[i] = i++` is not
- Use the comma operator only in `for` headers
- Prefer `p->m` over `(*p).m`

QUICK SUMMARY

- Assignment is an expression with a value
- `+= -= *= /= %= <<= >>= &= |= ^=`
- `,` — evaluate left, return right
- `.` / `->` — member access

Sheet 26

Precedence & Evaluation Order

precedence table · associativity · unsequenced i = i++ (UB) · argument evaluation order unspecified · *p++ vs (*p)++ · when to parenthesize

PRECEDENCE & ORDER

Precedence decides how an expression is grouped (`a + b * c` is `a + (b * c)`). **Evaluation order** is a separate question, and C mostly leaves it **unspecified**: in `f() + g()` either call may run first. Modifying the same variable twice without a **sequence point** between, as in `i = i++`, is undefined behavior. When in doubt, add parentheses and split statements.

SEQUENCE POINTS / ORDERING

- Full expression end (`;`)
- `&&` , `||` , `?:` and comma operator
- After evaluating all function arguments, before the call
- Function args: order unspecified

GROUPING VS ORDER

```c
int r = 2 + 3 * 4;          // 14
int s = (2 + 3) * 4;        // 20
int m = 1 << 2 + 1;         // 8: + binds tighter than <<

int *p = arr;
*p++;                       // *(p++): moves p, reads old element
(*p)++;                     // increments the element

int i = 1;
i = i++;                    // UB
a[i] = i++;                 // UB
printf("%d %d\n", i, i++);  // UB

int x = f() + g();          // f or g may run first
int t1 = f(); int t2 = g(); // fixed order when it matters
int y = t1 + t2;
```

CODE EXPLAINED

- → `*` before `+` — standard math precedence
- → `1 << 2 + 1` — shifts are below arithmetic
- → `*p++` — postfix `++` binds tighter than unary `*`
- → `i = i++` — two unsequenced writes to `i`
- → Separate statements to force an order

PRECEDENCE: HIGHEST

LEVEL	OPERATORS	ASSOC.
1	`() [] -> .` postfix `++ --`	→
2	prefix `++ -- + - ! ~ * & (type) sizeof`	←
3	`* / %`	→
4	`+ -`	→
5	`<< >>`	→
6	`< <= > >=`	→

PRECEDENCE: LOWEST

LEVEL	OPERATORS	ASSOC.
7	`== !=`	→
8–10	& then ^ then |	→
11–12	&& then ||	→
13	`?:`	←
14	`= += -= …`	←
15	`,`	→

⚠ COMMON MISTAKES

- `if (x & 1 == 0)` means `x & (1 == 0)` ; write `(x & 1) == 0`
- `*p++ = 0` vs `(*p)++` : one moves the pointer, the other changes the value
- Relying on argument order: `f(i++, i++)` is UB

✓ TIPS & BEST PRACTICES

- Parenthesize any mix of bitwise, shift and comparison operators
- Use `-Wsequence-point` (in `-Wall`) and `-Wparentheses`
- If a line needs a precedence table to read, split it

QUICK SUMMARY

- Precedence groups; it doesn't order evaluation
- Unary binds tighter than binary; `,` loosest
- Two unsequenced writes = UB
- When unsure, parenthesize

if / else if / else

IF / ELSE IF / ELSE

`if` runs a statement when its condition is nonzero. Chains of `else if` test conditions in order and stop at the first true one; a final `else` catches everything else. Without braces only **one statement** belongs to each branch, and an `else` always attaches to the **nearest** unmatched `if` (the dangling-else rule), so always using braces avoids a whole class of bugs.

NOTES

- Condition can be any scalar: int, float, pointer
- `if (ptr)` ⇔ `if (ptr != NULL)`
- A stray `;` after `if (...)` ends the statement
- No `elif` keyword; write `else if`

BRANCHING

```c
int score = 82;
char grade;
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';                  // this one runs
} else if (score >= 70) {
    grade = 'C';
} else {
    grade = 'F';
}

if (ptr) process(ptr);            // non-NULL

if ((n = read_count()) < 0)       // assign + test (extra parens)
    return -1;

if (x > 0);                       // BUG: empty statement
    puts("always prints");
```

CODE EXPLAINED

- → `else if` — tested only when earlier conditions fail
- → Braces — make each branch a block
- → `if (ptr)` — idiomatic null check
- → `((n = f()) < 0)` — the extra parens show the assignment is intended
- → `if (x > 0);` — the `;` is the whole body

FORMS

FORM	USE
`if (c) stmt;`	single action
`if (c) {...} else {...}`	two-way
`if / else if / else`	multi-way on ranges
`c ? a : b`	choose a value (Sheet 28)
`switch`	many fixed integer values (Sheet 29)

DANGLING ELSE

CODE	ELSE BELONGS TO
`if (a) if (b) x(); else y();`	`if (b)`
`if (a) { if (b) x(); } else y();`	`if (a)`

⚠ COMMON MISTAKES

- Adding a second statement to a brace-less `if`: only the first is conditional (Apple's `goto fail` bug)
- Testing a range with `if (0 < x < 10)`; write `if (x > 0 && x < 10)`
- Checking the same variable with `if` / `if` instead of `if` / `else if`, so multiple branches run

✓ TIPS & BEST PRACTICES

- Always use braces, even for one line; `-Wmisleading-indentation` catches some cases
- Return early on errors to keep the main path un-indented
- Order `else if` tests from most to least specific

QUICK SUMMARY

- `if / else if / else` — first true branch wins
- Nonzero is true
- `else` binds to nearest `if`
- Always use braces

Ternary Operator

cond ? a : b · result type rules · nesting (and when not to) · ternary in initializers · ternary inside macros · vs if

THE CONDITIONAL OPERATOR

`cond ? a : b` evaluates `cond` and then **only one** of `a` or `b`, producing a value. It's the expression form of `if/else`, handy for initializing variables, choosing arguments and writing macros. Both branches are converted to a common type, so mixing `int` and `double` gives a `double`. Nested ternaries quickly get hard to read.

NOTES

- Only the chosen branch is evaluated
- Right-associative: `a ? b : c ? d : e`
- Lower precedence than most operators
- Not an lvalue in C (it is in C++)

CHOOSING VALUES

```c
int a = 7, b = 3;
int max = (a > b) ? a : b;                // 7
const char *word = (n == 1) ? "item" : "items";
printf("%d %s\n", n, word);

printf("%s\n", ok ? "pass" : "fail");
int abs_x = x < 0 ? -x : x;

// Common type: int and double → double
double v = flag ? 1 : 2.5;                // 1.0 or 2.5

// Nested: readable only when laid out like a table
const char *size = n < 10   ? "small"
                 : n < 100  ? "medium"
                 :            "large";

#define MIN(x, y) ((x) < (y) ? (x) : (y))
```

CODE EXPLAINED

→ `(a > b) ? a : b` — parentheses around the condition aid reading

→ Inline in calls — no temporary needed

→ `flag ? 1 : 2.5` — both arms become `double`

→ Aligned nesting — reads like an `else if` chain

→ `MIN` macro — evaluates the winner twice (Sheet 77)

TERNARY VS IF

USE TERNARY WHEN	USE IF WHEN
choosing a value	running statements
initializing a const	branches have side effects
one level deep	logic is nested
both arms are short	arms are long

RESULT TYPE

ARMS	RESULT
int, int	int
int, double	double
char *, NULL	char *
void, void	void (statement use)

⚠ COMMON MISTAKES

- `x = cond ? a : b + 1;` adds 1 only to `b`; parenthesize: `(cond ? a : b) + 1`
- Putting side effects in both arms and expecting both to run
- Mixing signed and unsigned arms: `c ? -1 : 1u` gives `UINT_MAX`, not -1

✓ TIPS & BEST PRACTICES

- Great for `const` variables that need a computed value
- Use for plural words and "yes/no" in output
- Stop at one level of nesting unless formatted as a table

QUICK SUMMARY

- `cond ? a : b` — pick a value
- Only one arm is evaluated
- Arms convert to a common type
- Keep it short and shallow

switch

SWITCH

`switch` jumps to the `case` label matching an **integer** expression (including `char` and `enum`). Execution then **falls through** into the following cases until a `break`, so every case normally ends with `break`. `default` handles anything unmatched. Case labels must be compile-time integer constants; strings and ranges aren't allowed.

NOTES

- No strings: use `strcmp` + `if` chains
- Labels must be unique constants
- `default` can go anywhere
- Declare variables inside a `{ }` block

SWITCH WITH FALLTHROUGH

```c
switch (cmd) {
case 'q':
case 'Q':                       // grouped labels
    quit = true;
    break;
case 'h': {                     // braces allow declarations
    int lines = show_help();
    printf("%d lines\n", lines);
    break;
}
case 'v':
    verbose++;
    [[fallthrough]];            // (C23) intentional
case 'r':
    run();
    break;
default:
    fprintf(stderr, "unknown: %c\n", cmd);
}
```

CODE EXPLAINED

- → `case 'q': case 'Q':` — several labels share one body
- → `break` — leave the switch
- → `{ int lines ... }` — scope for case-local variables
- → `[[fallthrough]];` — silences `-Wimplicit-fallthrough`
- → `default` — runs when no label matches

RULES

ALLOWED	NOT ALLOWED
`case 3:`	`case x:` (variable)
`case 'a':`	`case "add":`
`case RED:` (enum)	`case 1.5:`
`case 1 + 2:`	duplicate labels

SWITCH VS IF-ELSE

SWITCH	IF / ELSE IF
equality with constants	ranges, any condition
may compile to a jump table	tests in order
enum coverage warnings	works with strings via `strcmp`

GNU EXTENSION

SYNTAX	NOTE
`case 1 ... 5:`	range label (GCC/Clang)
	not ISO C; use `-Wpedantic` to flag

⚠ COMMON MISTAKES

- Forgetting `break`, so the next case runs too. Enable `-Wimplicit-fallthrough`
- Declaring `int x = 1;` right after a `case` label: before C23 that's a syntax error, and other cases see `x` uninitialized. Add braces
- Adding `default:` to an `enum` switch hides warnings when new enum values are added

✓ TIPS & BEST PRACTICES

- Mark deliberate fallthrough with `[[fallthrough]];` or a `/* fallthrough */` comment
- For `enum` switches, omit `default` and handle every value so `-Wswitch` helps you
- Put the most common cases first only for readability; order doesn't affect speed

QUICK SUMMARY

- `switch (int)` — jump to matching `case`
- `break` — stop fallthrough
- `default` — unmatched values
- Labels: integer constants only

for Loops

FOR LOOPS

`for (init; condition; step) body` puts loop setup, test and update in one line. The condition is checked **before** each iteration, so the body may run zero times. Since C99 you can declare the loop variable in `init`, which limits its scope to the loop. Any of the three parts can be empty; `for (;;)` is an infinite loop.

NOTES

- Loop variable declared in `init` dies after the loop
- Use `size_t` for array indexes
- Half-open ranges: `i < n`, not `i <= n`
- `break` / `continue` work as usual (Sheet 32)

LOOP PATTERNS

```c
for (int i = 0; i < 5; i++)
    printf("%d ", i);                  // 0 1 2 3 4

int arr[] = {3, 1, 4, 1, 5};
size_t n = sizeof arr / sizeof arr[0];
for (size_t i = 0; i < n; i++) sum += arr[i];

for (size_t i = n; i-- > 0; )          // backward, safe with size_t
    printf("%d ", arr[i]);             // 5 1 4 1 3

for (int i = 0, j = 9; i < j; i++, j--) { }  // two variables

for (char *p = s; *p; p++) { }         // walk a string

for (int r = 0; r < 3; r++)            // nested
    for (int c = 0; c < 3; c++)
        grid[r][c] = 0;

for (;;) { if (done()) break; }        // infinite until break
```

CODE EXPLAINED

- → `int i = 0` in init — scoped to the loop (C99)
- → `sizeof arr / sizeof arr[0]` — element count
- → `i-- > 0` — test then decrement; stops after index 0
- → `*p` — loop until the `\0` terminator
- → `for (;;)` — no condition = always true

THE THREE PARTS

PART	RUNS	EXAMPLE
`init`	once, before the loop	`int i = 0`
`condition`	before each iteration	`i < n`
`step`	after each iteration	`i++`

COMMON RANGES

HEADER	VISITS
`i = 0; i < n; i++`	`0 … n−1`
`i = 1; i <= n; i++`	`1 … n`
`i = n; i-- > 0;`	`n−1 … 0`
`i = 0; i < n; i += 2`	even indexes

⚠ COMMON MISTAKES

- `for (size_t i = n - 1; i >= 0; i--)` never ends: `size_t` can't go negative
- `for (i = 0; i <= n; i++)` on an array of `n` reads one past the end
- `for (...);` with a trailing semicolon runs an empty body

✓ TIPS & BEST PRACTICES

- Compute `strlen` or `n` once, not in the condition
- Keep the loop variable local to the loop unless you need it afterwards
- Use a `while` loop when there's no natural counter

QUICK SUMMARY

- `for (init; cond; step)` — counted loop
- Declare `i` in init (C99)
- `i < n` — half-open range
- `for (;;)` — infinite loop

while & do-while

WHILE & DO-WHILE

`while (cond) body` checks the condition first, so the body may never run. `do body while (cond);` runs the body **at least once** and checks afterwards, and needs a semicolon at the end. `while` suits loops driven by input or state rather than a counter, such as reading characters until EOF.

NOTES

- `do ... while (cond);` needs the `;`
- `while (1)` / `for (;;)` — infinite loops
- Variables declared in the body aren't visible in a do-while condition
- `do { } while (0)` — macro idiom (Sheet 77)

CONDITION-DRIVEN LOOPS

```c
int c;
while ((c = getchar()) != EOF)      // read until end of input
    putchar(toupper(c));

int n = 12345, digits = 0;
while (n != 0) {                    // 0 iterations if n == 0
    n /= 10;
    digits++;
}

int choice;
do {                                // menu runs at least once
    printf("1) Start 2) Quit: ");
    if (scanf("%d", &choice) != 1) break;
} while (choice != 2);

while (node) {                      // walk a linked list
    sum += node->value;
    node = node->next;
}
```

CODE EXPLAINED

- → `(c = getchar()) != EOF` — read, assign, test
- → `while (n != 0)` — skips the body for `n == 0`
- → `do { } while (...);` — prompt shown before the first check
- → `while (node)` — loop until `NULL`

WHILE VS DO-WHILE

	WHILE	DO-WHILE
checks condition	before body	after body
minimum runs	0	1
ends with	}	} while (c);
typical use	read loops, lists	menus, retry

PATTERNS

PATTERN	CODE
sentinel	`while ((x = next()) != END)`
retry	`do { r = try(); } while (r == EAGAIN);`
count digits	`while (n) { n /= 10; d++; }`
drain input	`while (getchar() != '\n')`

⚠ COMMON MISTAKES

- Forgetting to change the loop variable inside `while`, creating an infinite loop
- Missing `;` after `while (cond)` in a do-while: a compile error
- `while (!feof(f))` reads one extra time; test the read call's result instead (Sheet 72)

✓ TIPS & BEST PRACTICES

- Prefer `for` when there's a counter; `while` when there isn't
- Put the read call in the condition so the loop stops on failure
- Use `do-while` for input validation: ask, then check

QUICK SUMMARY

- `while` — test first, 0+ runs
- `do-while` — test last, 1+ runs
- `while ((c = getchar()) != EOF)`
- Make sure the condition changes

break, continue & goto

JUMP STATEMENTS

`break` exits the innermost loop or `switch`; `continue` skips to the next iteration of the innermost loop. C has no labeled `break`, so leaving nested loops needs a flag, a function `return`, or `goto`. `goto` jumps to a label in the same function; it's discouraged in general but widely accepted for breaking out of deep nesting and for cleanup (Sheet 35).

NOTES

- `break` in a `switch` inside a loop leaves only the `switch`
- `continue` in `for` still runs the step
- Labels are function-scoped
- Can't `goto` into a VLA's scope

BREAK, CONTINUE, GOTO

```c
for (int i = 0; i < 10; i++) {
    if (i % 2) continue;       // skip odd numbers
    if (i > 6) break;          // stop entirely
    printf("%d ", i);          // 0 2 4 6
}

// Leaving nested loops with goto
for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++)
        if (grid[r][c] == target)
            goto found;
puts("not found");
return -1;
found:
    printf("found\n");

// Or move the search into a function and return early
bool contains(int g[][COLS], int rows, int t) { /* ... return true; */ }
```

CODE EXPLAINED

- → `continue` — jumps to `i++`, then the condition
- → `break` — ends the `for` loop
- → `goto found` — exits both loops at once
- → `found:` — label; must precede a statement
- → A function with `return` is often the cleanest exit

JUMP STATEMENTS

STATEMENT	EFFECT
`break;`	exit innermost loop / switch
`continue;`	next iteration of innermost loop
`return x;`	exit function
`goto label;`	jump within the function

WHEN GOTO IS REASONABLE

USE	WHY
`error cleanup`	one exit path frees resources
`break nested loops`	no labeled break in C
`state machines`	generated or performance code
`not for loops`	use `for` / `while`

⚠ COMMON MISTAKES

- Expecting `break` inside a `switch` to exit the surrounding loop; it only leaves the `switch`
- `continue` in a `while` loop that skips the increment, causing an infinite loop
- Jumping backwards with `goto` to make loops; use a real loop

✓ TIPS & BEST PRACTICES

- Extract nested searches into functions so `return` replaces `goto`
- Use a `bool done` flag when goto would hurt readability
- Keep `goto` forward-only and local to cleanup

QUICK SUMMARY

- `break` — leave loop / switch
- `continue` — next iteration
- No labeled break: `goto`, flag or `return`
- `goto` — forward jumps for cleanup

Return Codes & errno

ERRORS WITHOUT EXCEPTIONS

C has no exceptions. Functions report failure through their **return value**: −1, NULL, EOF or a non-zero status, depending on the API. Many library and POSIX calls also set the global `errno` to a code describing *why* they failed. `errno` is only meaningful right after a call reports failure, and successful calls may leave it unchanged.

NOTES

- `errno` is thread-local in modern libcs
- `0` is never a valid error code
- `perror(msg)` prints `msg: <reason>`
- Error codes are macros in `<errno.h>`

CHECKING CALLS

```c
#include <errno.h>
#include <string.h>

FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    perror("data.txt");                 // data.txt: No such file or directory
    return -1;
}

errno = 0;
long v = strtol(text, &end, 10);
if (errno == ERANGE) { /* out of range */ }

char *buf = malloc(n);
if (!buf) {
    fprintf(stderr, "alloc %zu: %s\n", n, strerror(errno));
    exit(EXIT_FAILURE);
}
int saved = errno;                      // save before other calls
```

CODE EXPLAINED

- → `fopen` returns `NULL` and sets `errno` on failure
- → `perror` — prefix + text for the current `errno`
- → `errno = 0` first when `0`/valid results can't signal errors
- → `strerror(errno)` — message string for formatting
- → Save `errno` before calling anything else (even `printf`)

RETURN CONVENTIONS

API RETURNS	MEANS FAILURE
`pointer`	NULL (`fopen`, `malloc`)
`int (POSIX)`	−1 (`open`, `read`)
`int (stdio)`	EOF (`fclose`, `fputs`)
`count`	less than requested (`fread`)
`status`	non-zero (`pthread_*` return the code)

COMMON ERRNO VALUES

CODE	MEANING
`ENOENT`	no such file or directory
`EACCES`	permission denied
`ENOMEM`	out of memory
`EINVAL`	invalid argument
`ERANGE`	result out of range
`EINTR`	interrupted by a signal

⚠ COMMON MISTAKES

- Checking `errno` without a failed return: a success can leave an old value there
- Calling `printf` before reading `errno`; it may overwrite it
- Ignoring return values of `fclose`, `fwrite` and `snprintf`; data loss goes unnoticed

✓ TIPS & BEST PRACTICES

- Mark your own functions `[[nodiscard]]` (C23) so callers can't ignore errors
- Return `0` for success and a negative or `enum` code for errors, consistently
- Wrap repeated checks: `void *xmalloc(size_t n)` that exits on failure

QUICK SUMMARY

- Check every return value
- `errno` — why it failed
- `perror` / `strerror` — readable messages
- Save `errno` immediately

assert, static_assert & exit

ASSERTIONS & EXITING

`assert(cond)` checks a condition at runtime and aborts with a message if it's false; it documents **programmer assumptions**, not user errors, and disappears when NDEBUG is defined. `static_assert` checks conditions at **compile time**. To end a program, `exit` runs cleanup (flushes streams, calls `atexit` handlers), while `abort` stops immediately.

NOTES

- `static_assert` is a keyword in C23 (`_Static_assert` in C11)
- C23 allows `static_assert(cond)` without a message
- `assert` calls `abort()` → exit code 134 on Unix
- `return n` from `main` = `exit(n)`

ASSERT, STATIC_ASSERT, EXIT

```c
#include <assert.h>
#include <stdlib.h>

static_assert(sizeof(int) == 4, "need 32-bit int");
static_assert(CHAR_BIT == 8);                    // (C23) no message

int average(const int *a, size_t n) {
    assert(a != NULL);
    assert(n > 0 && "n must be positive");      // message trick
    /* ... */
}

static void cleanup(void) { puts("bye"); }

int main(void) {
    atexit(cleanup);            // runs on exit/return from main
    if (!config_ok()) exit(EXIT_FAILURE);
    if (corrupt) abort();       // no cleanup, core dump
    return EXIT_SUCCESS;        // prints "bye"
}
```

CODE EXPLAINED

- → `static_assert` — compile error if false
- → `assert(p)` — `file.c:7: average: Assertion 'a != NULL' failed.`
- → `&& "msg"` — string is always true, appears in the message
- → `atexit` — register up to 32+ cleanup functions (LIFO)
- → `abort` — raises `SIGABRT`; no `atexit` handlers

WAYS TO END

CALL	FLUSHES STDIO	ATEXIT
`return from main`	yes	yes
`exit(n)`	yes	yes
`quick_exit(n)`	no	`at_quick_exit` only
`_Exit(n)`	no	no
`abort()`	no	no (signal)

ASSERT VS ERROR HANDLING

SITUATION	USE
`bad user input`	return an error
`file missing`	return an error
`"can't happen" state`	`assert`
`type size assumption`	`static_assert`

⚠ COMMON MISTAKES

- Putting side effects in `assert`: `assert(init() == 0)` vanishes with `NDEBUG`
- Using `assert` for runtime errors like a failed `fopen`; release builds won't check it
- Calling `exit` inside a library function; return an error and let the caller decide

✓ TIPS & BEST PRACTICES

- Keep asserts on in test builds; consider leaving them on in release for critical invariants
- Use `static_assert` next to code that depends on struct layout or type sizes
- Return `EXIT_FAILURE`, not `-1`, for portable failure codes

QUICK SUMMARY

- `assert(cond)` — runtime check, debug only
- `static_assert` — compile-time check
- `exit` — clean shutdown; `abort` — immediate
- `atexit(fn)` — register cleanup

Error Handling Patterns

STRUCTURING ERROR HANDLING

Without exceptions or destructors, C code handles errors with patterns. The most common is **single-exit cleanup**: on failure, `goto` a label that releases resources in reverse order of acquisition. Return a **status code** and pass results through **out-parameters**. `setjmp/longjmp` can unwind several call levels at once, but skip all cleanup in between.

NOTES

- The Linux kernel uses `goto` cleanup widely
- `free(NULL)` is a no-op, which simplifies cleanup
- `longjmp` doesn't free memory or close files
- `[[nodiscard]]` (C23) warns if a status is ignored

GOTO CLEANUP

```c
typedef enum { OK = 0, ERR_IO, ERR_NOMEM } status;

[[nodiscard]] status load(const char *path, char **out) {
    status rc = ERR_IO;
    char *buf = NULL;
    FILE *f = fopen(path, "rb");
    if (!f) goto done;

    buf = malloc(4096);
    if (!buf) { rc = ERR_NOMEM; goto close_file; }
    if (fread(buf, 1, 4096, f) == 0) goto free_buf;

    *out = buf;                 // success: hand ownership out
    buf = NULL;
    rc = OK;
free_buf:   free(buf);
close_file: fclose(f);
done:       return rc;
}
```

CODE EXPLAINED

- → `status` enum — one return type for all errors
- → Labels in reverse order — undo only what succeeded
- → `*out = buf; buf = NULL;` — transfer ownership, skip free
- → `[[nodiscard]]` — caller must use the result

PATTERNS

PATTERN	SHAPE
`status + out-param`	`int parse(const char *s, int *out)`
`NULL on failure`	`Foo *foo_new(void)`
`error struct`	`bool f(..., Error *err)`
`goto cleanup`	labels release in reverse order
`setjmp/longjmp`	non-local exit (parsers, interpreters)

SETJMP / LONGJMP

CALL	DOES
`jmp_buf env;`	saved context
`if (setjmp(env) == 0)`	first return: 0
`longjmp(env, 1);`	jump back; `setjmp` returns 1
`locals changed after setjmp`	must be `volatile`

⚠ COMMON MISTAKES

- Freeing on some error paths but not others; one cleanup block avoids leaks
- Using `longjmp` across functions that hold `malloc`ed memory or open files; they leak
- Returning `-1` from one function and `0`-for-error from another in the same API

✓ TIPS & BEST PRACTICES

- Initialize every resource to `NULL` at the top so cleanup can run unconditionally
- Document which out-parameters are set on failure (ideally: none)
- Provide `const char *status_str(status s)` for readable messages

QUICK SUMMARY

- `goto cleanup` — single exit, reverse-order release
- Status return + out-params
- `setjmp` / `longjmp` — unwind, no cleanup
- Be consistent across your API

Defining & Calling Functions

FUNCTIONS

A function has a return type, a name, a parameter list and a body. Before you call it, the compiler must see a **declaration** (prototype) so it can check arguments. Prototypes usually live in a header, and the **definition** in a `.c` file. `void` means "returns nothing", and `(void)` in a parameter list means "takes nothing".

NOTES

- C has no overloading or default arguments
- C23: `int f()` now means `int f(void)`
- Pre-C23 `int f()` = unspecified arguments
- Calling an undeclared function is an error in C99+

PROTOTYPE, DEFINITION, CALL

```c
/* geometry.h */
double area(double w, double h);      // prototype
void   print_area(double a);
int    read_count(void);              // takes no arguments

/* geometry.c */
#include "geometry.h"

double area(double w, double h) {     // definition
    return w * h;
}

void print_area(double a) {
    printf("area: %.2f\n", a);        // no return value
}

/* main.c */
double a = area(3.0, 4.5);            // 13.5
print_area(a);                        // area: 13.50
area(2, 3);                           // ints converted to double
```

CODE EXPLAINED

→ Prototype — lets the compiler check calls

→ `(void)` — explicitly no parameters

→ `return w * h;` — value converted to the return type

→ `area(2, 3)` — arguments converted like assignment

SIGNATURE FORMS

DECLARATION	MEANING
`int f(void);`	no arguments
`int f();`	unspecified (pre-C23) / none (C23)
`void f(int x);`	returns nothing
`static int f(int);`	private to this file
`int f(int, char *);`	parameter names optional

RETURN RULES

SITUATION	RESULT
`void function`	`return;` or fall off end
`non-void, falls off end`	UB if caller uses value
`main falls off end`	returns `0`
`returns struct`	copied to caller

⚠ COMMON MISTAKES

- Missing `return` in a non-void function: compiles with a warning, returns garbage
- Writing `int f()` and assuming it means "no args" in pre-C23 code; the compiler won't check calls
- Defining a non-`static` function in a header: duplicate symbols at link time

✓ TIPS & BEST PRACTICES

- Make file-local helpers `static`
- Keep functions short; one job each
- Enable `-Wmissing-prototypes` to catch public functions without a header declaration

QUICK SUMMARY

- Prototype in `.h`, definition in `.c`
- `(void)` — no parameters
- `void` return — no value
- `static` — file-private function

Sheet 37

Parameters & Pass-by-Value

everything is pass-by-value · pointer params to modify caller · arrays decay to pointers · const params · int a[static 10] · multiple results via pointers

PASSING ARGUMENTS

C passes **every argument by value**: the function gets a copy. To let a function change the caller's variable, pass its **address** and write through the pointer. Arrays are the apparent exception: an array argument **decays** to a pointer to its first element, so the function can modify the elements but no longer knows the length. Pass the length separately.

NOTES

- `int a[]` parameter = `int *a`
- `const` pointer params promise read-only
- Structs are copied by value (can be big)
- `int a[static 10]` — at least 10 elements

VALUE, POINTER, ARRAY

```
void no_effect(int x)  { x = 99; }        // changes the copy
void set(int *x)       { *x = 99; }       // changes caller's var

void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

void fill(int *a, size_t n, int v) {      // array + length
    for (size_t i = 0; i < n; i++) a[i] = v;
}
int sum(const int a[], size_t n);         // read-only array

void divmod(int a, int b, int *q, int *r) { *q = a / b; *r = a % b; }

int n = 1;
no_effect(n);  set(&n);                   // n == 99
int x = 1, y = 2;  swap(&x, &y);          // x == 2, y == 1
int arr[5];    fill(arr, 5, 0);
int q, r;      divmod(17, 5, &q, &r);     // q 3, r 2
```

CODE EXPLAINED

- → `&n` — pass the address so the callee can write to `n`
- → `*x = 99` — write through the pointer
- → `size_t n` — arrays don't carry their length
- → `const int a[]` — callee can't modify elements
- → `int *q, int *r` — return several results

WHAT THE CALLEE GETS

ARGUMENT	CALLEE CAN CHANGE
`int x`	only its copy
`int *p`	caller's `int`
`const int *p`	nothing (read-only)
`int a[] / int *a`	caller's array elements
`struct S s`	only its copy
`struct S *s`	caller's struct

ARRAY PARAMETER FORMS

FORM	MEANING
`int a[]`	pointer to int
`int a[10]`	still a pointer; 10 ignored
`int a[static 10]`	non-null, ≥ 10 elements
`int m[][4]`	pointer to rows of 4
`int n, int a[n]`	VLA parameter

⚠ COMMON MISTAKES

- Using `sizeof a` on an array parameter: it's `sizeof(int *)`, not the array size
- Passing a large struct by value in a hot loop; pass `const struct S *` instead
- Forgetting `&` with `scanf("%d", n)`; it needs the address `&n`

✓ TIPS & BEST PRACTICES

- Put output parameters last and inputs first
- Use `const` on every pointer the function only reads
- Return small structs by value; it's clear and compilers optimize it well

QUICK SUMMARY

- Everything is pass-by-value
- Pass `&x` to let a function modify `x`
- Arrays decay to pointers; pass the length
- `const T *` — read-only access

Variadic Functions

VARIADIC FUNCTIONS

A function declared with `...` accepts a variable number of extra arguments, like `printf`. Inside, `<stdarg.h>` macros walk them: `va_start`, `va_arg(ap, type)`, `va_end`. The function can't tell how many arguments there are or their types, so it needs a count, a sentinel or a format string. Extra arguments undergo **default promotions**: `char`/`short` → `int`, `float` → `double`.

NOTES

- Need at least one named parameter (pre-C23)
- C23: `int f(...)` and `va_start(ap)` allowed
- Wrong `va_arg` type is UB
- `vprintf` family accepts a `va_list`

WRITING A VARIADIC FUNCTION

```c
#include <stdarg.h>

int sum(int count, ...) {
    va_list ap;
    va_start(ap, count);            // C23: va_start(ap)
    int total = 0;
    for (int i = 0; i < count; i++)
        total += va_arg(ap, int);   // fetch next as int
    va_end(ap);
    return total;
}
sum(3, 10, 20, 30);                 // 60

void log_msg(const char *fmt, ...) {      // forward to vfprintf
    va_list ap;
    va_start(ap, fmt);
    fprintf(stderr, "[log] ");
    vfprintf(stderr, fmt, ap);
    va_end(ap);
}
```

CODE EXPLAINED

- → `count` — tells the function how many args follow
- → `va_arg(ap, int)` — type must match the promoted argument
- → `va_end` — required before returning
- → `vfprintf` — the `va_list` version of `fprintf`

<STDARG.H>

MACRO	PURPOSE
`va_list ap;`	argument cursor
`va_start(ap, last)`	start after named param `last`
`va_arg(ap, T)`	next argument as T
`va_copy(dst, src)`	duplicate a cursor (C99)
`va_end(ap)`	clean up

DEFAULT PROMOTIONS

PASSED	READ WITH
`char, short, bool`	`va_arg(ap, int)`
`float`	`va_arg(ap, double)`
`int, long, pointers`	same type

V* FUNCTIONS

FUNCTION	LIKE
`vprintf`	`printf`
`vfprintf`	`fprintf`
`vsnprintf`	`snprintf`
`vsscanf`	`sscanf`

⚠ COMMON MISTAKES

- `va_arg(ap, char)` or `va_arg(ap, float)`: those types are never passed; use `int` / `double`
- Passing `NULL` as a sentinel where `NULL` is `0` (an `int`); cast it: `(char *)NULL`
- Reusing a `va_list` after passing it to `vprintf`; make a `va_copy` first

✓ TIPS & BEST PRACTICES

- Add `__attribute__((format(printf, 1, 2)))` so GCC/Clang check your printf-style wrappers
- Prefer arrays + count over varargs for non-format APIs; they're type-checked
- Variadic macros (`__VA_ARGS__`) are covered in Sheet 77

QUICK SUMMARY

- `...` — variable arguments
- `va_start` / `va_arg` / `va_end`
- Caller must signal count or types
- `vfprintf` — forward a `va_list`

Scope & Linkage

SCOPE & LINKAGE

Scope is where a name is visible: inside a block { } (block scope) or from its declaration to the end of the file (file scope). **Linkage** is whether the same name in different files refers to the same thing. File-scope names have **external** linkage by default; `static` makes them **internal** (private to the file). `extern` declares a name defined elsewhere.

NOTES

- Inner names **shadow** outer ones
- Block-scope names have no linkage
- One definition per external name per program
- Function parameters have block scope

SCOPE & LINKAGE ACROSS FILES

```c
/* counter.c */
static int count = 0;          // internal: only this file
int total = 0;                 // external: visible to others

static void bump(void) { count++; }   // private helper
void record(int n) { bump(); total += n; }

/* counter.h */
extern int total;              // declaration, not definition
void record(int n);

/* main.c */
#include "counter.h"
int x = 1;                     // file scope
void f(void) {
    int x = 2;                 // shadows file-scope x
    { int x = 3; printf("%d\n", x); }   // 3
    printf("%d\n", x);         // 2
}
```

CODE EXPLAINED

- → `static int count` — invisible outside `counter.c`
- → `int total` — the one definition
- → `extern int total;` in the header — declares it for other files
- → Inner `int x` — hides the outer one inside its block

SCOPE

DECLARED	VISIBLE
`inside a block`	to the closing }
`for-init`	the loop only
`file level`	rest of the file
`function parameter`	the function body
`label`	whole function

LINKAGE

DECLARATION (FILE SCOPE)	LINKAGE
`int x;`	external
`static int x;`	internal
`extern int x;`	refers to external x
`void f(void);`	external
`static void f(void);`	internal

⚠ COMMON MISTAKES

- Defining `int total = 0;` in a header: every `.c` that includes it defines it again (link error since GCC 10's `-fno-common`)
- Accidentally shadowing a parameter or global; enable `-Wshadow`
- Making every helper external, so names collide across files

✓ TIPS & BEST PRACTICES

- Default to `static` for file-level functions and variables
- Declare globals `extern` in one header, define them in one `.c`
- Prefer passing data as parameters over sharing globals

QUICK SUMMARY

- Block scope — inside `{ }`
- File scope — whole file after declaration
- `static` — internal linkage
- `extern` — declared here, defined elsewhere

Storage Duration & Lifetime

automatic · static locals · allocated (heap) · thread_local · lifetime vs scope · returning a pointer to a local · register

HOW LONG OBJECTS LIVE

Storage duration is how long an object exists, separate from where its name is visible. **Automatic** objects (locals) live until their block ends. **Static** objects (globals and `static` locals) live for the whole program and are zero-initialized. **Allocated** objects live from `malloc` until `free`. **Thread** storage (`thread_local`) gives each thread its own copy.

NOTES

- Static locals initialize once, before first use
- Static initializers must be constant expressions
- Returning a local's address = dangling pointer
- `register` is obsolete; can't take its address

LIFETIMES

```c
int calls(void) {
    static int n = 0;          // lives across calls
    return ++n;                // 1, 2, 3, ...
}

int *bad(void) {
    int local = 42;
    return &local;             // BUG: local dies at return
}

int *good(void) {
    int *p = malloc(sizeof *p);  // allocated: lives until free
    if (p) *p = 42;
    return p;                    // caller must free(p)
}

const char *name(void) { return "fixed"; }  // literal: static storage

thread_local int errors;     // C23 keyword (C11: _Thread_local)
```

CODE EXPLAINED

- → `static int n` — one copy, keeps its value between calls
- → `&local` — address of an object that's gone after return
- → `malloc` — lifetime controlled by you
- → String literals — static storage, safe to return
- → `thread_local` — per-thread variable

STORAGE DURATIONS

DURATION	CREATED	DESTROYED	DEFAULT
automatic	block entry	block exit	indeterminate
static	program start	program end	zero
allocated	`malloc`	`free`	indeterminate
thread	thread start	thread end	zero

WHICH DURATION?

DECLARATION	DURATION
`int g;` (file scope)	static
`static int s;` (in function)	static
`int x;` (in function)	automatic
`malloc(n)`	allocated
`"literal"`	static

⚠ COMMON MISTAKES

- Returning a pointer to a local array: `char buf[64]; return buf;`
- Assuming a `static` local resets on each call
- Using `static` buffers in functions called from multiple threads (like `strtok`); results get clobbered

✓ TIPS & BEST PRACTICES

- Return heap memory or fill a caller-provided buffer, never a local's address
- Use `static` locals for one-time initialization or simple counters
- GCC/Clang warn with `-Wreturn-local-addr` / `-Wreturn-stack-address`

QUICK SUMMARY

- automatic — block lifetime
- static — whole program, zeroed
- allocated — `malloc` to `free`
- thread — per-thread copy

Function Pointers

FUNCTION POINTERS

A **function pointer** stores the address of a function so you can call it indirectly. They power callbacks (`qsort`'s comparator), plug-in style dispatch tables and simple polymorphism. The declaration syntax is awkward, `int (*fp)(int)`, so a `typedef` usually makes them readable. A function name used as a value converts to a pointer automatically.

NOTES

- `fp(x)` and `(*fp)(x)` are equivalent
- `&f` and `f` give the same pointer
- Signatures must match exactly
- Can't convert to `void *` portably

CALLBACKS & DISPATCH

```c
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int (*op)(int, int) = add;          // pointer to function
printf("%d\n", op(2, 3));           // 5
op = mul;
printf("%d\n", op(2, 3));           // 6

typedef int (*binop)(int, int);     // readable alias
int apply(binop f, int a, int b) { return f(a, b); }
apply(add, 4, 5);                   // 9

binop table[] = { add, mul };       // dispatch table
table[1](3, 4);                     // 12

int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}
qsort(arr, n, sizeof arr[0], cmp_int);
```

CODE EXPLAINED

- → `int (*op)(int, int)` — parens around `*op` are required
- → `typedef ... binop` — name the pointer type once
- → `apply(add, ...)` — pass a function as an argument
- → `table[1](3, 4)` — call through an array
- → `(x > y) - (x < y)` — overflow-safe comparator

READING DECLARATIONS

DECLARATION	MEANING
`int (*fp)(int);`	pointer to fn(int) → int
`int *fp(int);`	function returning `int *`
`void (*arr[4])(void);`	array of 4 fn pointers
`int (*get(void))(int);`	fn returning a fn pointer

COMMON USES

USE	EXAMPLE
sorting	`qsort(..., cmp)`
callbacks	`atexit(cleanup)`
event handlers	`on_click(btn, handler)`
state machines	`state = state_fn(input)`
vtables	Sheet 69

⚠ COMMON MISTAKES

- `int *fp(int)` declares a function, not a pointer; you need `int (*fp)(int)`
- Comparator `return x - y;` overflows for large values
- Calling a function through a pointer of a different type: UB

✓ TIPS & BEST PRACTICES

- Always `typedef` function pointer types used more than once
- Pass a `void *ctx` alongside callbacks so they can reach user data
- Check the pointer for `NULL` before calling an optional callback

QUICK SUMMARY

- `T (*fp)(args)` — function pointer
- `typedef` it
- Callbacks, dispatch tables, `qsort`
- Signatures must match

Recursion

RECURSION

A **recursive** function calls itself on a smaller problem until it reaches a **base case** that returns directly. Each call gets its own stack frame with its own locals, so deep recursion can overflow the stack (often 8 MB on Linux/macOS main threads, 1 MB on Windows). C compilers may turn tail calls into loops at –O2, but the standard doesn't guarantee it.

NOTES

- Every recursive function needs a base case
- Each call costs a stack frame
- Tail-call elimination isn't guaranteed
- Trees and divide-and-conquer suit recursion

CLASSIC EXAMPLES

```c
unsigned long long factorial(unsigned n) {
    if (n <= 1) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}
factorial(5);                       // 120

long fib(int n) {                   // exponential time: O(2^n)
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}

long fib_iter(int n) {              // same result, O(n), no stack growth
    long a = 0, b = 1;
    while (n-- > 0) { long t = a + b; a = b; b = t; }
    return a;
}

int sum_tail(const int *a, size_t n, int acc) {   // tail recursive
    return n == 0 ? acc : sum_tail(a + 1, n - 1, acc + a[0]);
}
```

CODE EXPLAINED

- → `if (n <= 1) return 1;` — stops the recursion
- → `n * factorial(n - 1)` — work done after the call returns
- → `fib` — recomputes subproblems; fine only for small `n`
- → `fib_iter` — loop version
- → `sum_tail` — last action is the call; may be optimized to a loop

FACTORIAL(4) TRACE

CALL	RETURNS
`factorial(4)`	`4 * factorial(3)` = 24
`factorial(3)`	`3 * factorial(2)` = 6
`factorial(2)`	`2 * factorial(1)` = 2
`factorial(1)`	1 (base)

RECURSION VS ITERATION

USE RECURSION	USE A LOOP
`tree / graph traversal`	linear sequences
`divide & conquer (merge sort)`	unbounded depth
`parsing nested data`	performance-critical paths
`depth is O(log n)`	depth could be O(n) large

⚠ COMMON MISTAKES

- Missing or unreachable base case: infinite recursion → stack overflow (segfault)
- Recursing on user-controlled depth (e.g. deeply nested JSON) without a limit
- Naive `fib(50)` takes billions of calls; memoize or iterate

✓ TIPS & BEST PRACTICES

- Convert deep recursion to a loop with an explicit stack (array)
- Memoize overlapping subproblems with a lookup table
- Check stack size with `ulimit -s`; raise it for threads via `pthread_attr_setstacksize`

QUICK SUMMARY

- Base case + smaller recursive case
- Each call uses stack space
- Tail calls may be optimized, not guaranteed
- Deep or linear → prefer loops

inline & Attributes

INLINE & ATTRIBUTES

`inline` suggests that calls be expanded in place and changes linkage rules; in practice, define small helpers as `static inline` in headers. **Attributes** give the compiler extra information. C23 standardized the `[[...]]` syntax: `[[nodiscard]]`, `[[maybe_unused]]`, `[[deprecated]]`, `[[fallthrough]]`, `[[noreturn]]`. Older code uses GCC/Clang's `__attribute__((...))`.

NOTES

- Compilers inline small functions at `-O2` anyway
- Plain `inline` (no `static`) needs one `extern` definition
- `_Noreturn` (C11) → `[[noreturn]]` (C23)
- Unknown `[[vendor::x]]` attributes are ignored

USING ATTRIBUTES

```c
/* util.h */
static inline int clamp(int v, int lo, int hi) {
    return v < lo ? lo : v > hi ? hi : v;
}

[[nodiscard]] int open_db(const char *path);
[[deprecated("use open_db")]] int db_open(const char *path);
[[noreturn]] void die(const char *msg);

void handler([[maybe_unused]] int sig) { }

// GCC/Clang spelling (works before C23)
__attribute__((format(printf, 1, 2)))
void log_info(const char *fmt, ...);
__attribute__((cold, noinline)) void report_error(void);
[[gnu::always_inline]] static inline int sq(int x) { return x * x; }

open_db("x");     // warning: ignoring return value
```

CODE EXPLAINED

- → `static inline` in a header — each file gets its own copy, no link clash
- → `[[nodiscard]]` — warn if the result is ignored
- → `[[deprecated("msg")]]` — warn on use
- → `format(printf, 1, 2)` — check format string (arg 1) vs args from 2
- → `[[gnu::...]]` — vendor attribute in C23 syntax

STANDARD ATTRIBUTES (C23)

ATTRIBUTE	EFFECT
`[[nodiscard]]`	warn if return value unused
`[[maybe_unused]]`	silence unused warnings
`[[deprecated]]`	warn when used
`[[fallthrough]]`	intentional switch fallthrough
`[[noreturn]]`	function never returns
`[[unsequenced]] / [[reproducible]]`	pure-function hints

COMMON GNU ATTRIBUTES

ATTRIBUTE	EFFECT
`format(printf, m, n)`	check printf-style calls
`always_inline / noinline`	force inlining choice
`cold / hot`	optimize for rare / hot paths
`packed`	no struct padding
`aligned(n)`	alignment
`unused / used`	silence warning / keep symbol

⚠ COMMON MISTAKES

- Defining a plain `inline` function in a header without an `extern inline` definition somewhere: undefined reference at `-O0`
- Expecting `inline` to force inlining; it's a hint
- Marking a function `[[noreturn]]` that can return: UB

✓ TIPS & BEST PRACTICES

- Use `static inline` for small header helpers; it's the least surprising form
- Add `[[nodiscard]]` to functions that return error codes or allocated memory
- Wrap attributes in macros if you must support both C23 and older compilers

QUICK SUMMARY

- `static inline` — header helpers
- `[[nodiscard]]` `[[deprecated]]` `[[noreturn]]`
- `__attribute__((...))` — GNU spelling
- `format(printf, …)` — check wrappers

Pointer Basics

WHAT A POINTER IS

A **pointer** is a variable that holds a memory address. &x gives the address of x; ∗p **dereferences** a pointer to read or write the object it points to. A pointer's type says what it points to, which tells the compiler how to read the bytes there. A pointer that points nowhere should be the **null pointer**: NULL, or `nullptr` in C23.

NOTES

- Pointer size is fixed per platform (8 bytes on 64-bit)
- `*` in a declaration declares; in an expression dereferences
- Dereferencing `NULL` is UB (usually a crash)
- `nullptr` (C23) has its own type, `nullptr_t`

ADDRESS-OF & DEREFERENCE

```c
int x = 10;
int *p = &x;            // p holds x's address

printf("%d\n", *p);     // 10  (read through p)
*p = 20;                // write through p
printf("%d\n", x);      // 20
printf("%p\n", (void *)p);   // 0x7ffc...

int *q = NULL;          // points nowhere
if (q != NULL) *q = 5;  // always check before deref

int **pp = &p;          // pointer to pointer
**pp = 30;              // x == 30

int *a, b;              // a: int*, b: int  (!)
int *r;                 // uninitialized: wild pointer
int *s = nullptr;       // (C23)
```

CODE EXPLAINED

- → `int *p = &x;` — declare a pointer and store `x`'s address
- → `*p` — the `int` that `p` points at
- → `NULL` — guaranteed not to point at any object
- → `**pp` — follow two levels
- → `int *r;` — holds garbage; never dereference

POINTER OPERATORS

EXPRESSION	MEANING
`&x`	address of x
`*p`	object p points to
`p->m`	member of struct ∗p
`p[i]`	same as ∗(p + i)
`p == NULL`	is it null?

DECLARATIONS

DECLARATION	TYPE OF NAME
`int *p;`	pointer to `int`
`char *s;`	pointer to `char` (string)
`int **pp;`	pointer to pointer to `int`
`void *v;`	pointer to anything
`int *a[3];`	array of 3 `int *`

MEMORY PICTURE

NAME	ADDRESS	VALUE
x	0x1000	20
p	0x1008	0x1000
*p	—	20
&p	—	0x1008

⚠ COMMON MISTAKES

- Dereferencing an uninitialized pointer: `int *p; *p = 1;` writes to a random address
- Forgetting `*`: `p = 5;` changes where `p` points, not the value
- Printing a pointer with `%d` or `%x`; use `%p` with `(void *)`

✓ TIPS & BEST PRACTICES

- Initialize every pointer: to a valid address or `NULL`
- Read `int *p` as "`*p` is an `int`"
- Use `nullptr` in C23 code; it can't be mistaken for an integer

QUICK SUMMARY

- `&x` — address of
- `*p` — dereference
- `NULL` / `nullptr` — points nowhere
- Never dereference wild or null pointers

Pointer Arithmetic

p + 1 scales by type · p[i] == *(p + i) · ptrdiff_t · comparing within one array · one-past-the-end · no arithmetic on void *

POINTER ARITHMETIC

Adding an integer to a pointer moves it by that many **elements**, not bytes: `p + 1` on an `int *` advances `sizeof(int)` bytes. Subtracting two pointers into the same array gives the number of elements between them as a `ptrdiff_t`. Arithmetic is only defined within one array (plus the position one past its end); going beyond that is UB even if you never dereference.

NOTES

- `p[i]` is defined as `*(p + i)`
- So `i[p]` also works (don't use it)
- `void *` arithmetic is a GNU extension
- Use `char *` for byte-level math

WALKING AN ARRAY

```c
int a[5] = {10, 20, 30, 40, 50};
int *p = a;                    // &a[0]

printf("%d\n", *(p + 2));      // 30
printf("%d\n", p[2]);          // 30 (same thing)
p++;                           // now &a[1]
printf("%d\n", *p);            // 20

int *end = a + 5;              // one past the end: OK to form
for (int *q = a; q < end; q++)
    printf("%d ", *q);         // 10 20 30 40 50

ptrdiff_t n = end - a;         // 5
int *mid = a + n / 2;          // &a[2]

char *bytes = (char *)a;
bytes += sizeof(int);          // moves 4 bytes → a[1]
// a + 6;                      // UB: beyond one-past-end
```

CODE EXPLAINED

- → `p + 2` — two `int`s forward (8 bytes)
- → `p++` — move to the next element
- → `a + 5` — valid to compute and compare; never dereference
- → `end - a` — element count as `ptrdiff_t`
- → `(char *)` — byte steps

ALLOWED OPERATIONS

EXPRESSION	RESULT
`p + n, p - n`	pointer n elements away
`p++, p--`	next / previous element
`p - q`	`ptrdiff_t` (same array)
`p < q, p == q`	compare positions (same array)
`p + q`	**not allowed**

STEP SIZE

POINTER TYPE	P + 1 MOVES
`char *`	1 byte
`int *`	4 bytes
`double *`	8 bytes
`struct S *`	`sizeof(struct S)`
`int (*)[10]`	40 bytes (one row)

⚠ COMMON MISTAKES

- Adding byte offsets to an `int *`: `p + 4` moves 16 bytes, not 4
- Comparing pointers from different arrays with `<`: unspecified/UB
- Decrementing below the start (`p = a - 1`) as a loop sentinel: UB

✓ TIPS & BEST PRACTICES

- Prefer index notation `a[i]` for clarity; compilers generate the same code
- Use `begin` / `end` pointer pairs for half-open ranges
- Print `ptrdiff_t` with `%td`

QUICK SUMMARY

- `p + n` — moves `n` elements
- `p[i] == *(p + i)`
- `p - q` — distance (`ptrdiff_t`)
- Stay within array + one past end

Pointers & Arrays

ARRAYS ARE NOT POINTERS

In most expressions an array name **decays** to a pointer to its first element, which is why you can pass arrays to functions expecting pointers. But an array is not a pointer: `sizeof` on an array gives the whole array's size, `&arr` has type "pointer to array", and arrays can't be assigned. Once passed to a function, only the pointer remains.

NO DECAY WHEN USED WITH

- `sizeof arr` — size of the whole array
- `&arr` — pointer to the whole array
- String literal initializing a `char` array
- `typeof(arr)` (C23)

DECAY IN ACTION

```c
int arr[10];
int *p = arr;                   // decay: &arr[0]

printf("%zu\n", sizeof arr);    // 40
printf("%zu\n", sizeof p);      // 8

void f(int a[]) {               // really int *a
    printf("%zu\n", sizeof a); // 8, not 40!
}

int (*whole)[10] = &arr;        // pointer to array of 10
printf("%p %p\n", (void *)arr, (void *)&arr);  // same address
arr + 1;                        // +4 bytes  (next int)
&arr + 1;                       // +40 bytes (past the array)

int *ptrs[10];                  // array of 10 pointers
int (*rows)[10];                // pointer to arrays of 10

// arr = p;                     // error: arrays aren't assignable
#define COUNT(a) (sizeof(a) / sizeof((a)[0]))
```

CODE EXPLAINED

- → `int *p = arr;` — implicit `&arr[0]`
- → `sizeof a` in `f` — the parameter is a pointer
- → `&arr` — same address, different type
- → `int *ptrs[10]` vs `int (*rows)[10]` — `[]` binds before `*`
- → `COUNT(a)` — works only on real arrays

INT ARR[10] VS INT *P = ARR

EXPRESSION	ARR	P
`sizeof`	`40`	`8`
`type`	`int[10]`	`int *`
`&x type`	`int (*)[10]`	`int **`
`assignable`	no	yes
`x[3]`	element	element

TRICKY DECLARATIONS

DECLARATION	READ AS
`int *a[10]`	array of 10 pointers to int
`int (*a)[10]`	pointer to array of 10 int
`int a[3][4]`	array of 3 arrays of 4 int
`char *argv[]`	array of `char *` (param: `char **`)

⚠ COMMON MISTAKES

- Computing an array parameter's length with `sizeof`; always pass the length
- Using the `COUNT` macro on a pointer: it silently returns `sizeof(ptr)/sizeof(*ptr)`
- Returning a local array from a function; it's destroyed on return

✓ TIPS & BEST PRACTICES

- Clang warns `-Wsizeof-array-argument`; GCC `-Wsizeof-pointer-div`
- Wrap array + length in a struct if you pass them together often
- Use cdecl.org to decode complex declarations

QUICK SUMMARY

- Arrays decay to `&arr[0]` in expressions
- `sizeof` sees the whole array only before decay
- `&arr` — pointer to whole array
- Pass lengths explicitly

Pointers to Pointers

DOUBLE POINTERS

A pointer to a pointer (`T **`) **lets a function change** which object** the caller's pointer refers to, not just the object's contents. You'll see them in `main`'s `argv`, in functions that allocate and return memory through a parameter, in arrays of strings, and in dynamically allocated 2D arrays made of separate rows.

NOTES

- `char **argv` ⇔ `char *argv[]` as a parameter
- `T **` **doesn't convert to** `const T **` implicitly
- Jagged rows can differ in length
- Each row allocation needs its own `free`

CHANGING THE CALLER'S POINTER

```c
int make_buffer(char **out, size_t n) {
    *out = malloc(n);              // set caller's pointer
    return *out ? 0 : -1;
}
char *buf = NULL;
if (make_buffer(&buf, 64) == 0) { /* use buf */ free(buf); }

for (char **a = argv; *a; a++)     // argv ends with NULL
    puts(*a);

int **grid = malloc(rows * sizeof *grid);       // jagged 2D
for (int r = 0; r < rows; r++)
    grid[r] = calloc(cols, sizeof **grid);
grid[1][2] = 7;
for (int r = 0; r < rows; r++) free(grid[r]);
free(grid);
```

CODE EXPLAINED

- → `char **out` — address of the caller's `char *`
- → `*out = malloc(n)` — writes into the caller's variable
- → `*a` — loop until the `NULL` after the last arg
- → `grid[r] = calloc(...)` — one allocation per row
- → Free rows first, then the row array

WHERE T ** APPEARS

USE	EXAMPLE
out-parameter pointer	`int get(char **out)`
command-line args	`char **argv`
string list	`char **lines`
jagged 2D array	`int **grid`
list deletion trick	`node **pp` (Sheet 58)

LEVELS OF INDIRECTION

EXPRESSION	TYPE
`grid`	`int **`
`grid[r]` / `*grid`	`int *`
`grid[r][c]` / `**grid`	`int`
`&grid`	`int ***`

⚠ COMMON MISTAKES

- Passing `buf` instead of `&buf`: the function changes its own copy and the caller's pointer stays `NULL`
- Freeing the row array before the rows: the row pointers are lost (leak)
- Treating `int **` as a true 2D array `int[R][C]`; their memory layouts differ

✓ TIPS & BEST PRACTICES

- For rectangular data, prefer one block: `int (*m)[cols] = malloc(rows * sizeof *m);`
- Check every row allocation and clean up the ones that succeeded on failure
- Name out-parameters clearly: `out_buf`, `out_len`

QUICK SUMMARY

- `T **` — pointer to a pointer
- Pass `&ptr` to let a function reassign it
- `argv` — `NULL`-terminated `char **`
- Jagged arrays: free rows, then spine

const & void Pointers

CONST AND VOID POINTERS

`const` can apply to the pointed-to data, the pointer itself, or both, depending on where it sits: read declarations **right to left**. `void *` is a generic pointer that can hold the address of any object; C converts it to and from other object pointer types **without a cast**. `restrict` promises the compiler that no other pointer touches the same data, enabling optimizations.

NOTES

- `T *` → `const T *` is implicit and safe
- `const T *` → `T *` needs a cast (and care)
- `void *` can't be dereferenced
- `memcpy` uses `restrict`; `memmove` doesn't

CONST PLACEMENT & VOID *

```c
int x = 1, y = 2;

const int *a = &x;        // pointer to const int
// *a = 5;                // error: data is read-only
a = &y;                   // OK: pointer can move

int *const b = &x;        // const pointer to int
*b = 5;                   // OK
// b = &y;                // error: pointer can't move

const int *const c = &x;  // neither can change

void *v = &x;             // any object pointer → void *
int  *ip = v;             // void * → int *, no cast in C
size_t len(const char *s);   // promises not to modify s

void add(int *restrict out, const int *restrict in, size_t n);
```

CODE EXPLAINED

- → `const int *a` — "a is a pointer to const int"
- → `int *const b` — "b is a const pointer to int"
- → Right to left — `const` applies to what's on its left (or right, if first)
- → `void *` ↔ `T *` — implicit in C (not in C++)
- → `restrict` — `out` and `in` don't overlap

CONST PLACEMENT

DECLARATION	CHANGE *P?	CHANGE P?
`int *p`	yes	yes
`const int *p`	no	yes
`int const *p`	no	yes
`int *const p`	yes	no
`const int *const p`	no	no

VOID * RULES

OPERATION	ALLOWED?
`void *v = &x;`	yes
`int *p = v;`	yes (C), needs cast in C++
`*v`	no
`v + 1`	no (GNU extension)
`function ptr ↔ void *`	not portable (POSIX allows)

⚠ COMMON MISTAKES

- Casting away `const` and writing to an object that was defined `const`: UB
- Casting `malloc`'s result in C: unnecessary, and it can hide a missing `#include <stdlib.h>` in old compilers
- Using `restrict` when pointers might alias; the optimizer will produce wrong results

✓ TIPS & BEST PRACTICES

- Make every read-only pointer parameter `const T *`
- Use `void *` + size for generic APIs (`qsort`, `memcpy`)
- Paste confusing declarations into cdecl.org for an English reading

QUICK SUMMARY

- `const T *` — read-only data
- `T *const` — fixed pointer
- `void *` — generic, converts implicitly
- `restrict` — no aliasing promise

Pointer Pitfalls

POINTER BUGS

Pointer bugs compile cleanly and often seem to work, then crash or corrupt data later. The main families are pointers to memory that's gone (**dangling**, **use-after-free**), pointers that were never valid (**wild**, **NULL**), accessing outside an object (**out-of-bounds**), freeing incorrectly (**double free**, freeing non-heap memory), and reading memory through the wrong type (**strict aliasing**).

DETECT WITH

- `-fsanitize=address` — most of these at runtime
- `-Wall -Wextra` — some at compile time
- Valgrind (Linux) — no rebuild needed
- `clang --analyze` / `gcc -fanalyzer`

SPOT THE BUG

```c
int *p = malloc(4 * sizeof *p);
free(p);
p[0] = 1;                   // use-after-free
free(p);                    // double free

int *q;                     // wild: uninitialized
*q = 3;

int a[4];
a[4] = 0;                   // out of bounds (valid: 0..3)

char *s = get_name();       // may return NULL
printf("%zu\n", strlen(s)); // NULL deref if it did

int *dangle(void) { int x = 1; return &x; }   // dangling

float f = 1.0f;
int bits = *(int *)&f;      // strict aliasing violation
int ok;  memcpy(&ok, &f, sizeof ok);          // correct way
```

CODE EXPLAINED

- → After `free`, set `p = NULL` to make reuse crash predictably
- → `a[4]` — one past the end; writes over a neighbor
- → Check returns that may be `NULL`
- → `return &x` — `x` dies at return
- → `memcpy` — the legal way to reinterpret bytes

BUG FAMILIES

BUG	TYPICAL CAUSE	FIX
`use-after-free`	pointer kept after `free`	`p = NULL` after free
`double free`	two owners free it	single clear owner
`dangling`	address of a local returned	heap or caller buffer
`out-of-bounds`	<= loop, bad length	half-open ranges, `size_t`
`NULL deref`	unchecked `malloc`/lookup	check before use
`wild pointer`	uninitialized	initialize to `NULL`

ASAN REPORT NAMES

MESSAGE	BUG
`heap-use-after-free`	use-after-free
`heap-buffer-overflow`	out-of-bounds (heap)
`stack-buffer-overflow`	out-of-bounds (stack)
`attempting double-free`	double free
`SEGV on unknown address 0x0`	NULL deref

⚠ COMMON MISTAKES

- "It works on my machine": UB can pass tests and fail in production or after a compiler upgrade
- Freeing memory you didn't `malloc` (stack arrays, string literals, `p + 1`)
- Type punning through pointer casts instead of `memcpy` or a `union`

✓ TIPS & BEST PRACTICES

- Run every test suite under ASan + UBSan at least in CI
- Decide and document who owns each pointer
- Keep pointer lifetimes short and scopes small

QUICK SUMMARY

- Dangling / use-after-free — memory gone
- Wild / NULL — never valid
- Out-of-bounds — past the object
- ASan catches most at runtime

malloc & free

HEAP ALLOCATION

`malloc(n)` reserves n bytes on the **heap** and returns a `void *` to them, or `NULL` if it fails. The memory is uninitialized and stays allocated until you pass the same pointer to `free`. Every successful `malloc` needs exactly one matching `free`. Size allocations from the pointer itself, `sizeof *p`, so they stay correct if the type changes.

NOTES

- `malloc(0)` may return `NULL` or a unique pointer
- `free(NULL)` does nothing
- Don't cast `malloc` in C
- Memory is returned to the OS only sometimes; that's normal

MALLOC / FREE

```c
#include <stdlib.h>

int *nums = malloc(100 * sizeof *nums);   // 100 ints
if (nums == NULL) {
    perror("malloc");
    return -1;
}
for (size_t i = 0; i < 100; i++) nums[i] = (int)i;
free(nums);
nums = NULL;                  // avoid dangling use

struct point *pt = malloc(sizeof *pt);    // one struct
if (!pt) return -1;
*pt = (struct point){ .x = 1, .y = 2 };
free(pt);

char *copy = malloc(strlen(src) + 1);     // +1 for '\0'
if (copy) strcpy(copy, src);
```

CODE EXPLAINED

- → `100 * sizeof *nums` — size follows the pointer type
- → `if (nums == NULL)` — always check
- → `nums = NULL` — later misuse becomes a clean crash
- → `strlen(src) + 1` — room for the terminator

ALLOCATION API

FUNCTION	DOES
`malloc(size)`	uninitialized block
`calloc(n, size)`	zeroed block of `n * size`
`realloc(p, size)`	resize (Sheet 51)
`aligned_alloc(al, size)`	aligned block (C11)
`free(p)`	release

OWNERSHIP CONVENTIONS

FUNCTION NAME	CALLER SHOULD
`foo_create / foo_new`	call `foo_destroy` / `foo_free`
`strdup / asprintf`	`free` the result
`getenv / strerror`	**not** free
`fopen`	`fclose`

⚠ COMMON MISTAKES

- `malloc(n)` for `n` ints instead of `n * sizeof(int)`: allocates bytes, not elements
- `malloc(strlen(s))` without `+ 1` for the `'\0'`
- Losing the only pointer to a block (reassigning it) before freeing: a leak

✓ TIPS & BEST PRACTICES

- Write `T *p = malloc(n * sizeof *p);` every time
- Pair allocation and free in the same module, ideally the same function
- On Linux, `malloc` rarely returns `NULL` (overcommit), but check anyway for portability

QUICK SUMMARY

- `malloc(n * sizeof *p)` — allocate
- Check for `NULL`
- `free(p); p = NULL;` — release
- One `free` per allocation

calloc, realloc & Aligned Allocation

ZEROED, RESIZED & ALIGNED MEMORY

`calloc(n, size)` allocates an array and sets every byte to zero; it also checks `n * size` for overflow. `realloc(p, size)` grows or shrinks a block, possibly moving it, and returns the new address. If it fails it returns NULL and **leaves the old block untouched**, so assign the result to a temporary first. `aligned_alloc` returns memory with a stricter alignment.

NOTES

- `realloc(NULL, n)` acts like `malloc(n)`
- `realloc(p, 0)` is UB in C23; avoid it
- Growing by ×2 (or ×1.5) keeps appends O(1) amortized
- `free_sized(p, n)` (C23) — free with known size

CALLOC & SAFE REALLOC

```c
int *counts = calloc(256, sizeof *counts);   // 256 zeros
if (!counts) return -1;

size_t cap = 16, len = 0;
int *items = malloc(cap * sizeof *items);

if (len == cap) {
    if (cap > SIZE_MAX / 2 / sizeof *items) return -1;  // overflow check
    size_t new_cap = cap * 2;
    int *tmp = realloc(items, new_cap * sizeof *items);
    if (!tmp) { free(items); return -1; }    // old block still valid
    items = tmp;
    cap = new_cap;
}
items[len++] = 42;

void *buf = aligned_alloc(64, 4096);   // size must be multiple of 64
free(buf);
```

CODE EXPLAINED

- → `calloc` — zero-filled, overflow-checked size
- → `SIZE_MAX / 2 / sizeof` — make sure the new size fits
- → `tmp = realloc(...)` — never assign straight to `items`
- → `items = tmp` — the block may have moved
- → `aligned_alloc(64, ...)` — cache-line aligned

REALLOC OUTCOMES

CALL	RESULT
`realloc(p, bigger)`	may move; new bytes uninitialized
`realloc(p, smaller)`	usually same address
`realloc(NULL, n)`	like `malloc(n)`
`failure`	returns NULL, p still valid

MALLOC VS CALLOC

	MALLOC	CALLOC
`arguments`	total bytes	count, size
`contents`	indeterminate	all zero
`overflow check`	no	yes
`speed`	fastest	often free for fresh pages

⚠ COMMON MISTAKES

- `p = realloc(p, n);` — on failure `p` becomes `NULL` and the old block leaks
- Keeping other pointers into a block after `realloc` moved it; they now dangle
- Assuming `calloc` zero means `0.0` / `NULL` for every type; true on all mainstream platforms, not guaranteed

✓ TIPS & BEST PRACTICES

- Wrap growth logic in one function (`vec_reserve`) and reuse it
- Store indexes, not pointers, into arrays that may be `realloc` ed
- Use `calloc` for arrays of counters and structs you want zeroed

QUICK SUMMARY

- `calloc(n, size)` — zeroed array
- `tmp = realloc(p, n)` — safe resize
- Blocks may move on `realloc`
- `aligned_alloc` — custom alignment

Memory Functions

RAW MEMORY FUNCTIONS

`<string.h>` includes functions that work on raw bytes rather than strings: they take an explicit length and don't stop at `'\0'`. `memcpy` copies between non-overlapping regions, `memmove` handles overlap, `memset` fills bytes, `memcmp` compares and `memchr` searches. They're the building blocks for copying structs, arrays and buffers.

NOTES

- Sizes are in **bytes**
- `memset` sets each byte to the low 8 bits of its value
- `memcmp` on structs also compares padding
- `memset_explicit` (C23) won't be optimized away

COPY, MOVE, FILL, COMPARE

```c
#include <string.h>

int src[5] = {1, 2, 3, 4, 5}, dst[5];
memcpy(dst, src, sizeof src);            // copy 20 bytes

int a[6] = {1, 2, 3, 4, 5, 6};
memmove(a + 1, a, 5 * sizeof *a);        // overlap: {1,1,2,3,4,5}

char buf[64];
memset(buf, 0, sizeof buf);              // zero it
memset(buf, '-', 10);                    // ten dashes

int ints[4];
memset(ints, 0xFF, sizeof ints);         // each int = -1
// memset(ints, 1, sizeof ints);         // each int = 0x01010101!

if (memcmp(src, dst, sizeof src) == 0) puts("same");
char *at = memchr(buf, '-', sizeof buf); // first '-'
memset_explicit(key, 0, sizeof key);     // (C23) wipe secrets
```

CODE EXPLAINED

- → `memcpy` — fastest copy, regions must not overlap
- → `memmove` — correct even when they overlap
- → `memset(p, 0, n)` — zero `n` bytes
- → `memset(ints, 1, ...)` — sets bytes, not ints
- → `memset_explicit` — survives optimization

FUNCTIONS

FUNCTION	DOES	RETURNS
`memcpy(d, s, n)`	copy, no overlap	d
`memmove(d, s, n)`	copy, overlap-safe	d
`memset(p, c, n)`	fill bytes	p
`memcmp(a, b, n)`	compare bytes	<0, 0, >0
`memchr(p, c, n)`	find byte	pointer or NULL
`memccpy(d, s, c, n)`	copy until byte c (C23)	after c or NULL

MEM* VS STR*

MEM*	STR*
explicit length	stops at '\0'
any bytes	text only
memcpy	strcpy
memcmp	strcmp

⚠ COMMON MISTAKES

- `memcpy` with overlapping regions (e.g. shifting an array): UB; use `memmove`
- `memset(arr, 1, n)` to set ints to 1; only 0 and -1 (0xFF) work this way
- Passing an element count instead of a byte count: `memcpy(d, s, n)` for `n` ints

✓ TIPS & BEST PRACTICES

- Use `sizeof dst` or `n * sizeof *dst` for the length
- Copy structs with `=`; it's clearer than `memcpy` and type-checked
- Clear passwords with `memset_explicit` (or `explicit_bzero` on BSD/glibc)

QUICK SUMMARY

- `memcpy` — copy, no overlap
- `memmove` — copy, overlap OK
- `memset` — fill bytes
- `memcmp` / `memchr` — compare / find

Memory Layout

WHERE THINGS LIVE

A running C program's memory is divided into regions. The **stack** holds function locals and grows and shrinks with calls. The **heap** holds `malloc`ed blocks. **Static** storage holds globals and `static` variables: `.data` for initialized ones and `.bss` for zero-initialized ones. Code and string literals sit in read-only **text/rodata** segments. Objects also have **alignment** requirements.

NOTES

- Stack size: ~8 MB main thread (Linux/macOS), 1 MB (Windows)
- Secondary threads often get smaller stacks
- `.bss` takes no space in the executable file
- `alignas` / `alignof` are C23 keywords (`_Alignas` C11)

WHICH REGION?

```c
int counter = 5;               // .data (initialized static)
int table[1000];               // .bss (zeroed static)
const char *msg = "hi";        // msg: .data; "hi": .rodata

void f(void) {
    int local = 1;             // stack
    static int calls;          // .bss
    int *heap = malloc(64);    // pointer on stack, block on heap
    char big[1 << 20];         // 1 MB on the stack: risky!
    free(heap);
}

alignas(64) static char cache_line[64];  // 64-byte aligned
printf("%zu\n", alignof(double));        // 8

struct s { char c; int i; };   // 3 padding bytes after c
printf("%zu\n", sizeof(struct s));       // 8
```

CODE EXPLAINED

- → `.data` / `.bss` — static storage, exists for the whole run
- → `"hi"` — read-only literal
- → `char big[1 << 20]` — large locals can overflow the stack
- → `alignas(64)` — request stronger alignment
- → Padding — keeps `int i` 4-byte aligned (Sheet 65)

MEMORY REGIONS

REGION	HOLDS	LIFETIME
text	machine code	program
rodata	string literals, `const` globals	program
data	initialized globals / statics	program
bss	zero-initialized globals / statics	program
heap	`malloc` blocks	until `free`
stack	locals, return addresses	until block/function exit

INSPECT IT

COMMAND	SHOWS
`size ./app`	text / data / bss sizes
`nm ./app`	symbols and sections
`ulimit -s`	stack limit (KB)
`objdump -h`	section headers (Linux)

⚠ COMMON MISTAKES

- Big local arrays (`char buf[10'000'000]`) overflow the stack; use `malloc` or `static`
- Assuming stack memory starts zeroed; locals are indeterminate
- Assuming heap blocks are contiguous with each other

✓ TIPS & BEST PRACTICES

- Keep stack usage per function small; warn with `-Wstack-usage=N` (GCC)
- Use `static` for large tables that never change size
- Check alignment with `alignof` before using SIMD types

QUICK SUMMARY

- stack — locals, automatic
- heap — `malloc` / `free`
- data/bss — globals & statics
- rodata — literals, read-only

Memory Debugging

AddressSanitizer · UBSan · Valgrind (Linux) · leaks (macOS) · MallocStackLogging · ownership conventions · common leak patterns

FINDING MEMORY BUGS

Memory bugs are easiest to catch with tools. **AddressSanitizer** (ASan) instruments the build and reports overflows, use-after-free and leaks with stack traces. **UBSan** catches undefined behavior such as signed overflow. **Valgrind** runs an unmodified binary under a checker on Linux. On macOS, `leaks` and Xcode Instruments check a running process.

NOTES

- ASan slows code ~2× and uses ~3× memory
- Valgrind slows code 20–50×
- LeakSanitizer is built into ASan on Linux
- Apple clang's ASan has no leak checker; use `leaks`

RUNNING THE TOOLS

```
# AddressSanitizer + UBSan
cc -g -O1 -fsanitize=address,undefined -fno-omit-frame-pointer \
   main.c -o app
./app

# Valgrind (Linux)
cc -g -O0 main.c -o app
valgrind --leak-check=full --track-origins=yes ./app

# macOS: leaks tool
MallocStackLogging=1 leaks --atExit -- ./app

# Static analysis
gcc -fanalyzer main.c
clang --analyze main.c
```

CODE EXPLAINED

- → `-fno-omit-frame-pointer` — better stack traces
- → `--leak-check=full` — list each leaked block
- → `--track-origins=yes` — where uninitialized values came from
- → `leaks --atExit` — report leaks when the program ends
- → `-fanalyzer` — compile-time path analysis (GCC 10+)

TOOLS

TOOL	PLATFORM	FINDS
ASan	GCC/Clang, all	overflows, UAF, double free, leaks
UBSan	GCC/Clang, all	UB: overflow, shifts, alignment
MSan	Clang, Linux	uninitialized reads
Valgrind	Linux	leaks, invalid access, uninit
leaks / Instruments	macOS	leaks in a running process

OWNERSHIP RULES OF THUMB

RULE	EXAMPLE
creator documents who frees	`/* caller frees */`
pair names	`x_create` / `x_destroy`
borrowed pointers aren't freed	`getenv`, `strerror`
transfer = set source to NULL	`*out = p; p = NULL;`

⚠ COMMON MISTAKES

- Running ASan builds with Valgrind; use one or the other
- Ignoring "still reachable" leaks forever; they hide real ones in long-running programs
- Debugging a crash at `-O2` without `-g`; the stack trace is unreadable

✓ TIPS & BEST PRACTICES

- Make an ASan+UBSan build target and run tests with it in CI
- Use `ASAN_OPTIONS=abort_on_error=1` to get a core dump / debugger stop
- Fix the **first** error reported; later ones are often consequences

QUICK SUMMARY

- ASan — memory errors, fast
- UBSan — undefined behavior
- Valgrind — Linux, no rebuild
- `leaks` — macOS

Array Basics

FIXED-SIZE ARRAYS

An array is a fixed number of elements of one type stored **contiguously**. Its size must be known when it's created and can't change. Indexes run from `0` to `n − 1`, and C never checks them: an out-of-range index silently reads or corrupts other memory. Missing initializer values are set to zero, so `= {0}` clears a whole array.

NOTES

- Arrays can't be assigned or compared with `==`
- Copy with `memcpy` or wrap in a struct
- `= {}` empty initializer is C23
- Size from initializer: `int a[] = {1, 2, 3};`

CREATING & USING ARRAYS

```c
int scores[5];                          // indeterminate (local)
int zeros[100] = {0};                   // all zero
int empty[10] = {};                     // (C23) all zero
int primes[] = {2, 3, 5, 7, 11};        // size 5 from initializer
int sparse[10] = {[2] = 5, [7] = 9};    // designated; rest 0

size_t n = sizeof primes / sizeof primes[0];   // 5
for (size_t i = 0; i < n; i++)
    printf("%d ", primes[i]);           // 2 3 5 7 11

primes[0] = 1;                          // modify
// primes[5] = 13;                      // out of bounds: UB, no error

int copy[5];
memcpy(copy, primes, sizeof primes);    // copy contents
// copy = primes;                       // error: not assignable
```

CODE EXPLAINED

- → `{0}` — first element 0, the rest default to 0
- → `[2] = 5` — designated initializer (C99)
- → `sizeof a / sizeof a[0]` — element count
- → `primes[5]` — compiles, corrupts memory
- → `memcpy` — the way to copy an array

INITIALIZERS

CODE	RESULT
`int a[4] = {1, 2};`	`{1, 2, 0, 0}`
`int a[4] = {0};`	all zero
`int a[] = {1, 2, 3};`	size 3
`int a[5] = {[4] = 9};`	`{0, 0, 0, 0, 9}`
`char s[] = "abc";`	size 4 (with `\0`)

COMMON OPERATIONS

TASK	CODE
length	`sizeof a / sizeof a[0]`
copy	`memcpy(dst, src, sizeof src)`
compare	`memcmp(a, b, sizeof a) == 0`
clear	`memset(a, 0, sizeof a)`
pass to function	`f(a, n)`

⚠ COMMON MISTAKES

- Looping `i <= n`; the last valid index is `n - 1`
- Using `sizeof a / sizeof a[0]` inside a function on a parameter: gives pointer math, not the length
- Leaving a local array uninitialized and reading it

✓ TIPS & BEST PRACTICES

- Define the size once: `enum { N = 64 }; int a[N];`
- Use designated initializers for lookup tables indexed by `enum`
- Compile tests with ASan to catch out-of-bounds indexes

QUICK SUMMARY

- `T a[N]` — fixed size, contiguous
- Indexes `0` … `N-1`, unchecked
- `= {0}` — zero-initialize
- Copy with `memcpy`

Multidimensional Arrays

ARRAYS OF ARRAYS

A 2D array like `int m[3][4]` is an array of 3 rows, each an array of 4 `int`s, stored **row by row** in one block (row-major). When passed to a function, it decays to a pointer to its first row, so every dimension except the first must be known. **Variable-length arrays** (VLAs) let dimensions come from parameters, but they're optional since C11 and unsupported by MSVC.

NOTES

- `m[r][c]` is at offset `r * cols + c`
- Loop rows outside, columns inside (cache-friendly)
- `sizeof m` = `rows * cols * sizeof(int)`
- `__STDC_NO_VLA__` defined → no VLA support

2D ARRAYS

```c
int m[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12},
};
printf("%d\n", m[1][2]);                 // 7

void print_fixed(int a[][4], size_t rows);       // cols fixed
void print_vla(size_t r, size_t c, int a[r][c]); // (VLA param)

for (size_t r = 0; r < 3; r++)
    for (size_t c = 0; c < 4; c++)
        m[r][c] *= 2;

size_t rows = 100, cols = 200;           // heap 2D, one block
int (*g)[cols] = malloc(rows * sizeof *g);
g[10][20] = 1;
free(g);

int *flat = malloc(rows * cols * sizeof *flat);  // manual
flat[10 * cols + 20] = 1;
```

CODE EXPLAINED

→ Nested braces — one per row

→ `int a[][4]` — column count required

→ `int a[r][c]` — VLA parameter, dimensions passed first

→ `int (*g)[cols]` — pointer to rows; `g[r][c]` works

→ `r * cols + c` — manual flattening

MEMORY LAYOUT OF M[2][3]

INDEX	OFFSET
`m[0][0] m[0][1] m[0][2]`	0 1 2
`m[1][0] m[1][1] m[1][2]`	3 4 5

WAYS TO MAKE A 2D GRID

APPROACH	PROS / CONS
`int m[R][C]`	simple; size fixed at compile time
`int (*m)[C] = malloc`	one block, real `m[r][c]`
`int *m + r*C+c`	portable, any size
`int **m (jagged)`	rows can differ; many allocations

⚠ COMMON MISTAKES

- Passing `int m[3][4]` to a parameter `int **`; the types are incompatible
- Iterating columns in the outer loop over large arrays; much slower due to cache misses
- Using stack VLAs with user-controlled sizes; they can overflow the stack

✓ TIPS & BEST PRACTICES

- Prefer one contiguous block for grids, images and matrices
- Wrap `rows`, `cols` and data in a struct with an accessor macro
- Compile with `-Wvla` if you want to forbid VLAs

QUICK SUMMARY

- `T m[R][C]` — row-major block
- Params need all but the first dimension
- `int (*p)[C]` — heap 2D in one block
- VLAs — optional, avoid large ones

Dynamic Arrays

struct { T *data; size_t len, cap; } · push · capacity doubling · realloc · pop / remove · free

GROWABLE ARRAYS

C has no built-in resizable array, so the standard pattern is a struct holding a **data pointer, length and capacity**. Appending checks whether there's room and, if not, `reallocs` to a larger capacity (usually double). Doubling keeps the average cost of each append constant. The struct owns the buffer, so it needs init and free functions.

NOTES

- Grow geometrically (×2 or ×1.5), never +1
- Indexes stay valid after growth; pointers don't
- Start capacity at 0 or a small number
- Popular single-header versions exist (stb_ds)

A MINIMAL INT VECTOR

```c
typedef struct { int *data; size_t len, cap; } IntVec;

int vec_push(IntVec *v, int x) {
    if (v->len == v->cap) {
        size_t cap = v->cap ? v->cap * 2 : 8;
        int *p = realloc(v->data, cap * sizeof *p);
        if (!p) return -1;
        v->data = p;  v->cap = cap;
    }
    v->data[v->len++] = x;
    return 0;
}
int  vec_pop(IntVec *v) { return v->data[--v->len]; }  // check len first
void vec_free(IntVec *v) { free(v->data); *v = (IntVec){0}; }

IntVec v = {0};
for (int i = 0; i < 100; i++) vec_push(&v, i);
printf("%zu %zu\n", v.len, v.cap);       // 100 128
vec_free(&v);
```

CODE EXPLAINED

- → `{data, len, cap}` — the whole state
- → `v->cap ? v->cap * 2 : 8` — first allocation, then double
- → `realloc(NULL, ...)` — works for the first growth
- → `*v = (IntVec){0}` — reset after freeing
- → `IntVec v = {0};` — valid empty vector

OPERATIONS & COST

OPERATION	COST
push (amortized)	O(1)
pop	O(1)
index `v.data[i]`	O(1)
insert/remove middle	O(n) with `memmove`
search unsorted	O(n)

REMOVING ELEMENT I

GOAL	CODE
keep order	`memmove(&d[i], &d[i+1], (len-i-1)*sizeof *d); len--;`
fast, any order	`d[i] = d[--len];`
clear all	`v.len = 0;` (keeps capacity)

⚠ COMMON MISTAKES

- Growing by one element each push: O(n²) copying
- Holding `int *p = &v.data[3]` across a push; `realloc` may move the buffer
- Popping from an empty vector: `--len` underflows to `SIZE_MAX`

✓ TIPS & BEST PRACTICES

- Make the element type a macro parameter to generate vectors for many types (Sheet 95)
- Add `vec_reserve(v, n)` to pre-size when the count is known
- Check `cap * 2 * sizeof` for overflow in library-grade code

QUICK SUMMARY

- `struct { T *data; size_t len, cap; }`
- Double capacity on growth
- `realloc` via a temp pointer
- Pointers into it go stale on growth

Linked Lists

LINKED LISTS

A **linked list** stores each element in a separately allocated **node** that points to the next node. Inserting or removing at a known position is O(1) with no shifting, but finding an element is O(n) and nodes are scattered in memory. A **doubly linked** list adds a `prev` pointer. Using a pointer-to-pointer (`node **`) removes special cases for the head.

NOTES

- Last node's `next` is `NULL`
- Empty list = `NULL` head pointer
- Pass `&head` to functions that change the head
- Arrays are usually faster in practice (cache)

SINGLY LINKED LIST

```c
typedef struct Node { int value; struct Node *next; } Node;

int push_front(Node **head, int v) {      // O(1)
    Node *n = malloc(sizeof *n);
    if (!n) return -1;
    *n = (Node){ .value = v, .next = *head };
    *head = n;
    return 0;
}

void remove_value(Node **pp, int v) {     // no head special case
    while (*pp && (*pp)->value != v) pp = &(*pp)->next;
    if (*pp) { Node *dead = *pp; *pp = dead->next; free(dead); }
}

void list_free(Node *n) {
    while (n) { Node *next = n->next; free(n); n = next; }
}

Node *head = NULL;
push_front(&head, 3); push_front(&head, 2);   // 2 -> 3
for (Node *n = head; n; n = n->next) printf("%d ", n->value);
```

CODE EXPLAINED

- → `struct Node *next` — self-referential type needs the tag
- → `Node **head` — lets the function change the caller's head
- → `pp = &(*pp)->next` — walk the **links**, not the nodes
- → `*pp = dead->next` — unlink, whether head or middle
- → Save `next` before `free(n)`

LIST VS ARRAY

OPERATION	LIST	ARRAY
`index i`	O(n)	O(1)
`insert front`	O(1)	O(n)
`insert after known node`	O(1)	O(n)
`append (with tail ptr)`	O(1)	O(1) amortized
`memory per element`	+1–2 pointers	none

VARIANTS

KIND	NODE HAS
`singly linked`	`next`
`doubly linked`	`next`, `prev`
`circular`	last `next` → head
`intrusive`	link fields inside your struct (Linux kernel)

⚠ COMMON MISTAKES

- Freeing a node and then reading `n->next`: use-after-free
- Passing `head` instead of `&head` to `push_front`; the caller's list never changes
- Forgetting to free every node when done

✓ TIPS & BEST PRACTICES

- Keep a `tail` pointer for O(1) appends
- Use the `Node **` technique to delete without tracking `prev`
- Consider a dynamic array first; lists win only with frequent middle inserts

QUICK SUMMARY

- Node = value + `next` pointer
- `NULL` head = empty list
- `Node **` — edit links uniformly
- Free nodes one by one, saving `next`

Stacks, Queues & Ring Buffers

STACKS, QUEUES & RING BUFFERS

A **stack** is last-in, first-out: push and pop at one end, easily built on an array. A **queue** is first-in, first-out: add at the back, remove from the front. A **ring buffer** (circular buffer) implements a bounded queue in a fixed array using head and tail indexes that wrap around with % or a power-of-two mask.

NOTES

- Track `count` to tell full from empty
- Power-of-two capacity → `& (cap - 1)` wrap
- Linked lists make unbounded queues
- Ring buffers suit producer/consumer streams

ARRAY STACK & RING BUFFER

```c
enum { CAP = 8 };
typedef struct { int items[CAP]; size_t top; } Stack;
bool push(Stack *s, int v) {
    if (s->top == CAP) return false;        // full
    s->items[s->top++] = v;
    return true;
}
int pop(Stack *s) { return s->items[--s->top]; }  // check top > 0

typedef struct { int buf[CAP]; size_t head, tail, count; } Ring;

bool ring_put(Ring *r, int v) {
    if (r->count == CAP) return false;      // full
    r->buf[r->tail] = v;
    r->tail = (r->tail + 1) % CAP;          // wrap around
    r->count++;
    return true;
}
bool ring_get(Ring *r, int *v) {
    if (r->count == 0) return false;        // empty
    *v = r->buf[r->head];
    r->head = (r->head + 1) % CAP;
    r->count--;
    return true;
}
```

CODE EXPLAINED

- → `items[top++]` / `items[--top]` — push / pop
- → `tail` — next write slot; `head` — next read slot
- → `% CAP` — wrap to 0 at the end
- → `count` — distinguishes full from empty
- → `bool` return — caller learns about full / empty

OPERATIONS

STRUCTURE	ADD	REMOVE	ORDER
`stack`	push top	pop top	LIFO
`queue`	enqueue back	dequeue front	FIFO
`deque`	either end	either end	both
`ring buffer`	at tail	from head	FIFO, bounded

TYPICAL USES

STRUCTURE	USED FOR
`stack`	undo, parsing, DFS, call frames
`queue`	BFS, task scheduling
`ring buffer`	audio, serial I/O, logs
`priority queue`	heap (Sheet 61)

⚠ COMMON MISTAKES

- Popping an empty stack without checking: reads `items[-1]` (UB via `size_t` wrap)
- Using only `head == tail` to test empty; it's also true when full unless you track `count` or waste a slot
- Using `%` with a negative index when moving backward

✓ TIPS & BEST PRACTICES

- Use power-of-two capacities and `& (CAP - 1)` in hot paths
- Return `bool` and pass the value out, so errors aren't mixed with data
- For lock-free single-producer/single-consumer rings, use `_Atomic` indexes

QUICK SUMMARY

- Stack — LIFO, array + `top`
- Queue — FIFO
- Ring — fixed array, wrapping indexes
- Track `count` for full vs empty

Hash Tables

HASH TABLES

A **hash table** maps keys to values with average O(1) lookup. A **hash function** turns the key into an index into an array of **buckets**. Collisions are handled by **chaining** (each bucket holds a linked list) or **open addressing** (probe for the next free slot). When the **load factor** (entries / buckets) gets too high, the table grows and rehashes every entry.

NOTES

- C has no standard hash table
- POSIX `hsearch` exists but is limited (one global table)
- Keep load ≤ ~0.75 (chaining) or ≤ ~0.5–0.7 (open addressing)
- Decide who owns key strings

CHAINED HASH MAP (STRING → INT)

```c
typedef struct E { char *key; int val; struct E *next; } Entry;
typedef struct { Entry **buckets; size_t nbuckets, count; } Map;

static uint64_t fnv1a(const char *s) {          // FNV-1a hash
    uint64_t h = 14695981039346656037u;
    while (*s) { h ^= (unsigned char)*s++; h *= 1099511628211u; }
    return h;
}
int *map_get(Map *m, const char *key) {
    Entry *e = m->buckets[fnv1a(key) % m->nbuckets];
    for (; e; e = e->next)
        if (strcmp(e->key, key) == 0) return &e->val;
    return NULL;
}
int map_put(Map *m, const char *key, int val) {  // key is new
    Entry *e = malloc(sizeof *e);
    if (!e || !(e->key = strdup(key))) { free(e); return -1; }
    size_t i = fnv1a(key) % m->nbuckets;
    e->val = val; e->next = m->buckets[i]; m->buckets[i] = e;
    m->count++;                 // grow when count > 0.75 * nbuckets
    return 0;
}
```

CODE EXPLAINED

- → `fnv1a` — simple, decent-quality string hash
- → `% nbuckets` — map hash to a bucket
- → Chain walk with `strcmp` — handle collisions
- → `strdup(key)` — the map owns its own key copy
- → Update existing keys via `map_get`; it returns `int *`

COLLISION STRATEGIES

STRATEGY	PROS	CONS
chaining	simple; deletes easy	pointer chasing
linear probing	cache-friendly	clustering; deletes need tombstones
Robin Hood	even probe lengths	more complex

HASH FUNCTIONS

NAME	NOTES
djb2	h = h * 33 + c; tiny, OK quality
FNV-1a	simple, good for short keys
xxHash / wyhash	fast, high quality
SipHash	resists hash-flooding attacks

⚠ COMMON MISTAKES

- Storing the caller's key pointer instead of a copy; the key changes or is freed later
- Never resizing: chains grow long and lookups become O(n)
- Using `%` with a poor hash and a power-of-two table size; low bits collide

✓ TIPS & BEST PRACTICES

- Use an existing library for production (khash, uthash, stb_ds)
- Use SipHash or a random seed when keys come from untrusted input
- Keep bucket count a power of two and mix the hash, or use a prime with `%`

QUICK SUMMARY

- hash(key) → bucket index
- Chaining or open addressing
- Resize when load factor is high
- Own (copy) your keys

Trees & Heaps

BST node · insert / search · in- / pre- / post-order · delete · free a tree · array-backed binary heap

TREES & HEAPS

A **binary search tree** (BST) keeps smaller keys in the left subtree and larger ones in the right, giving O(log n) search when balanced (O(n) when not). Recursion fits trees naturally: insert, search, traverse and free are each a few lines. A **binary heap** is a complete tree stored in an **array**, where each parent is ≤ (min-heap) its children; it powers priority queues.

NOTES

- In-order traversal of a BST yields sorted order
- Free a tree in post-order
- Heap: children of `i` are `2i+1`, `2i+2`
- Parent of `i` is `(i-1)/2`

BST & HEAP BASICS

```c
typedef struct Tree { int key; struct Tree *left, *right; } Tree;

Tree *insert(Tree *t, int key) {
    if (!t) {
        t = calloc(1, sizeof *t);
        if (t) t->key = key;
        return t;
    }
    if (key < t->key)      t->left  = insert(t->left, key);
    else if (key > t->key) t->right = insert(t->right, key);
    return t;
}
void in_order(const Tree *t) {
    if (!t) return;
    in_order(t->left); printf("%d ", t->key); in_order(t->right);
}
void tree_free(Tree *t) {
    if (!t) return;
    tree_free(t->left); tree_free(t->right); free(t);
}
void sift_up(int *h, size_t i) {                 // min-heap insert step
    while (i > 0 && h[(i - 1) / 2] > h[i]) {
        size_t p = (i - 1) / 2;
        int t = h[p]; h[p] = h[i]; h[i] = t;  i = p;
    }
}
```

CODE EXPLAINED

- → `insert` returns the (possibly new) subtree root
- → Duplicates ignored — neither `<` nor `>`
- → `in_order` — left, node, right → sorted output
- → `tree_free` — children before parent
- → `sift_up` — swap with parent until order holds

TRAVERSALS

ORDER	VISIT	USE
pre-order	node, L, R	copy / serialize
in-order	L, node, R	sorted output
post-order	L, R, node	free / evaluate

COSTS

OPERATION	BALANCED BST	BINARY HEAP
find min	O(log n)	O(1)
insert	O(log n)	O(log n)
remove min	O(log n)	O(log n)
search any key	O(log n)	O(n)

⚠ COMMON MISTAKES

- Inserting sorted data into a plain BST: it degrades into a linked list (O(n))
- Freeing a node before its children; the child pointers are lost
- Deep recursion on unbalanced trees can overflow the stack

✓ TIPS & BEST PRACTICES

- Use a balanced tree (red-black, AVL) or a library when input may be sorted
- Store heaps in arrays: no pointers, cache-friendly
- Write insert/delete to return the new subtree root; it removes parent bookkeeping

QUICK SUMMARY

- BST — left < node < right
- In-order = sorted
- Heap — array, parent `(i-1)/2`
- Free in post-order

Sorting & Searching

qsort · int cmp(const void *, const void *) · not stable · bsearch · insertion sort · hand-written binary search

SORTING & SEARCHING

<stdlib.h> provides `qsort` to sort any array and `bsearch` to binary-search a sorted one. Both are generic: they take a `void *` base, element count, element size and a **comparator** function returning negative, zero or positive. `qsort` is not guaranteed to be stable. For small or nearly sorted arrays, a hand-written insertion sort is simple and fast.

NOTES

- Comparator gets pointers to elements
- `bsearch` needs the same comparator order
- `qsort` is often introsort/merge sort; O(n log n)
- C has no stable sort in the standard library

QSORT, BSEARCH, INSERTION SORT

```c
int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);                  // no overflow
}
int cmp_str(const void *a, const void *b) {
    return strcmp(*(const char *const *)a, *(const char *const *)b);
}

int v[] = {42, 7, 19, 3};
qsort(v, 4, sizeof v[0], cmp_int);             // 3 7 19 42
int key = 19;
int *hit = bsearch(&key, v, 4, sizeof v[0], cmp_int);  // &v[2]

const char *names[] = {"cy", "al", "bo"};
qsort(names, 3, sizeof names[0], cmp_str);     // al bo cy

void insertion_sort(int *a, size_t n) {
    for (size_t i = 1; i < n; i++) {
        int x = a[i]; size_t j = i;
        while (j > 0 && a[j - 1] > x) { a[j] = a[j - 1]; j--; }
        a[j] = x;
    }
}
```

CODE EXPLAINED

- → `(x > y) - (x < y)` — returns -1, 0, 1 safely
- → `cmp_str` — elements are `char *`, so args are `char **`
- → `bsearch` — returns pointer to a match or `NULL`
- → `sizeof v[0]` — element size, not array size
- → insertion sort — O(n²) but great for tiny n

API

CALL	NOTES
`qsort(base, n, size, cmp)`	in place, unstable
`bsearch(&key, base, n, size, cmp)`	sorted input required
`cmp(a, b) < 0`	a sorts first
`cmp(a, b) == 0`	equal

COMPARATOR RECIPES

SORT BY	RETURN
`int ascending`	`(x > y) - (x < y)`
`int descending`	`(x < y) - (x > y)`
`double`	`(x > y) - (x < y)` (NaN-unsafe)
`struct field`	compare `pa->age`, `pb->age`
`string array`	`strcmp(*(char )a, *(char )b)`

⚠ COMMON MISTAKES

- `return x - y;` in a comparator overflows for large or negative values
- Comparator for `char *` arrays that calls `strcmp(a, b)` on the `char **` directly
- Calling `bsearch` on unsorted data, or with a different comparator than the sort

✓ TIPS & BEST PRACTICES

- Break ties on a second field to make results deterministic
- Pre-sort once and use `bsearch` for many lookups
- For stability, sort an array of indexes and tie-break on the index

QUICK SUMMARY

- `qsort` — generic, unstable sort
- `bsearch` — binary search, sorted data
- Comparator: negative / 0 / positive
- Avoid `x - y` comparators

Struct Basics

GROUPING DATA WITH STRUCT

A `struct` groups named **members** of different types into one object. You access members with `.`, initialize them in order or by name with **designated initializers**, and copy a whole struct with plain assignment. Structs are values: assignment and passing to functions copy every member. C can't compare structs with ==; compare member by member.

NOTES

- The type name is `struct point`, not `point`
- Unnamed members in initializers are zeroed
- Structs can contain arrays and other structs
- Struct copy is shallow: pointers are copied, not targets

DEFINING & USING STRUCTS

```
struct point {
    int x;
    int y;
};                                  // semicolon required

struct point a = {3, 4};            // positional
struct point b = {.y = 7, .x = 1};  // designated, any order
struct point c = {0};               // all zero
struct point d = {};                // (C23) all zero

a.x = 10;                           // member access
b = a;                              // copies x and y
printf("(%d, %d)\n", b.x, b.y);     // (10, 4)

struct rect { struct point tl, br; char label[16]; };
struct rect r = { .tl = {0, 0}, .br = {5, 5}, .label = "box" };
int w = r.br.x - r.tl.x;            // 5

bool eq = a.x == b.x && a.y == b.y;  // no a == b in C
```

CODE EXPLAINED

- → `struct point { ... };` — defines the type
- → `.y = 7` — designated initializer (C99)
- → `b = a` — member-wise copy
- → `r.br.x` — nested access
- → Compare member by member

INITIALIZATION FORMS

CODE	RESULT
`{1, 2}`	x=1, y=2
`{.y = 2}`	x=0, y=2
`{0}`	all members zero
`(struct point){1, 2}`	compound literal (Sheet 64)

WHAT WORKS ON STRUCTS

OPERATION	ALLOWED?
`a = b`	yes (copy)
`pass / return by value`	yes
`a == b`	no
`sizeof a`	yes (includes padding)
`memcmp(&a, &b, ...)`	risky: padding bytes

⚠ COMMON MISTAKES

- Forgetting the `;` after the closing `}`: errors appear on the next line
- Comparing with `memcmp`; padding bytes may differ even when members match
- Assuming a struct copy duplicates pointed-to data; both copies share it

✓ TIPS & BEST PRACTICES

- Use designated initializers; they survive member reordering
- Write `bool point_eq(struct point a, struct point b)` helpers
- Group related parameters into a struct to shorten signatures

QUICK SUMMARY

- `struct T { ... };` — define a record
- `.member` — access
- `{.x = 1}` — designated init
- Assignment copies; `==` doesn't exist

Struct Pointers & typedef

POINTERS TO STRUCTS & TYPEDEF

Most structs are passed around by **pointer** to avoid copying and to let functions modify them. `p->m` is shorthand for `(*p).m`. A `typedef` gives the struct a one-word name. A struct can contain a pointer to its own type, which is how lists and trees are built. **Compound literals** create unnamed struct values inline.

NOTES

- `typedef struct Node Node;` lets you use `Node` before its body
- Self-reference needs the tag: `struct Node *next`
- Returning small structs by value is fine
- Compound literals are lvalues with block lifetime

->, TYPEDEF, COMPOUND LITERALS

```c
typedef struct {
    char   name[32];
    double balance;
} Account;

void deposit(Account *acct, double amt) {
    acct->balance += amt;            // same as (*acct).balance
}

Account a = {.name = "Ana", .balance = 10.0};
deposit(&a, 5.0);                    // a.balance == 15.0

typedef struct Node Node;            // forward typedef
struct Node { int value; Node *next; };

typedef struct { int x, y; } Point;
Point add(Point p, Point q) { return (Point){p.x + q.x, p.y + q.y}; }
Point r = add((Point){1, 2}, (Point){3, 4});   // {4, 6}

draw(&(Point){.x = 5, .y = 5});      // address of a temporary
```

CODE EXPLAINED

- → `typedef struct { ... } Account;` — anonymous struct with a name
- → `acct->balance` — member via pointer
- → `typedef struct Node Node;` — then `Node *next` works inside
- → `(Point){...}` — compound literal (C99)
- → `&(Point){...}` — pass a temporary by pointer

ACCESS SYNTAX

HAVE	WRITE
`struct value s`	`s.m`
`pointer p`	`p->m`
`pointer to pointer pp`	`(*pp)->m`
`array of structs a`	`a[i].m`
`array of pointers ap`	`ap[i]->m`

PASSING STRUCTS

SIGNATURE	WHEN
`void f(T t)`	small, read-only
`void f(const T *t)`	large, read-only
`void f(T *t)`	function modifies it
`T f(void)`	return small values

⚠ COMMON MISTAKES

- Using `.` on a pointer (`p.x`) or `->` on a value (`s->x`)
- Returning `&(Point){...}` from a function: the compound literal dies at block end
- `typedef` hiding a pointer (`typedef struct X *XPtr;`) makes `const` and ownership confusing

✓ TIPS & BEST PRACTICES

- Pass `const T *` for large read-only structs
- Keep the struct tag and typedef name the same: `typedef struct Node Node;`
- Use compound literals for one-off arguments and resetting: `*p = (T){0};`

QUICK SUMMARY

- `p->m` == `(*p).m`
- `typedef struct {...} T;` — short name
- Self-reference via `struct Tag *`
- `(T){...}` — compound literal

Struct Layout & Padding

PADDING & ALIGNMENT

The compiler inserts **padding** bytes between members so each one sits at an address that's a multiple of its **alignment** (often its size). The struct's total size is also rounded up to its largest alignment so arrays of it stay aligned. Ordering members from largest to smallest usually minimizes padding. A **flexible array member** lets the last member be a variable-length array.

NOTES

- Members stay in declaration order; no reordering
- `offsetof(T, m)` from `<stddef.h>`
- Packed structs may be slow or fault on some CPUs
- Padding bytes have unspecified values

LAYOUT & FLEXIBLE ARRAYS

```c
#include <stddef.h>

struct bad  { char a; double b; char c; };   // 1+7, 8, 1+7 = 24
struct good { double b; char a; char c; };   // 8, 1, 1+6   = 16

printf("%zu %zu\n", sizeof(struct bad), sizeof(struct good));  // 24 16
printf("%zu\n", offsetof(struct bad, b));     // 8

struct __attribute__((packed)) wire { uint8_t type; uint32_t len; };
// sizeof(struct wire) == 5 (GCC/Clang)

typedef struct {
    size_t len;
    char   data[];                  // flexible array member (C99)
} Buf;

Buf *b = malloc(sizeof *b + 100);   // header + 100 bytes
b->len = 100;
memcpy(b->data, src, 100);
free(b);                            // one allocation, one free
```

CODE EXPLAINED

- → `struct bad` — 7 bytes padding after each `char`
- → Largest-first ordering — less padding
- → `offsetof` — byte offset of a member
- → `packed` — no padding; unaligned access
- → `data[]` — sized at allocation time

TYPICAL ALIGNMENT (64-BIT)

TYPE	SIZE	ALIGN
`char`	1	1
`short`	2	2
`int / float`	4	4
`long / double / pointer`	8	8
`long double`	16	16 (x86-64 Linux)

LAYOUT TOOLS

TOOL	USE
`offsetof(T, m)`	member offset
`alignof(T)`	alignment requirement
`alignas(N)`	raise a member's alignment
`#pragma pack(1)`	MSVC/GCC packing
`pahole ./app`	show holes in structs (Linux)

⚠ COMMON MISTAKES

- Writing a struct to a file or socket with `fwrite` and expecting other compilers to read it: padding and endianness differ
- Taking the address of a packed member and dereferencing it as an aligned pointer
- Using `sizeof(Buf) + n` but forgetting the flexible member isn't counted in `sizeof`

✓ TIPS & BEST PRACTICES

- Order members by decreasing alignment
- Serialize field by field with fixed-width types for file formats
- Use `static_assert(sizeof(T) == N)` to lock in a layout that matters

QUICK SUMMARY

- Padding aligns members
- Largest-first ordering saves space
- `offsetof` / `alignof` — inspect layout
- `T data[]` — flexible array member

Unions

UNIONS

A `union` overlays all its members on the **same memory**, so it's only as big as its largest member and holds one value at a time. Unions save space when an object can be one of several types, usually paired with a **tag** that records which member is active (a tagged union). C also allows reading a different member than the one written (type punning), with implementation-defined results.

NOTES

- `sizeof(union)` ≥ largest member
- Writing one member changes the others' bytes
- Anonymous unions/structs are C11
- Initializer sets the **first** member unless designated

TAGGED UNION

```c
typedef enum { NUM, STR, BOOL } Kind;

typedef struct {
    Kind kind;
    union {                     // anonymous union (C11)
        double num;
        char  *str;
        bool   flag;
    };
} Value;

void print_value(const Value *v) {
    switch (v->kind) {
    case NUM:  printf("%g\n", v->num); break;
    case STR:  printf("%s\n", v->str); break;
    case BOOL: printf("%s\n", v->flag ? "true" : "false"); break;
    }
}
Value v = {.kind = NUM, .num = 3.5};
print_value(&v);                // 3.5

union { float f; uint32_t u; } pun = {.f = 1.0f};
printf("%08x\n", pun.u);        // 3f800000
```

CODE EXPLAINED

- → `Kind kind` — the tag says which member is valid
- → Anonymous union — access `v->num` directly
- → `switch` on the tag — handle each case
- → `.num = 3.5` — designate the active member
- → `pun.u` — reinterpret float bits (type punning)

STRUCT VS UNION

	STRUCT	UNION
`memory`	sum of members + padding	largest member
`valid members`	all at once	one at a time
`use`	records	variants, punning

COMMON USES

USE	EXAMPLE
`tagged variant`	JSON value, AST node
`type punning`	float ↔ bits
`byte view`	`union { uint32_t u; uint8_t b[4]; }`
`saving memory`	message payloads

⚠ COMMON MISTAKES

- Reading the member that wasn't written last, without a tag, by accident
- Forgetting to update the tag when changing the active member
- Byte-view unions assume an endianness; results differ on big-endian CPUs

✓ TIPS & BEST PRACTICES

- Always pair a union with a tag `enum` in the same struct
- Use `switch` on the tag with no `default` so `-Wswitch` flags new kinds
- Use `memcpy` instead of a union for punning if the code might be compiled as C++

QUICK SUMMARY

- `union` — members share memory
- Size = largest member
- Tagged union = enum + union
- Punning: implementation-defined in C

Enums

ENUMERATIONS

An `enum` defines named integer constants. Values start at 0 and count up unless you assign them. Enumerators are plain `int` constants usable anywhere a constant expression is needed, including `case` labels and array sizes. C enums aren't type-safe: any integer converts to an enum. C23 lets you fix the **underlying type**, e.g. `enum : uint8_t`.

NOTES

- Enumerators share one namespace with variables
- Prefix names to avoid clashes: `COLOR_RED`
- A trailing `COUNT` member gives the number of values
- Underlying type pre-C23 is implementation-defined

ENUMS IN PRACTICE

```c
enum color { RED, GREEN, BLUE };          // 0, 1, 2
enum status { OK = 0, ERR_IO = 10, ERR_MEM };   // ERR_MEM = 11

typedef enum { LOG_DEBUG, LOG_INFO, LOG_WARN, LOG_COUNT } LogLevel;
static const char *level_names[LOG_COUNT] = {
    [LOG_DEBUG] = "debug", [LOG_INFO] = "info", [LOG_WARN] = "warn",
};
printf("%s\n", level_names[LOG_WARN]);    // warn

enum perm { P_READ = 1 << 0, P_WRITE = 1 << 1, P_EXEC = 1 << 2 };
unsigned p = P_READ | P_WRITE;            // flags
if (p & P_WRITE) puts("writable");

enum : uint8_t { SMALL_A, SMALL_B };      // (C23) fixed type
enum { BUF_SIZE = 256 };                  // named constant
char buf[BUF_SIZE];

enum color c = 42;                        // allowed: no type safety
```

CODE EXPLAINED

→ Implicit values — previous + 1

→ `LOG_COUNT` — sizes arrays indexed by the enum

→ Designated array init by enum — order-proof lookup table

→ `1 << n` — bit flags combined with `|`

→ `enum { BUF_SIZE = 256 }` — typed constant pre-C23

VALUE ASSIGNMENT

DECLARATION	VALUES
`enum { A, B, C }`	0, 1, 2
`enum { A = 5, B, C }`	5, 6, 7
`enum { A = 1, B = 1 }`	duplicates allowed
`enum { A = -1, B }`	-1, 0

ENUM VS #DEFINE

ENUM	#DEFINE
`scoped like variables`	global text replacement
`visible in debugger`	gone after preprocessing
`-Wswitch coverage`	no checks
`int only (pre-C23)`	any expression

⚠ COMMON MISTAKES

- Assuming an enum variable can only hold listed values; C won't stop `enum color c = 42;`
- Two enums with the same enumerator name in one scope: redefinition error
- Printing an enum and expecting its name; you get the number. Use a lookup table

✓ TIPS & BEST PRACTICES

- End with a `_COUNT` member for loops and lookup tables
- Use `switch` without `default` over enums so new values trigger warnings
- Use C23 `enum : uint8_t` to control size in packed structs and protocols

QUICK SUMMARY

- `enum { A, B }` — named int constants
- Values start at 0 unless set
- `1 << n` — flag enums
- `enum : T` (C23) — fixed type

Bit-Fields

BIT-FIELDS

A **bit-field** member declares an exact number of bits: `unsigned mode : 3;`. The compiler packs adjacent bit-fields into storage units, saving memory and making flag sets readable. The catch is portability: bit order, packing across units and padding are all **implementation-defined**, so bit-fields don't reliably match hardware registers or file formats across compilers.

NOTES

- Allowed types: `unsigned int`, `signed int`, `_Bool` (+ others as extensions)
- Can't take the address of a bit-field
- Plain `int` bit-fields may be signed or unsigned
- `: 0` forces the next field to a new unit

DECLARING BIT-FIELDS

```c
struct flags {
    unsigned is_open   : 1;
    unsigned is_dirty  : 1;
    unsigned mode      : 3;     // 0..7
    unsigned           : 0;     // start a new storage unit
    unsigned priority  : 4;     // 0..15
};

struct flags f = {.is_open = 1, .mode = 5};
f.priority = 12;
f.mode = 9;                      // truncated to 9 & 7 == 1
printf("%zu\n", sizeof f);       // 8 (typical)
// unsigned *p = &f.mode;        // error: no address

// Portable alternative: masks on a fixed-width integer
enum { OPEN = 1u << 0, DIRTY = 1u << 1, MODE_SHIFT = 2, MODE_MASK = 7u << 2 };
uint32_t reg = OPEN | (5u << MODE_SHIFT);
unsigned mode = (reg & MODE_MASK) >> MODE_SHIFT;   // 5
```

CODE EXPLAINED

- → `: 1` — one bit, 0 or 1
- → `: 0` unnamed — alignment break
- → Out-of-range assignment — keeps the low bits (unsigned)
- → No `&` on bit-fields
- → Masks & shifts — same idea with defined layout

IMPLEMENTATION-DEFINED

ASPECT	VARIES BY
bit order in a unit	compiler / ABI / endianness
crossing unit boundaries	compiler
plain int signedness	compiler
allowed types beyond int/_Bool	compiler

BIT-FIELDS VS MASKS

BIT-FIELDS	MASKS
readable f.mode = 3	verbose x = (x & ~M) | (3 << S)
compiler does shifts	you do shifts
layout not portable	layout exact
no atomic access	works with _Atomic

⚠ COMMON MISTAKES

- Mapping bit-fields onto hardware registers or network headers and expecting the same layout on every compiler
- Using signed 1-bit fields: `int b : 1` can only hold 0 and -1
- Updating bit-fields from multiple threads; neighbors share a memory unit

✓ TIPS & BEST PRACTICES

- Use bit-fields for in-memory flag sets; masks for external formats
- Always use `unsigned` (or `bool`) for bit-field types
- Check sizes with `static_assert` if memory layout matters

QUICK SUMMARY

- `unsigned x : n` — n-bit member
- Packing & order are implementation-defined
- No address-of
- Masks for portable layouts

Opaque Types & OOP Patterns

OBJECTS IN C

C has no classes, but the same ideas work with conventions. An **opaque type** declares `struct Foo;` in the header and defines it only in the `.c` file, so users can hold `Foo *` but can't touch its fields: real encapsulation. **Create/destroy** functions stand in for constructors and destructors. A struct of **function pointers** (a vtable) gives polymorphism, and embedding a base struct as the first member gives simple inheritance.

NOTES

- Pointer to struct = pointer to its first member
- `container_of` recovers the outer struct from a member
- Name functions `type_verb`: `stack_push`
- GObject and the Linux kernel use these patterns

OPAQUE TYPE + VTABLE

```c
/* shape.h */
typedef struct Shape Shape;                 // forward declaration
typedef struct {
    double (*area)(const Shape *);
    void   (*destroy)(Shape *);
} ShapeOps;
struct Shape { const ShapeOps *ops; };      // "base class"
/* circle.c */
typedef struct { Shape base; double r; } Circle;   // hidden in .c
static double circle_area(const Shape *s) {
    const Circle *c = (const Circle *)s;    // downcast
    return 3.14159 * c->r * c->r;
}
static void circle_destroy(Shape *s) { free(s); }
static const ShapeOps CIRCLE_OPS = { circle_area, circle_destroy };

Shape *circle_new(double r) {
    Circle *c = malloc(sizeof *c);
    if (c) *c = (Circle){ .base.ops = &CIRCLE_OPS, .r = r };
    return (Shape *)c;
}
```

CODE EXPLAINED

- → `Circle` — defined only in `circle.c`; users hold `Shape *`
- → `ShapeOps` — table of "virtual" functions
- → `Shape base` first — `Circle *` ↔ `Shape *` cast is valid
- → `static` functions — private to `circle.c`
- → `circle_new` — the constructor

OOP CONCEPT → C PATTERN

CONCEPT	C PATTERN
class	`struct` + functions taking `T *`
private fields	opaque `struct T;` in header
constructor / destructor	`t_new()` / `t_free()`
method	`t_verb(T *self, ...)`
virtual method	function pointer in an ops table
inheritance	base struct as first member

CONTAINER_OF

PIECE	CODE
macro	`#define container_of(p, T, m) \`
	`((T *)((char *)(p) - offsetof(T, m)))`
use	`Circle *c = container_of(s, Circle, base);`

⚠ COMMON MISTAKES

- Putting the base struct anywhere but first and then casting `Derived *` ↔ `Base *`
- Exposing the struct definition in the header "for convenience"; users start depending on fields
- Forgetting to call the destroy function, or calling `free` directly on an object with owned resources

✓ TIPS & BEST PRACTICES

- Accept `NULL` in destroy functions, like `free` does
- Make ops tables `static const` so there's one shared copy
- Keep the public header minimal: typedef + function prototypes

QUICK SUMMARY

- Opaque `struct T;` — encapsulation
- `t_new` / `t_free` — lifecycle
- Ops table — polymorphism
- Base-first struct — inheritance

Console Input

READING FROM STDIN

`scanf` parses formatted input directly but is easy to misuse: it leaves the newline in the buffer, stops at whitespace for `%s`, and overflows buffers without a width. The robust pattern is to read a **whole line** with `fgets`, then parse it with `sscanf` or `strtol`. Always check the return value: `scanf` returns the number of fields it filled, or `EOF`.

NOTES

- `scanf` needs addresses: `&n` (not for arrays)
- `%s` reads one word; always give a width
- `fgets` keeps the `\n` if it fits
- `gets` was removed in C11; never use it

SCANF VS FGETS + SSCANF

```c
int age;
printf("Age: ");
if (scanf("%d", &age) != 1) {           // returns fields matched
    fprintf(stderr, "not a number\n");
    return 1;
}

char name[32];
scanf("%31s", name);                    // width = size - 1, no &

char line[128];
int qty; double price;
if (fgets(line, sizeof line, stdin)) {
    line[strcspn(line, "\n")] = '\0';   // strip newline
    if (sscanf(line, "%d %lf", &qty, &price) == 2)
        printf("%d x %.2f\n", qty, price);
}

int c;
while ((c = getchar()) != '\n' && c != EOF) { }   // discard rest of line
```

CODE EXPLAINED

- → `!= 1` — check how many fields were read
- → `%31s` — limit prevents overflow of `name[32]`
- → `fgets(buf, size, stdin)` — reads one line safely
- → `%lf` — `double` in `scanf` (but `%f` in `printf`)
- → `getchar` loop — clear leftover input after a bad `scanf`

SCANF SPECIFIERS

SPEC	READS INTO
`%d`	`int *`
`%ld / %lld`	`long *` / `long long *`
`%f / %lf`	`float *` / `double *`
`%c`	`char *` (no whitespace skip)
`%31s`	`char[32]`, one word
`%[^\n]`	up to newline (scanset)

RETURN VALUES

CALL	RETURNS
`scanf(...)`	fields assigned, or `EOF`
`fgets(buf, n, f)`	`buf`, or `NULL` on EOF/error
`getchar()`	char as `int`, or `EOF`
`sscanf(s, ...)`	fields assigned

⚠ COMMON MISTAKES

- `scanf("%d", age)` without `&`: writes to a garbage address
- Mixing `scanf("%d")` and `fgets`: the leftover `\n` makes `fgets` return an empty line
- Looping on `scanf` without checking its result; bad input causes an infinite loop

✓ TIPS & BEST PRACTICES

- Read lines with `fgets` and parse them; it's easier to recover from bad input
- Put a space before `%c` (`" %c"`) to skip leftover whitespace
- Use `getline` (POSIX) for lines of unknown length (Sheet 72)

QUICK SUMMARY

- `scanf` — check the return value
- `%Ns` — always set a width
- `fgets` + `sscanf` — robust input
- `%lf` for `double` in scanf

Opening & Closing Files

FILE STREAMS

Standard C I/O works through `FILE *` **streams**. `fopen(path, mode)` opens a file and returns a stream, or `NULL` with `errno` set. Three streams are always open: `stdin`, `stdout` and `stderr`. Every stream you open must be closed with `fclose`, which also flushes buffered output; ignoring its return value can hide write failures.

NOTES

- `"b"` matters only on Windows (no `\r\n` translation)
- `"x"` (C11) fails if the file exists
- Max open files is limited (`FOPEN_MAX` , ulimit)
- `freopen` redirects an existing stream

OPEN, USE, CLOSE

```c
FILE *f = fopen("notes.txt", "r");
if (!f) {
    perror("notes.txt");               // notes.txt: No such file or directory
    return -1;
}
/* ... read ... */
fclose(f);

FILE *out = fopen("out.txt", "w");     // create or truncate
FILE *log = fopen("app.log", "a");     // append, create if missing
FILE *db  = fopen("data.bin", "r+b");  // read/write, must exist
FILE *nw  = fopen("new.txt", "wx");    // (C11) fail if it exists

if (fclose(out) == EOF) perror("close out.txt");   // write error!

freopen("errors.log", "a", stderr);    // redirect stderr
fprintf(stdout, "to the terminal\n");
```

CODE EXPLAINED

- → `fopen` returns `NULL` on failure; `errno` says why
- → `"w"` — truncates an existing file to zero length
- → `"a"` — writes always go to the end
- → `"r+b"` — update an existing binary file
- → `fclose` result — last chance to see a failed write

FOPEN MODES

MODE	READ	WRITE	FILE MUST EXIST	TRUNCATES
`"r"`	✔		yes	no
`"w"`		✔	no	yes
`"a"`		✔ end	no	no
`"r+"`	✔	✔	yes	no
`"w+"`	✔	✔	no	yes
`"a+"`	✔	✔ end	no	no

STANDARD STREAMS

STREAM	DEFAULT	BUFFERING
`stdin`	keyboard	line (terminal)
`stdout`	terminal	line (terminal) / full (file)
`stderr`	terminal	unbuffered

⚠ COMMON MISTAKES

- Opening with `"w"` when you meant to append: the file is erased immediately
- Not checking `fopen` for `NULL` before reading
- Leaking file handles on early returns; use the `goto cleanup` pattern (Sheet 35)

✓ TIPS & BEST PRACTICES

- Include the path in error messages: `perror(path)`
- Use `"wx"` to avoid clobbering existing files
- Write to a temp file and `rename` it over the original for safe saves

QUICK SUMMARY

- `fopen(path, mode)` → `FILE *` or `NULL`
- `"r" "w" "a"` + `+` , `b` , `x`
- `fclose` — flush and close; check it
- `stdin` `stdout` `stderr` — always open

Reading Text Files

READING TEXT

Read text a line at a time with `fgets` (fixed buffer) or POSIX `getline` (grows its buffer), or a character at a time with `fgetc`. Loop on the **result of the read call**, not on `feof`: `feof` only becomes true after a read has already failed, so a `while (!feof(f))` loop processes the last line twice or handles garbage. After the loop, `ferror` tells you whether it ended due to an error.

NOTES

- `fgets` reads at most `n-1` chars and adds `\0`
- A line longer than the buffer arrives in pieces
- `getline` allocates; `free` the buffer once at the end
- `fgetc` returns `int` so `EOF` is distinct

LINE & CHARACTER LOOPS

```c
char line[256];
int n = 0;
while (fgets(line, sizeof line, f)) {        // NULL at EOF/error
    line[strcspn(line, "\n")] = '\0';
    printf("%3d: %s\n", ++n, line);
}
if (ferror(f)) perror("read");

char *buf = NULL;  size_t cap = 0;  ssize_t len;  // POSIX getline
while ((len = getline(&buf, &cap, f)) != -1)
    printf("%zd chars\n", len);
free(buf);

int c;  long lines = 0;
while ((c = fgetc(f)) != EOF)                // char by char
    if (c == '\n') lines++;

int id; char name[32];                       // structured: fscanf
while (fscanf(f, "%d %31s", &id, name) == 2) { }
```

CODE EXPLAINED

- → `while (fgets(...))` — stops on EOF or error
- → `strcspn` trick — drop the trailing newline
- → `getline(&buf, &cap, f)` — reallocates `buf` as needed
- → `int c` — must be `int` to hold `EOF`
- → `== 2` — stop when a record doesn't parse

READ FUNCTIONS

FUNCTION	READS	END SIGNAL
`fgets(s, n, f)`	line (≤ n-1 chars)	NULL
`getline(&s, &n, f)`	whole line (POSIX)	−1
`fgetc(f) / getc(f)`	one char	EOF
`fscanf(f, fmt, ...)`	formatted fields	count / EOF
`fread(p, sz, n, f)`	binary items	short count

AFTER THE LOOP

CHECK	MEANS
`feof(f)`	stopped at end of file
`ferror(f)`	stopped on an I/O error
`clearerr(f)`	reset both flags

⚠ COMMON MISTAKES

- `while (!feof(f)) { fgets(...); ... }` handles the last line twice
- Storing `fgetc`'s result in `char`: `0xFF` bytes look like `EOF` (or `EOF` is never seen)
- Assuming `fgets` always returns a full line; check for `\n` to detect truncation

✓ TIPS & BEST PRACTICES

- Use `getline` on POSIX systems for unknown line lengths
- Track line numbers for error messages: `file:line: message`
- Read files in binary mode on Windows if you need exact bytes

QUICK SUMMARY

- Loop on `fgets` / `getline` / `fgetc` results
- Never `while (!feof(f))`
- `ferror` after the loop
- `int c` for `fgetc`

Writing Text Files

WRITING TEXT

`fprintf` writes formatted text to a stream, `fputs` writes a string and `fputc` a character. Output is **buffered**: data sits in memory until the buffer fills, a newline is written to a line-buffered terminal, `fflush` is called or the file is closed. Write errors (disk full, closed pipe) often appear only at `fflush` or `fclose`, so check those.

NOTES

- `fputs` doesn't add a newline (`puts` does)
- `fprintf` returns chars written or negative
- `fflush(NULL)` flushes every output stream
- `fflush(stdin)` is UB

WRITING & FLUSHING

```c
FILE *f = fopen("report.csv", "w");
if (!f) { perror("report.csv"); return -1; }

fprintf(f, "name,score\n");
for (size_t i = 0; i < n; i++)
    fprintf(f, "%s,%d\n", names[i], scores[i]);
fputs("# end\n", f);
fputc('\n', f);

if (ferror(f)) perror("write");
if (fclose(f) == EOF) perror("close");   // flush errors show here

printf("Working...");          // no newline: may not appear yet
fflush(stdout);                // force it out

setvbuf(stdout, NULL, _IONBF, 0);        // unbuffered
static char big[1 << 16];
setvbuf(f2, big, _IOFBF, sizeof big);    // 64 KB buffer, before first I/O
```

CODE EXPLAINED

- → `fprintf(f, ...)` — like `printf`, to any stream
- → `ferror(f)` — any write failed so far?
- → `fclose` — flushes the remaining buffer; may fail
- → `fflush(stdout)` — show a prompt with no newline
- → `setvbuf` — choose buffering; call before other I/O

WRITE FUNCTIONS

FUNCTION	WRITES	ERROR RETURN
`fprintf(f, fmt, ...)`	formatted text	negative
`fputs(s, f)`	string as-is	EOF
`fputc(c, f) / putc`	one char	EOF
`fwrite(p, sz, n, f)`	binary items	short count

BUFFERING MODES

MODE	FLUSHES WHEN
`_IOFBF (full)`	buffer fills (files)
`_IOLBF (line)`	newline written (terminals)
`_IONBF (none)`	every write (stderr)

⚠ COMMON MISTAKES

- Debug output that disappears on a crash because it was still buffered; print to `stderr` or `fflush`
- Ignoring `fclose` errors; on a full disk that's where the failure shows
- Calling `setvbuf` after writing to the stream: UB

✓ TIPS & BEST PRACTICES

- Use `stderr` for logs and diagnostics; it's unbuffered
- For crash-safe saves: write a temp file, `fflush` + `fsync` (POSIX), then `rename`
- Batch small writes; buffered `fprintf` is far faster than many `write()` calls

QUICK SUMMARY

- `fprintf` / `fputs` / `fputc`
- Output is buffered
- `fflush` — push it out now
- Check `ferror` and `fclose`

Binary Files

BINARY I/O

`fread` and `fwrite` move raw bytes between memory and a file, counted in **items** of a given size; they return how many full items were transferred. `fseek` moves the file position, `ftell` reports it and `rewind` goes back to the start. Writing a struct directly is fast but ties the file to your compiler's padding, type sizes and **endianness**.

NOTES

- Open with `"rb"` / `"wb"` for binary on Windows
- Short count → check `feof` vs `ferror`
- `ftell` returns `long`; use `fseeko` / `ftello` for >2 GB on 32-bit
- Switching read ↔ write needs `fseek` / `fflush` between

FREAD, FWRITE, FSEEK

```c
typedef struct { uint32_t id; float score; } Rec;
Rec recs[3] = {{1, 9.5f}, {2, 7.0f}, {3, 8.25f}};

FILE *f = fopen("recs.bin", "wb");
size_t w = fwrite(recs, sizeof recs[0], 3, f);    // 3 on success
fclose(f);

f = fopen("recs.bin", "rb");
fseek(f, 0, SEEK_END);
long bytes = ftell(f);                            // 24
rewind(f);                                        // back to start

Rec r;
fseek(f, 2 * (long)sizeof r, SEEK_SET);           // jump to record 2
if (fread(&r, sizeof r, 1, f) == 1)
    printf("%u %.2f\n", r.id, r.score);           // 3 8.25

uint8_t b[4];                // portable: decode little-endian
uint32_t v = b[0] | b[1] << 8 | b[2] << 16 | (uint32_t)b[3] << 24;
```

CODE EXPLAINED

- → `fwrite(ptr, size, count, f)` — returns items written
- → `fseek(f, 0, SEEK_END)` + `ftell` — file size
- → `SEEK_SET` — offset from the start
- → `fread(...) == 1` — got one whole record
- → Assemble bytes manually to control endianness

FSEEK ORIGINS

ORIGIN	OFFSET FROM
`SEEK_SET`	beginning
`SEEK_CUR`	current position
`SEEK_END`	end of file

FUNCTIONS

FUNCTION	RETURNS
`fread(p, sz, n, f)`	items read
`fwrite(p, sz, n, f)`	items written
`fseek(f, off, origin)`	`0` on success
`ftell(f)`	position or `-1L`
`rewind(f)`	nothing (clears error flag)

RAW STRUCTS: RISKS

ISSUE	FIX
`padding`	write fields one by one
`int sizes`	use `uint32_t` etc.
`endianness`	fixed byte order
`floats`	IEEE 754 (usually fine)

⚠ COMMON MISTAKES

- `fread(buf, 1, n, f)` vs `fread(buf, n, 1, f)`: the second returns 0 or 1, hiding partial reads
- Reading binary files in text mode on Windows: `\r\n` and `0x1A` bytes get mangled
- Writing structs with pointers to a file; the addresses are meaningless when read back

✓ TIPS & BEST PRACTICES

- Use `fread(buf, 1, n, f)` when you want a byte count back
- Define file formats as explicit byte layouts with fixed-width fields
- Add a magic number and version field at the start of every binary format

QUICK SUMMARY

- `fread` / `fwrite` — items, not bytes
- `fseek` / `ftell` / `rewind` — position
- `"rb"` / `"wb"` — binary mode
- Raw structs aren't portable

Files & Directories (POSIX)

BEYOND STDIO

Standard C knows little about the file system: it can `remove`, `rename` and create a `tmpfile`, and that's it. Directories, permissions and file metadata come from **POSIX** (`<unistd.h>`, `<sys/stat.h>`, `<dirent.h>`) on Linux and macOS, or the Win32 API on Windows. POSIX also offers low-level **file descriptors** (`open`, `read`, `write`, `close`) underneath `FILE *`.

NOTES

- File descriptors are `int` s: 0 stdin, 1 stdout, 2 stderr
- `read` / `write` can transfer fewer bytes than asked
- `fileno(f)` gets the fd behind a `FILE *`
- C has no portable `mkdir` or directory listing

STANDARD + POSIX FILE OPS

```c
remove("old.txt");                          // delete (C)
rename("draft.txt", "final.txt");           // move/rename (C)
FILE *tmp = tmpfile();                      // auto-deleted on close
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <dirent.h>

int fd = open("data.bin", O_RDONLY);        // POSIX descriptor
char buf[4096];
ssize_t got = read(fd, buf, sizeof buf);    // may be < 4096
close(fd);
struct stat st;
if (stat("data.bin", &st) == 0)
    printf("%lld bytes\n", (long long)st.st_size);
if (access("cfg.ini", R_OK) == 0) { /* readable */ }
mkdir("out", 0755);

DIR *d = opendir(".");
for (struct dirent *e; d && (e = readdir(d)); )
    puts(e->d_name);
if (d) closedir(d);
```

CODE EXPLAINED

- → `remove` / `rename` — the only standard file-system calls
- → `open` / `read` / `close` — unbuffered POSIX I/O
- → `read` result — bytes actually read, `0` at EOF, `-1` error
- → `stat` — size, times, mode
- → `opendir` / `readdir` — list a directory

STANDARD C VS POSIX

TASK	STANDARD C	POSIX
delete file	remove	unlink
rename	rename	rename
file size	fseek/ftell	stat → st_size
make directory	—	mkdir(path, mode)
list directory	—	opendir / readdir
exists / perms	try fopen	access(path, R_OK)

OPEN() FLAGS

FLAG	MEANING
O_RDONLY / O_WRONLY / O_RDWR	access mode
O_CREAT	create if missing (needs mode)
O_TRUNC	truncate
O_APPEND	always write at end
O_EXCL	with O_CREAT: fail if exists

⚠ COMMON MISTAKES

- Assuming one `read` or `write` call transfers everything; loop until done
- Checking `access()` then opening (TOCTOU race); just open and handle the error
- Forgetting `closedir` / `close`; descriptors are a limited resource

✓ TIPS & BEST PRACTICES

- Prefer `FILE *` for text; use descriptors for sockets, pipes and `poll`
- On Windows, use `_mkdir`, `FindFirstFile`, or a portability library
- Use `mkstemp` (POSIX) instead of `tmpnam` for named temp files

QUICK SUMMARY

- `remove` / `rename` / `tmpfile` — standard
- `open` / `read` / `write` / `close` — POSIX fds
- `stat` — metadata
- `opendir` / `readdir` — directories

#include & Header Files

HEADER FILES

`#include` pastes another file's text in place. **Headers** (`.h`) hold what other files need to know: function prototypes, type definitions, macros and `extern` declarations. **Source files** (`.c`) hold definitions. Every header needs an **include guard** (or `#pragma once`) so including it twice in one translation unit doesn't cause redefinition errors.

NOTES

- `<file.h>` searches system paths first
- `"file.h"` searches the including file's dir first
- `#pragma once` isn't standard but is universal
- Each header should compile on its own

A WELL-FORMED HEADER

```c
/* stack.h */
#ifndef STACK_H
#define STACK_H

#include <stddef.h>               // only what the header itself needs

typedef struct Stack Stack;       // opaque type

Stack *stack_new(size_t capacity);
void   stack_free(Stack *s);
int    stack_push(Stack *s, int v);
extern int stack_debug;           // declared here, defined in stack.c

#endif /* STACK_H */

/* stack.c */
#include "stack.h"                // own header first
#include <stdlib.h>
int stack_debug = 0;
```

CODE EXPLAINED

- → `#ifndef / #define / #endif` — include guard
- → Minimal includes — keep compile times and coupling low
- → Prototypes + types — the public interface
- → `extern` — variable declaration only
- → Own header first — proves the header is self-contained

WHAT GOES WHERE

.H (HEADER)	.C (SOURCE)
function prototypes	function definitions
typedefs, struct definitions	`static` helpers
#define constants & macros	global variable definitions
extern declarations	private structs (opaque types)
static inline functions	#include of its own .h

INCLUDE SEARCH

FORM / FLAG	LOOKS IN
`#include "x.h"`	current dir, then –I, then system
`#include <x.h>`	–I dirs, then system
`-Ipath`	add a search directory
`-isystem path`	system dir (warnings suppressed)
`cc -H`	print every included file

⚠ COMMON MISTAKES

- Defining variables or non-`static` functions in a header: multiple definitions at link time
- Guard names that collide between libraries (`UTILS_H`); prefix with the project name
- `#include`-ing `.c` files instead of linking them

✓ TIPS & BEST PRACTICES

- Include what you use; don't rely on another header pulling it in
- Forward-declare structs (`struct Foo;`) instead of including their headers when you only use pointers
- Use `#pragma once` *and* guards if you want maximum compatibility

QUICK SUMMARY

- `.h` — declarations; `.c` — definitions
- `#ifndef X_H` guard or `#pragma once`
- `""` local, `<>` system
- Headers should be self-contained

Macros

MACROS

`#define` creates a **macro**: the preprocessor replaces its name with text before compilation. **Object-like** macros name constants; **function-like** macros take arguments. Because it's pure text substitution, every argument and the whole body need parentheses, arguments with side effects may be evaluated more than once, and multi-statement macros need the `do { ... } while (0)` wrapper.

NOTES

- No space between name and `(` in function-like macros
- Macros ignore scope; `#undef` removes one
- `__VA_ARGS__` — variadic macros (C99)
- Prefer `static inline` functions when possible

WRITING SAFE MACROS

```c
#define PI        3.14159265358979
#define BUF_SIZE  256
#define SQUARE(x) ((x) * (x))            // parenthesize everything
#define BAD_SQ(x) x * x
BAD_SQ(1 + 2);                           // 1 + 2 * 1 + 2 = 5 (!)
SQUARE(1 + 2);                           // 9

#define MAX(a, b) ((a) > (b) ? (a) : (b))
int i = 5;  MAX(i++, 3);                 // i++ runs twice (!)

#define SWAP(a, b) do { typeof(a) t_ = (a); (a) = (b); (b) = t_; } while (0)
if (x > y) SWAP(x, y); else puts("ok"); // works thanks to do/while

#define LOG(fmt, ...) fprintf(stderr, "[%s:%d] " fmt "\n", \
                              __FILE__, __LINE__, __VA_ARGS__)
LOG("x=%d", x);                          // [main.c:12] x=5
#undef BUF_SIZE
```

CODE EXPLAINED

- → `((x) * (x))` — parens around args and body
- → `BAD_SQ(1 + 2)` — expands to `1 + 2 * 1 + 2`
- → `MAX(i++, 3)` — argument evaluated twice
- → `do { } while (0)` — acts like one statement, needs `;`
- → `\` — continue a macro on the next line

MACRO KINDS

FORM	EXAMPLE
object-like	`#define N 10`
function-like	`#define SQ(x) ((x)*(x))`
variadic	`#define P(...) printf(__VA_ARGS__)`
empty	`#define DEBUG`
remove	`#undef N`

MACRO VS INLINE FUNCTION

MACRO	STATIC INLINE
no type checking	type-checked
args may evaluate twice	args evaluated once
works with any type	one type (or `_Generic`)
can use #, ##, __LINE__	can't

⚠ COMMON MISTAKES

- Missing parentheses: `#define DOUBLE(x) x + x` then `DOUBLE(3) * 2` gives 9
- Passing `i++` or a function call to a macro that uses its argument twice
- A trailing `;` in the macro body: `#define N 10;` breaks `int a[N];`

✓ TIPS & BEST PRACTICES

- Name macros in `ALL_CAPS` so readers know text substitution is happening
- Use `enum` / `constexpr` for constants and `static inline` for small functions
- See the expansion with `cc -E file.c`

QUICK SUMMARY

- `#define NAME value` — text replacement
- Parenthesize args and body
- `do { } while (0)` — statement macros
- `__VA_ARGS__` — variadic macros

Conditional Compilation

CONDITIONAL COMPILATION

Preprocessor conditionals include or drop code **before** compilation, based on macros. They handle platform differences, debug-only code, feature detection and optional dependencies. `#ifdef` tests whether a macro is defined; `#if` evaluates an integer expression and can use `defined()`. C23 adds `#elifdef`, `#elifndef`, `#warning` and `__has_include`.

NOTES

- Undefined names in `#if` evaluate to `0`
- Define macros at build time: `-DDEBUG`
- `#error` stops compilation with a message
- List predefined macros: `cc -dM -E - < /dev/null`

PLATFORM & FEATURE CHECKS

```c
#ifdef DEBUG
#  define DBG(...) fprintf(stderr, __VA_ARGS__)
#else
#  define DBG(...) ((void)0)
#endif

#if defined(_WIN32)
#  include <windows.h>
#elif defined(__APPLE__)
#  include <TargetConditionals.h>
#elif defined(__linux__)
#  include <unistd.h>
#else
#  error "unsupported platform"
#endif
#if __STDC_VERSION__ < 201112L
#  error "C11 or newer required"
#endif

#if __has_include(<threads.h>)          // (C23; earlier as extension)
#  include <threads.h>
#endif
```

CODE EXPLAINED

- → `#ifdef DEBUG` — only when built with `-DDEBUG`
- → `((void)0)` — no-op that still needs a `;`
- → `#if defined(...) / #elif` — platform branches
- → `#error` — fail early with a clear message
- → `__has_include` — test whether a header exists

DIRECTIVES

DIRECTIVE	MEANING
`#if expr`	include if non-zero
`#ifdef X / #ifndef X`	if (not) defined
`#elif / #else / #endif`	branches
`#elifdef X (C23)`	`#elif defined(X)`
`#error / #warning (C23)`	stop / warn

PLATFORM MACROS

MACRO	DEFINED ON
`_WIN32`	Windows (32 & 64-bit)
`__APPLE__`	macOS / iOS
`__linux__`	Linux
`__unix__`	most Unix (not macOS)
`__GNUC__ / __clang__`	GCC-compatible / Clang
`_MSC_VER`	MSVC

⚠ COMMON MISTAKES

- Typos in macro names: `#ifdef DEBGU` is silently false. Use `#if` with `-Wundef`
- Checking `__GNUC__` to mean GCC; Clang defines it too. Check `__clang__` first
- Large `#ifdef` blocks that are never compiled in CI and rot

✓ TIPS & BEST PRACTICES

- Keep platform-specific code in separate files behind a common header
- Prefer `#if FEATURE` with `-DFEATURE=0/1` over `#ifdef` so typos warn
- Use CMake/configure checks to define feature macros instead of guessing

QUICK SUMMARY

- `#if` / `#ifdef` / `#elif` / `#endif`
- `-DNAME` — define at build time
- `_WIN32` `__APPLE__` `__linux__`
- `__has_include` / `#error`

Advanced Preprocessor

PREPROCESSOR TRICKS

Two operators work only inside macros: # turns an argument into a string literal (**stringizing**) and ## glues tokens together (**token pasting**). The preprocessor also predefines macros like __FILE__ and __LINE__, and the compiler provides __func__. **X-macros** keep parallel lists (enum values and their names) in sync. C23 adds #embed to include a binary file as data.

NOTES

- Arguments of `#` / `##` aren't macro-expanded first
- Use two-level macros to expand then stringize
- `__func__` is a variable, not a macro
- `#embed` is new; check compiler support

#, ##, X-MACROS, #EMBED

```c
#define STR(x)   #x
#define XSTR(x)  STR(x)                 // expand first, then stringize
#define VERSION  3
STR(VERSION);                           // "VERSION"
XSTR(VERSION);                          // "3"

#define CAT(a, b) a##b
int CAT(var, 1) = 10;                   // int var1 = 10;
#define CHECK(e) if (!(e)) fprintf(stderr, "%s:%d %s: failed: %s\n", \
                                   __FILE__, __LINE__, __func__, #e)
#define COLORS X(RED) X(GREEN) X(BLUE)  // X-macro list
#define X(name) name,
enum color { COLORS };                  // RED, GREEN, BLUE
#undef X
#define X(name) #name,
const char *color_names[] = { COLORS }; // "RED", "GREEN", "BLUE"
#undef X

static const unsigned char icon[] = {
#embed "icon.png"                       // (C23) file bytes as initializer
};
```

CODE EXPLAINED

→ `#x` — argument text as a string

→ `XSTR` — extra level so `VERSION` expands to `3`

→ `a##b` — build identifiers

→ `#e` in `CHECK` — print the failed expression

→ X-macro — one list, many expansions

PREDEFINED NAMES

NAME	VALUE
`__FILE__`	current file name
`__LINE__`	current line number
`__func__`	current function name
`__DATE__ / __TIME__`	build date / time
`__STDC_VERSION__`	language standard
`__COUNTER__`	incrementing int (extension)

MACRO OPERATORS

OPERATOR	EFFECT
`#arg`	stringize
`a ## b`	paste tokens
`__VA_ARGS__`	variadic arguments
`__VA_OPT__(x) (C23)`	x only if args present
`defined(X)`	in #if

⚠ COMMON MISTAKES

- Expecting `STR(VERSION)` to give `"3"`; `#` doesn't expand its argument. Use a helper macro
- Using `__DATE__` / `__TIME__` in reproducible builds; output changes every build
- Pasting tokens that don't form a valid token: `CAT(+, +)` style errors

✓ TIPS & BEST PRACTICES

- Use X-macros for enum ↔ string tables and command dispatch
- Write `ASSERT` -style macros with `#e`, `__FILE__`, `__LINE__` for clear failures
- Debug macro expansion with `cc -E` and search for your line

QUICK SUMMARY

- `#x` — stringize; `##` — paste
- `__FILE__` `__LINE__` `__func__`
- X-macros — keep lists in sync
- `#embed` (C23) — binary data

Multi-File Projects & Libraries

MULTI-FILE PROGRAMS & LIBRARIES

Real programs split code across files: each `.c` compiles to an object file, and the **linker** joins them. Reusable code can be packaged as a **static library** (`.a`, copied into each executable) or a **shared library** (`.so` on Linux, `.dylib` on macOS, `.dll` on Windows) loaded at run time. `pkg-config` reports the flags needed to use installed libraries.

NOTES

- Static libs are linked only for symbols you use
- Shared libs need `-fPIC` when compiling
- `LD_LIBRARY_PATH` / `DYLD_LIBRARY_PATH` find shared libs
- `-Wl,-rpath,...` bakes a search path into the binary

BUILDING LIBRARIES

```
# Separate compilation
cc -c main.c util.c                  # main.o util.o
cc main.o util.o -o app

# Static library
cc -c vec.c map.c
ar rcs libds.a vec.o map.o
cc main.c -L. -lds -o app            # -lds → libds.a

# Shared library
cc -fPIC -c vec.c map.c
cc -shared vec.o map.o -o libds.so           # Linux
cc -dynamiclib vec.o map.o -o libds.dylib    # macOS
cc main.c -L. -lds -Wl,-rpath,'$ORIGIN' -o app

# Installed libraries
cc main.c $(pkg-config --cflags --libs libcurl) -o app

nm -g libds.a                        # exported symbols
ldd ./app        # Linux deps;  otool -L ./app on macOS
```

CODE EXPLAINED

- → `ar rcs` — create/replace/index an archive
- → `-L.` `-lds` — search `.` for `libds.a` / `libds.so`
- → `-fPIC` — position-independent code for shared libs
- → `$ORIGIN` — find libs next to the executable (Linux)
- → `pkg-config` — correct `-I`, `-L`, `-l` flags

LIBRARY KINDS

	STATIC	SHARED
`file`	`.a` / `.lib`	`.so` / `.dylib` / `.dll`
`linked`	at build time, copied in	loaded at run time
`update lib`	relink programs	replace the file
`binary size`	larger	smaller

INSPECTION TOOLS

TOOL	SHOWS
`nm`	symbols (T defined, U undefined)
`ldd / otool -L`	shared lib dependencies
`objdump -d`	disassembly
`readelf -d`	ELF dynamic section (Linux)

⚠ COMMON MISTAKES

- Library order: with GNU ld, `-lfoo` must come after the objects that use it
- Shared library found at build time but not at run time: set rpath or the loader path
- Two libraries defining the same global symbol; mark internals `static`

✓ TIPS & BEST PRACTICES

- Hide internal symbols: `-fvisibility=hidden` plus `__attribute__((visibility("default")))` on the API
- Version shared libraries with sonames (`libds.so.1`)
- Let CMake handle library details for anything non-trivial (Sheet 82)

QUICK SUMMARY

- `.c` → `.o` → link
- `ar rcs libx.a` — static library
- `-shared -fPIC` — shared library
- `pkg-config --cflags --libs` — dependency flags

Makefiles

MAKEFILES

make rebuilds only what changed. A **Makefile** lists **rules**: a target, the prerequisites it depends on, and a recipe of shell commands (indented with a **tab**). If any prerequisite is newer than the target, the recipe runs. Variables like CC and CFLAGS, automatic variables like $@, and **pattern rules** keep Makefiles short.

NOTES

- Recipes **must** start with a tab character
- First target is the default (`make` alone)
- `make -j8` builds in parallel
- `-MMD -MP` auto-tracks header dependencies

A PRACTICAL MAKEFILE

```
CC      := cc
CFLAGS  := -std=c17 -Wall -Wextra -g -MMD -MP
LDFLAGS :=
LDLIBS  := -lm

SRCS := $(wildcard src/*.c)
OBJS := $(SRCS:src/%.c=build/%.o)

app: $(OBJS)
    $(CC) $(LDFLAGS) $^ $(LDLIBS) -o $@

build/%.o: src/%.c | build
    $(CC) $(CFLAGS) -c $< -o $@

build:
    mkdir -p build

.PHONY: clean
clean:
    rm -rf build app

-include $(OBJS:.o=.d)
```

CODE EXPLAINED

- → `:=` — assign once (simply expanded)
- → `$(SRCS:src/%.c=build/%.o)` — map sources to objects
- → `$@` target, `$<` first prereq, `$^` all prereqs
- → `| build` — order-only: dir must exist
- → `-include *.d` — header deps from `-MMD`

AUTOMATIC VARIABLES

VAR	MEANS
$@	target name
$<	first prerequisite
$^	all prerequisites
$?	prerequisites newer than target
$*	stem matched by %

CONVENTIONAL VARIABLES

VARIABLE	HOLDS
CC	C compiler
CFLAGS	compile flags
CPPFLAGS	preprocessor flags (-I, -D)
LDFLAGS	linker flags (-L)
LDLIBS	libraries (-lm)

COMMANDS

COMMAND	DOES
make	build default target
make -j$(nproc)	parallel build
make -n	print, don't run
make CFLAGS=-O2	override a variable
make -B	rebuild everything

⚠ COMMON MISTAKES

- Spaces instead of a tab before a recipe: `missing separator`
- Not listing headers as dependencies, so changing a `.h` doesn't rebuild; use `-MMD -MP`
- Forgetting `.PHONY` for `clean`; a file named `clean` stops it from running

✓ TIPS & BEST PRACTICES

- Put objects in a separate `build/` directory
- Put `LDLIBS` after the objects in the link line
- For multi-platform projects, consider CMake (Sheet 82)

QUICK SUMMARY

- `target: prereqs` + tab recipe
- `$@ $< $^` — automatic variables
- `%.o: %.c` — pattern rule
- `-MMD -MP` — header tracking

CMake

CMAKE

CMake generates build files (Makefiles, Ninja, Xcode, Visual Studio) from a portable `CMakeLists.txt`. Modern CMake is **target-based**: you create executables and libraries with `add_executable` / `add_library`, then attach include paths, flags and dependencies to each target with `target_*` commands. Builds happen **out of source** in a separate directory.

NOTES

- `PUBLIC` / `PRIVATE` / `INTERFACE` control propagation
- Use `-G Ninja` for fast builds
- `CMAKE_EXPORT_COMPILE_COMMANDS` helps clangd
- `find_package` locates installed libraries

CMAKELISTS.TXT

```
cmake_minimum_required(VERSION 3.20)
project(inventory LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

add_library(ds STATIC src/vec.c src/map.c)
target_include_directories(ds PUBLIC include)
target_compile_options(ds PRIVATE -Wall -Wextra)

add_executable(app src/main.c)
target_link_libraries(app PRIVATE ds m)

enable_testing()
add_executable(test_vec tests/test_vec.c)
target_link_libraries(test_vec PRIVATE ds)
add_test(NAME vec COMMAND test_vec)
```

CODE EXPLAINED

- → `CMAKE_C_STANDARD 17` — adds `-std=c17` (or equivalent)
- → `add_library(ds STATIC ...)` — a library target
- → `PUBLIC include` — users of `ds` get the include path too
- → `target_link_libraries(app ... m)` — link `ds` and libm
- → `add_test` — run with `ctest`

COMMAND LINE

COMMAND	DOES
`cmake -B build`	configure into `build/`
`cmake -B build -G Ninja`	use Ninja
`cmake -B build -DCMAKE_BUILD_TYPE=Release`	optimized build
`cmake --build build -j`	build
`ctest --test-dir build`	run tests

VISIBILITY KEYWORDS

KEYWORD	APPLIES TO
`PRIVATE`	this target only
`PUBLIC`	this target and its users
`INTERFACE`	users only (header-only libs)

⚠ COMMON MISTAKES

- Using global `include_directories()` / `add_compile_options()` instead of `target_*` commands
- Building in the source directory; use `-B build`
- Forgetting `CMAKE_BUILD_TYPE` with single-config generators, getting an unoptimized build

✓ TIPS & BEST PRACTICES

- Symlink `build/compile_commands.json` to the project root for editors
- Use `find_package(CURL REQUIRED)` then link `CURL::libcurl`
- Add sanitizer presets in `CMakePresets.json`

QUICK SUMMARY

- `add_executable` / `add_library`
- `target_*` — per-target settings
- `cmake -B build && cmake --build build`
- `ctest` — run registered tests

stdlib.h Utilities

GENERAL UTILITIES

`<stdlib.h>` is the grab bag of the standard library: memory allocation, string-to-number conversion, random numbers, sorting and searching, program termination, environment access and integer helpers like `abs` and `div`. Its `rand` is fine for games and tests but **not** for security or serious simulations.

NOTES

- `RAND_MAX` is at least 32767
- `abs(INT_MIN)` is UB (no positive equivalent)
- `getenv` result must not be modified or freed
- `system` runs a shell command; avoid with user input

STDLIB.H IN ACTION

```c
#include <stdlib.h>
#include <time.h>

printf("%d %ld\n", abs(-5), labs(-7L));      // 5 7
div_t d = div(17, 5);                        // d.quot 3, d.rem 2

srand((unsigned)time(NULL));                 // seed once
int die = rand() % 6 + 1;                    // 1..6 (slightly biased)
double u = rand() / (RAND_MAX + 1.0);        // [0, 1)

const char *home = getenv("HOME");           // NULL if unset
if (home) printf("%s\n", home);

int rc = system("ls -l");                    // shell exit status
char *end;
double x = strtod("2.5kg", &end);            // 2.5, end → "kg"
qsort(arr, n, sizeof arr[0], cmp_int);       // Sheet 62
exit(EXIT_SUCCESS);
```

CODE EXPLAINED

- → `div` — quotient and remainder together
- → `srand(time(NULL))` — different sequence each run
- → `rand() % 6` — small bias; fine for games
- → `getenv` — read environment variables
- → `system` — returns implementation-defined status

FUNCTION GROUPS

GROUP	FUNCTIONS
memory	`malloc` `calloc` `realloc` `free` `aligned_alloc`
conversion	`strtol` `strtoul` `strtod` `atoi`
random	`rand` `srand`
search / sort	`qsort` `bsearch`
integer math	`abs` `labs` `llabs` `div` `ldiv`
process	`exit` `atexit` `abort` `getenv` `system`

BETTER RANDOM SOURCES

SOURCE	USE
`rand()`	quick, low quality
`arc4random_uniform(n)`	BSD/macOS, glibc 2.36+
`getrandom()` / `/dev/urandom`	Linux, secure bytes
PCG / xoshiro	fast seeded PRNG (copy-in)

⚠ COMMON MISTAKES

- Calling `srand` inside a loop; every call resets the sequence
- Using `rand()` for passwords, tokens or keys
- Modifying the string returned by `getenv`

✓ TIPS & BEST PRACTICES

- Seed once at startup; use a fixed seed in tests for reproducibility
- Use `arc4random_uniform(n)` where available for unbiased ranges
- Prefer `posix_spawn` / `fork` + `exec` over `system` for running programs

QUICK SUMMARY

- `abs` `div` — integer helpers
- `rand` / `srand` — basic PRNG
- `getenv` — environment
- `exit` / `system` — process control

Math

MATH FUNCTIONS

<math.h> provides floating-point math: roots, powers, rounding, trigonometry, logarithms and classification. Functions take and return double, with f and l suffixed versions for float and long double (sqrtf, sqrtl). <tgmath.h> picks the right version automatically. On Linux you must link the math library with -lm.

NOTES

- Trig functions use **radians**
- M_PI isn't standard C (POSIX/MSVC need a define)
- Domain errors return NaN (e.g. sqrt(-1))
- <complex.h> adds double complex

COMMON CALLS

```c
#include <math.h>                // link with -lm on Linux

sqrt(16.0);          // 4.0
pow(2.0, 10);        // 1024.0
cbrt(27.0);          // 3.0
hypot(3, 4);         // 5.0
fabs(-2.5);          // 2.5   (abs() is for ints!)
floor(-2.5);         // -3.0
ceil(-2.5);          // -2.0
round(-2.5);         // -3.0  (half away from zero)
trunc(-2.7);         // -2.0
fmod(7.5, 2.0);      // 1.5
exp(1.0);            // 2.718281828...
log(M_E);            // 1.0   natural log
log10(1000);         // 3.0
sin(M_PI / 2);       // 1.0   (radians)
atan2(1.0, 1.0);     // 0.785398 (π/4)
```

CODE EXPLAINED

- → pow(x, y) — x to the power y
- → fabs — floating absolute value
- → round vs trunc — nearest vs toward zero
- → fmod — floating remainder
- → atan2(y, x) — angle with the correct quadrant

ROUNDING

FUNCTION	2.5	-2.5
floor	2	-3
ceil	3	-2
round	3	-3
trunc	2	-2
rint / nearbyint	2	-2

MORE FUNCTIONS

FUNCTION	COMPUTES
fmin / fmax	min / max (NaN-aware)
lround / llround	round to integer type
log2 / exp2	base-2 log / power
isnan / isinf / isfinite	classify
copysign / signbit	sign handling

CONSTANTS

NAME	VALUE
M_PI	3.14159… (POSIX)
M_E	2.71828… (POSIX)
M_SQRT2	1.41421… (POSIX)
HUGE_VAL	overflow result
NAN / INFINITY	special values

⚠ COMMON MISTAKES

- Linker error undefined reference to 'sqrt': add -lm after your sources
- Using abs() on a double; it converts to int. Use fabs
- Passing degrees to sin; convert: deg * M_PI / 180

✓ TIPS & BEST PRACTICES

- Define _USE_MATH_DEFINES before <math.h> on MSVC for M_PI
- Use hypot(x, y) instead of sqrt(x*x + y*y); it avoids overflow
- Use float versions (sinf) in float-heavy code to avoid conversions

QUICK SUMMARY

- sqrt pow fabs floor ceil round
- Radians for trig
- -lm on Linux
- f / l suffixes or <tgmath.h>

Time & Dates

DATES & TIMES

<time.h> measures time two ways. **Calendar time** (`time_t`, usually seconds since 1970-01-01 UTC) is converted to broken-down `struct tm` fields with `localtime` or `gmtime` and formatted with `strftime`. **Processor time** from `clock()` measures CPU use. For precise wall-clock intervals, use `timespec_get` (C11) or POSIX `clock_gettime`.

NOTES

- `tm_year` is years since 1900; `tm_mon` is 0–11
- `localtime` returns a shared static buffer
- Use `localtime_r` / `gmtime_r` (POSIX) in threads
- `mktime` normalizes out-of-range fields

GETTING & FORMATTING TIME

```c
#include <time.h>

time_t now = time(NULL);                    // seconds since epoch
struct tm local = *localtime(&now);         // copy the static result
printf("%d-%02d-%02d\n", local.tm_year + 1900, local.tm_mon + 1, local.tm_mday);

char buf[64];
strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S", &local);   // 2026-10-12 09:41:07
strftime(buf, sizeof buf, "%A, %B %d", &local);           // Monday, October 12

struct tm due = local;
due.tm_mday += 30;                          // 30 days later
mktime(&due);                               // normalizes month/year

clock_t c0 = clock();  work();
double cpu = (double)(clock() - c0) / CLOCKS_PER_SEC;

struct timespec a, b;                       // wall-clock timing
timespec_get(&a, TIME_UTC);  work();  timespec_get(&b, TIME_UTC);
double secs = (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / 1e9;
```

CODE EXPLAINED

- → `time(NULL)` — current calendar time
- → `*localtime(&now)` — copy before another call overwrites it
- → `strftime` — format with `%` codes
- → `mktime` — fixes `tm_mday = 42` into the next month
- → `clock()` — CPU time; `timespec_get` — wall time

STRFTIME CODES

CODE	OUTPUT
`%Y-%m-%d`	2026-10-12
`%H:%M:%S`	09:41:07
`%a` / `%A`	Mon / Monday
`%b` / `%B`	Oct / October
`%j`	day of year 285
`%z` / `%Z`	-0500 / CDT

STRUCT TM FIELDS

FIELD	RANGE
`tm_year`	years since 1900
`tm_mon`	0–11
`tm_mday`	1–31
`tm_hour` / `tm_min`	0–23 / 0–59
`tm_wday` / `tm_yday`	0–6 (Sun=0) / 0–365
`tm_isdst`	>0 DST, 0 no, <0 unknown

⚠ COMMON MISTAKES

- Printing `tm_year` directly (126) or `tm_mon` (9 for October)
- Using `clock()` to time code that sleeps or waits on I/O; it measures CPU time only
- Keeping the pointer from `localtime` and calling it again; the first result changes

✓ TIPS & BEST PRACTICES

- Store and compute in UTC (`gmtime`), convert to local only for display
- Use ISO 8601 (`%Y-%m-%dT%H:%M:%SZ`) for logs and files
- Use `clock_gettime(CLOCK_MONOTONIC, ...)` (POSIX) for benchmarks

QUICK SUMMARY

- `time_t` + `localtime` / `gmtime` → `struct tm`
- `strftime` — format dates
- `mktime` — normalize / convert back
- `clock` (CPU) vs `timespec_get` (wall)

Debugging with gdb & lldb

DEBUGGING WITH GDB & LLDB

A debugger runs your program under control: stop at **breakpoints**, step line by line, inspect variables and memory, and print the **call stack** when it crashes. Compile with `-g` (debug info) and `-O0` or `-Og` so variables aren't optimized away. Linux uses **gdb**; macOS ships **lldb** (gdb is hard to set up there). The commands are similar.

NOTES

- Pressing Enter repeats the last command
- `bt` after a crash shows where it happened
- Core dumps: `ulimit -c unlimited` first
- Editors (VS Code, CLion) drive the same debuggers

A DEBUGGING SESSION

```
cc -g -O0 main.c -o app

gdb --args ./app input.txt        # or: lldb -- ./app input.txt
(gdb) break parse_line            # stop at a function
(gdb) break main.c:42 if i == 10  # conditional breakpoint
(gdb) run
(gdb) next                        # step over
(gdb) step                        # step into
(gdb) print count                 # $1 = 7
(gdb) print *node                 # whole struct
(gdb) print arr[0]@5              # 5 array elements
(gdb) watch total                 # stop when total changes
(gdb) backtrace                   # call stack
(gdb) frame 1                     # inspect caller
(gdb) finish                      # run until function returns
(gdb) continue
```

CODE EXPLAINED

- → `--args` — pass program arguments
- → `break file:line if cond` — conditional stop
- → `next` vs `step` — over vs into calls
- → `print arr[0]@5` — gdb array slice
- → `watch` — data breakpoint

GDB ↔ LLDB

ACTION	GDB	LLDB
breakpoint	b main	b main
run	r	r
step over / into	n / s	n / s
print	p x	p x / v x
locals	info locals	frame variable
stack	bt	bt
watch	watch x	w s v x
quit	q	q

POST-MORTEM

STEP	COMMAND
enable cores	ulimit -c unlimited
open core	gdb ./app core
macOS	lldb -c /cores/core.1234
where it died	bt

⚠ COMMON MISTAKES

- Debugging an `-O2` build: variables show `<optimized out>` and lines jump around
- Forgetting `-g`: no source lines, only addresses
- Using `print` to debug memory corruption; use ASan first, the debugger second

✓ TIPS & BEST PRACTICES

- Run under ASan with `ASAN_OPTIONS=abort_on_error=1` and debug the stop
- Use `tui enable` (gdb) for a source view in the terminal
- Save breakpoints in `.gdbinit` / `.lldbinit` for repeated sessions

QUICK SUMMARY

- `-g -O0` — debuggable build
- `break` / `run` / `next` / `step`
- `print` / `watch` / `bt`
- gdb on Linux, lldb on macOS

Unit Testing

UNIT TESTING IN C

C has no built-in test framework, but testing is still straightforward. The smallest option is a file of `asserts` run as a program. Frameworks add test discovery, readable failure messages and fixtures: **Unity** (tiny, embedded-friendly), **Criterion** (auto-registers tests), and **CMocka** (adds mocking). Register test executables with CTest or a `make test` target.

NOTES

- A test program returns non-zero on failure
- Run tests under ASan/UBSan
- Test static functions by `#include`-ing the `.c` in the test file
- `ctest --output-on-failure` shows failing output

MINIMAL HARNESS & UNITY

```c
/* test_vec.c — no framework */
#include <assert.h>
#include "vec.h"
int main(void) {
    IntVec v = {0};
    assert(vec_push(&v, 1) == 0);
    assert(v.len == 1 && v.data[0] == 1);
    vec_free(&v);
    puts("ok");
    return 0;
}

/* test_math.c — Unity */
#include "unity.h"
void setUp(void) {}  void tearDown(void) {}
void test_add(void) { TEST_ASSERT_EQUAL_INT(5, add(2, 3)); }
void test_div(void) { TEST_ASSERT_DOUBLE_WITHIN(1e-9, 2.5, divide(5, 2)); }
int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_add);  RUN_TEST(test_div);
    return UNITY_END();
}
```

CODE EXPLAINED

- → `assert` — aborts on the first failure
- → `setUp` / `tearDown` — run around each test
- → `TEST_ASSERT_EQUAL_INT(expected, actual)`
- → `..._WITHIN(tol, ...)` — float comparison
- → `UNITY_END()` — returns failure count

FRAMEWORKS

FRAMEWORK	STRENGTHS
Unity	two files, embedded-friendly
Criterion	auto-registered tests, parallel
CMocka	mocks, used by many C projects
greatest / utest.h	single header
Check	forks per test (crash isolation)

WIRING TESTS IN

TOOL	HOW
make	`test: test_vec` then `./test_vec`
CMake	`add_test(NAME vec COMMAND test_vec)`
CTest	`ctest --test-dir build`
CI	build with `-fsanitize=address,undefined`

⚠ COMMON MISTAKES

- Testing with `assert` in a build that defines `NDEBUG`: nothing is checked
- Tests that depend on run order or leftover global state
- Comparing floats with exact equality in tests

✓ TIPS & BEST PRACTICES

- Keep each test small and named after the behavior: `test_push_grows_capacity`
- Test error paths too: allocation failure, bad input, empty collections
- Add a regression test for every bug you fix

QUICK SUMMARY

- Test = program that exits non-zero on failure
- Unity / Criterion / CMocka
- Register with CTest or `make test`
- Run tests under sanitizers

Static Analysis & Formatting

STATIC ANALYSIS & FORMATTING

Static analyzers read your code without running it and find bugs the compiler doesn't warn about: null dereferences on rare paths, leaks, uninitialized values and API misuse. **clang-tidy** combines analysis with style and modernization checks, **cppcheck** is a standalone analyzer, and GCC has `-fanalyzer`. **clang-format** formats code consistently from a `.clang-format` file.

NOTES

- Analyzers need your real compile flags
- `compile_commands.json` supplies those flags
- Expect some false positives; suppress narrowly
- Run formatters in pre-commit hooks or CI

RUNNING THE TOOLS

```
# Compile database (needed by clang-tidy / clangd)
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
bear -- make                          # for Makefile projects

# Analyzers
clang-tidy src/*.c -p build -checks='clang-analyzer-*,bugprone-*,cert-*'
cppcheck --enable=warning,style --std=c17 src/
gcc -fanalyzer -c src/parser.c
scan-build make                       # Clang static analyzer report

# Formatting
clang-format -i src/*.c include/*.h  # rewrite in place
clang-format --dry-run --Werror src/*.c   # CI check

# .clang-format
#   BasedOnStyle: LLVM
#   IndentWidth: 4
#   ColumnLimit: 100
```

CODE EXPLAINED

- → `-p build` — where `compile_commands.json` lives
- → `-checks=` — choose check groups
- → `scan-build make` — wrap any build, HTML report
- → `clang-format -i` — format files in place
- → `--dry-run --Werror` — fail CI on unformatted code

TOOLS

TOOL	FINDS / DOES
`clang-tidy`	bugs, style, CERT rules, readability
`clang --analyze / scan-build`	path-sensitive bugs, leaks
`gcc -fanalyzer`	leaks, double free, NULL deref
`cppcheck`	bounds, leaks, style; no compiler needed
`clang-format`	formatting
`include-what-you-use`	unneeded / missing includes

USEFUL CLANG-TIDY GROUPS

GROUP	COVERS
`bugprone-*`	likely bugs
`cert-*`	CERT C secure coding
`clang-analyzer-*`	static analyzer
`misc-* / readability-*`	clarity
`performance-*`	inefficiencies

⚠ COMMON MISTAKES

- Running clang-tidy without a compile database: it guesses flags and reports noise
- Turning on every check at once, then ignoring the flood
- Mixing formatting changes with logic changes in one commit

✓ TIPS & BEST PRACTICES

- Start with `bugprone-*` and `clang-analyzer-*`, then add more
- Commit a `.clang-format` and `.clang-tidy` at the repo root
- Silence one line with `// NOLINT(check-name)` and a reason

QUICK SUMMARY

- clang-tidy / cppcheck / `-fanalyzer`
- `compile_commands.json` — real flags
- clang-format + `.clang-format`
- Run all of them in CI

Popular C Libraries

WIDELY USED C LIBRARIES

C's standard library is small, so most projects lean on a few well-known libraries: **cJSON** for JSON, **libcurl** for HTTP, **SQLite** for an embedded database, **zlib** for compression, and **SDL** or **raylib** for graphics, input and audio. Install them with your package manager and get build flags from `pkg-config` or CMake's `find_package`.

NOTES

- macOS: `brew install cjson curl sqlite zlib sdl2 raylib`
- Debian: `apt install libcjson-dev libcurl4-openssl-dev ...`
- Most return error codes and need explicit cleanup
- Single-file libs (SQLite amalgamation, stb_*) can be vendored

CJSON, LIBCURL, SQLITE

```c
cJSON *root = cJSON_Parse("{\"name\":\"Ana\",\"age\":31}");
const cJSON *name = cJSON_GetObjectItem(root, "name");
if (cJSON_IsString(name)) puts(name->valuestring);   // Ana
cJSON_Delete(root);
CURL *h = curl_easy_init();
curl_easy_setopt(h, CURLOPT_URL, "https://example.com");
CURLcode rc = curl_easy_perform(h);          // body → stdout by default
if (rc != CURLE_OK) fprintf(stderr, "%s\n", curl_easy_strerror(rc));
curl_easy_cleanup(h);
sqlite3 *db;  sqlite3_stmt *st;
sqlite3_open("app.db", &db);
sqlite3_prepare_v2(db, "SELECT name FROM users WHERE id = ?",
                   -1, &st, NULL);
sqlite3_bind_int(st, 1, 42);
while (sqlite3_step(st) == SQLITE_ROW)
    puts((const char *)sqlite3_column_text(st, 0));
sqlite3_finalize(st);  sqlite3_close(db);
```

CODE EXPLAINED

- → `cJSON_Parse` / `cJSON_Delete` — parse tree, free it
- → `curl_easy_*` — one-request API
- → `sqlite3_prepare_v2` + `?` — parameterized query
- → `sqlite3_step` loop — one row at a time
- → Headers: `<cjson/cJSON.h>`, `<curl/curl.h>`, `<sqlite3.h>`

LIBRARIES

LIBRARY	PURPOSE	LINK
`cJSON`	JSON parse/print	`-lcjson`
`libcurl`	HTTP(S), FTP clients	`-lcurl`
`SQLite`	embedded SQL database	`-lsqlite3`
`zlib`	deflate / gzip	`-lz`
`SDL2 / raylib`	graphics, input, audio	`pkg-config sdl2` / `-lraylib`
`OpenSSL / libsodium`	crypto, TLS	`-lcrypto` / `-lsodium`

SINGLE-HEADER PICKS

LIBRARY	GIVES YOU
`stb_image.h`	load PNG/JPG
`stb_ds.h`	dynamic arrays, hash maps
`utest.h / greatest.h`	unit tests
`miniaudio.h`	audio playback

⚠ COMMON MISTAKES

- Building SQL with `snprintf` and user input: SQL injection. Always bind parameters
- Forgetting `cJSON_Delete` / `curl_easy_cleanup` / `sqlite3_finalize`: leaks
- Not checking return codes (`CURLcode`, `SQLITE_OK`)

✓ TIPS & BEST PRACTICES

- Use `pkg-config --cflags --libs libcurl sqlite3` for portable flags
- Vendor single-file libraries into `third_party/` with their license
- Call `curl_global_init` once at startup in multithreaded programs

QUICK SUMMARY

- cJSON — JSON
- libcurl — HTTP
- SQLite — embedded SQL
- `pkg-config` / `find_package` — wire them in

Undefined Behavior

UB vs unspecified vs implementation-defined · signed overflow · out-of-bounds · uninitialized reads · strict aliasing · how optimizers exploit UB · UBSan

UNDEFINED BEHAVIOR

Undefined behavior (UB) means the standard places no requirements on what happens. The program might crash, appear to work, or behave differently at –O2. Optimizers **assume UB never happens** and transform code on that basis, so a signed-overflow check or a null check placed after a dereference can be deleted. It's different from **implementation-defined** (documented choice) and **unspecified** (one of several outcomes) behavior.

NOTES

- UB anywhere can affect code before it executes
- "It works" proves nothing about UB
- UBSan catches many cases at runtime
- C23 Annex J lists ~200 UB cases

UB THE OPTIMIZER EXPLOITS

```c
int will_overflow(int x) {
    return x + 1 < x;          // "always false" → compiled to: return 0
}

int get(int *p) {
    int v = *p;                // deref: compiler now assumes p != NULL
    if (p == NULL) return -1;  // check may be removed
    return v;
}

int table[4];
bool exists(int v) {
    for (int i = 0; i <= 4; i++)   // reads table[4]: UB
        if (table[i] == v) return true;
    return false;              // may be optimized to "return true"
}

int x;  printf("%d\n", x);     // uninitialized read
int s = 1 << 32;               // shift ≥ width
float f = 1.5f; int bits = *(int *)&f;   // strict aliasing
```

CODE EXPLAINED

- → `x + 1 < x` — signed overflow "can't happen", so it's false
- → Deref before check — the null check is dead code to the compiler
- → `i <= 4` — out-of-bounds lets the compiler assume the loop exits early
- → Uninitialized reads, big shifts, type-punned pointers — all UB

THREE KINDS OF LOOSENESS

KIND	MEANING	EXAMPLE
`undefined`	no rules at all	signed overflow
`unspecified`	one of several outcomes	argument evaluation order
`implementation-defined`	compiler documents it	`char` signedness, >> on negatives

COMMON UB

CASE	FIX
`signed overflow`	check first / `ckd_add`
`out-of-bounds access`	check indexes
`NULL / dangling deref`	check, set to NULL
`uninitialized read`	initialize
`shift ≥ width / negative`	unsigned, range check
`i = i++`	split statements
`strict aliasing`	`memcpy` / union

⚠ COMMON MISTAKES

- Testing for overflow with `a + b < a` after the fact
- Assuming `-O0` behavior is "the real" behavior; optimized builds may differ legally
- Reasoning about UB in terms of what the hardware does

✓ TIPS & BEST PRACTICES

- Build tests with `-fsanitize=undefined,address`
- Use `-fno-strict-aliasing` / `-fwrapv` only as a last resort for legacy code
- Read John Regehr's and the LLVM blog posts on UB for intuition

QUICK SUMMARY

- UB — no guarantees at all
- Optimizers assume it never happens
- Unspecified / impl-defined are milder
- UBSan + warnings catch much of it

C11 Threads

C11 THREADS

C11 added a portable threading API in `<threads.h>`: `thrd_create` starts a thread, `thrd_join` waits for it, `mtx_t` mutexes protect shared data, `cnd_t` condition variables let threads wait for events, and `call_once` runs initialization exactly once. Support is **optional**: glibc 2.28+, musl and MSVC 2022 provide it, but **macOS doesn't**, so many projects still use pthreads.

NOTES

- `__STDC_NO_THREADS__` defined → not available
- Thread functions return `int`
- Return codes: `thrd_success`, `thrd_error`, …
- `thread_local` gives per-thread variables

THREADS + MUTEX

```c
#include <threads.h>
#include <stdio.h>

static long counter = 0;
static mtx_t lock;
int worker(void *arg) {
    int n = *(int *)arg;
    for (int i = 0; i < n; i++) {
        mtx_lock(&lock);
        counter++;                    // protected
        mtx_unlock(&lock);
    }
    return 0;
}
int main(void) {
    mtx_init(&lock, mtx_plain);
    thrd_t t[4];  int n = 100000;
    for (int i = 0; i < 4; i++) thrd_create(&t[i], worker, &n);
    for (int i = 0; i < 4; i++) thrd_join(t[i], NULL);
    printf("%ld\n", counter);         // 400000
    mtx_destroy(&lock);
}
```

CODE EXPLAINED

- → `int worker(void *arg)` — required thread function signature
- → `mtx_lock` / `mtx_unlock` — critical section
- → `mtx_init(&lock, mtx_plain)` — plain (non-recursive) mutex
- → `thrd_create(&t, fn, arg)` — start a thread
- → `thrd_join` — wait; second arg receives the result

API

FUNCTION	PURPOSE
`thrd_create / thrd_join`	start / wait
`thrd_detach`	run without joining
`mtx_init / lock / unlock`	mutual exclusion
`cnd_wait / cnd_signal`	wait / wake one
`cnd_broadcast`	wake all
`call_once(&flag, fn)`	one-time init

AVAILABILITY

PLATFORM	<THREADS.H>
`glibc 2.28+ (Linux)`	yes
`musl`	yes
`MSVC (VS 2022 17.8+)`	yes
`macOS libSystem`	no → use pthreads
`FreeBSD`	yes

⚠ COMMON MISTAKES

- Incrementing a shared counter without a lock: lost updates (data race = UB)
- Passing the address of a loop variable that changes before the thread reads it
- Forgetting to join or detach; resources leak

✓ TIPS & BEST PRACTICES

- Wrap threading behind your own small API to switch between C11 and pthreads
- Use `_Atomic` counters for simple shared counts (Sheet 93)
- Test with `-fsanitize=thread` to find races

QUICK SUMMARY

- `thrd_create` / `thrd_join`
- `mtx_t` — protect shared data
- `cnd_t` — wait for conditions
- Not on macOS → pthreads

POSIX Threads

POSIX THREADS

pthreads is the threading API on Linux, macOS and other Unix systems. It mirrors C11 threads with more features: attributes for stack size and detach state, read-write locks, barriers and thread cancellation. Thread functions take and return `void *`. Compile and link with `-pthread`. Most pthread functions **return** an error number instead of setting `errno`.

NOTES

- Always wait on a condvar inside a `while` loop
- Spurious wakeups are allowed
- Lock order must be consistent to avoid deadlock
- Windows: use C11 threads or Win32 APIs

PRODUCER / CONSUMER

```c
#include <pthread.h>
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  ready = PTHREAD_COND_INITIALIZER;
static int queue_len = 0;
void *producer(void *arg) {
    pthread_mutex_lock(&m);
    queue_len++;
    pthread_cond_signal(&ready);        // wake a waiter
    pthread_mutex_unlock(&m);
    return NULL;
}
void *consumer(void *arg) {
    pthread_mutex_lock(&m);
    while (queue_len == 0)              // loop: spurious wakeups
        pthread_cond_wait(&ready, &m);  // unlocks m while waiting
    queue_len--;
    pthread_mutex_unlock(&m);
    return NULL;
}
pthread_t p, c;
pthread_create(&c, NULL, consumer, NULL);
pthread_create(&p, NULL, producer, NULL);
pthread_join(p, NULL);  pthread_join(c, NULL);
```

CODE EXPLAINED

- → `PTHREAD_MUTEX_INITIALIZER` — static init, no `init` call
- → `pthread_cond_signal` — wake one waiting thread
- → `while (cond) pthread_cond_wait(...)` — the only correct pattern
- → `pthread_cond_wait` — atomically unlocks, sleeps, relocks
- → `pthread_create(&t, attr, fn, arg)`

CORE API

FUNCTION	PURPOSE
`pthread_create / join`	start / wait
`pthread_detach`	auto-cleanup on exit
`pthread_mutex_lock / unlock`	mutex
`pthread_cond_wait / signal`	condition variable
`pthread_rwlock_rdlock / wrlock`	many readers, one writer
`pthread_once`	one-time init

PTHREADS ↔ C11

PTHREADS	C11
`pthread_t`	`thrd_t`
`pthread_mutex_t`	`mtx_t`
`pthread_cond_t`	`cnd_t`
`pthread_key_t`	`tss_t` / `thread_local`
`void *(*)(void *)`	`int (*)(void *)`

⚠ COMMON MISTAKES

- `if` instead of `while` around `pthread_cond_wait`
- Checking `errno` after a pthread call; the error is the return value
- Locking two mutexes in different orders in different threads: deadlock

✓ TIPS & BEST PRACTICES

- Compile and link with `-pthread`, not just `-lpthread`
- Keep critical sections short; never call slow I/O while holding a lock
- Run under ThreadSanitizer (`-fsanitize=thread`) in CI

QUICK SUMMARY

- `pthread_create` / `pthread_join`
- mutex + condvar in a `while` loop
- Errors are return values
- Build with `-pthread`

Atomics

ATOMICS

`<stdatomic.h>` (C11) provides **atomic** types and operations that are safe to use from multiple threads without a lock: `_Atomic int counter;` makes `counter++` an indivisible read-modify-write. Every atomic operation takes a **memory order**; the default, `memory_order_seq_cst`, is the easiest to reason about. A **data race** on a non-atomic variable is undefined behavior.

NOTES

- Plain `++` on `_Atomic` is atomic (seq_cst)
- `atomic_int` etc. are typedefs for `_Atomic int`
- `atomic_flag` is the only guaranteed lock-free type
- `volatile` is **not** atomic

ATOMIC COUNTERS & FLAGS

```c
#include <stdatomic.h>

atomic_int hits = 0;                      // _Atomic int
atomic_bool ready = false;

void on_request(void) {
    atomic_fetch_add(&hits, 1);           // same as hits++ here
}

atomic_fetch_add_explicit(&hits, 1, memory_order_relaxed);  // just counting
int seen = atomic_load(&hits);

int expected = 0;                         // compare-and-swap
if (atomic_compare_exchange_strong(&hits, &expected, 100))
    puts("was 0, now 100");               // else expected = current value

atomic_flag lock = ATOMIC_FLAG_INIT;      // tiny spinlock
while (atomic_flag_test_and_set(&lock)) { /* spin */ }
/* critical section */
atomic_flag_clear(&lock);

atomic_store_explicit(&ready, true, memory_order_release);   // publish
```

CODE EXPLAINED

- → `atomic_fetch_add` — atomic `+=`, returns old value
- → `memory_order_relaxed` — atomicity only, no ordering
- → `compare_exchange` — set only if current == expected
- → `atomic_flag_test_and_set` — basic spinlock
- → `release` store / `acquire` load — publish data safely

OPERATIONS

FUNCTION	DOES
`atomic_load / atomic_store`	read / write
`atomic_exchange`	swap, return old
`atomic_fetch_add / sub / or / and`	RMW, return old
`atomic_compare_exchange_strong`	CAS
`..._weak`	CAS that may fail spuriously (loops)

MEMORY ORDERS

ORDER	USE
`seq_cst`	default; one global order
`acquire / release`	lock-free publish / consume
`acq_rel`	read-modify-write both ways
`relaxed`	counters, statistics

⚠ COMMON MISTAKES

- Using `volatile int` for thread communication: no atomicity, no ordering
- Mixing atomic and non-atomic access to the same variable
- Choosing `relaxed` to publish data another thread will read; use release/acquire

✓ TIPS & BEST PRACTICES

- Default to `seq_cst`; optimize memory orders only with measurements
- Prefer a mutex for anything more complex than a counter or flag
- Check `atomic_is_lock_free(&x)` for large atomic types

QUICK SUMMARY

- `_Atomic T` / `atomic_int`
- `atomic_fetch_add`, CAS
- seq_cst default; relaxed for counters
- Data races on plain vars = UB

Signals & Processes

SIGNALS & PROCESSES

A **signal** is an asynchronous notification sent to a process: `SIGINT` on Ctrl-C, `SIGTERM` to request shutdown, `SIGSEGV` on a bad memory access. Standard C offers `signal` and `raise`; POSIX adds the more reliable `sigaction`. Handlers may only do very little: set a `volatile sig_atomic_t` flag and return. POSIX also creates processes with `fork`, `exec*` and `waitpid`.

NOTES

- Only async-signal-safe functions in handlers
- `printf` and `malloc` are **not** signal-safe
- `SIGKILL` / `SIGSTOP` can't be caught
- `fork` duplicates the process; `exec` replaces it

GRACEFUL SHUTDOWN & CHILD PROCESS

```c
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

static volatile sig_atomic_t stop = 0;
static void on_sigint(int sig) { (void)sig; stop = 1; }   // just set a flag

struct sigaction sa = {0};
sa.sa_handler = on_sigint;
sigaction(SIGINT, &sa, NULL);           // POSIX; signal() is simpler
while (!stop) do_work();
puts("shutting down");                  // safe here, not in handler

pid_t pid = fork();
if (pid == 0) {                         // child
    execlp("ls", "ls", "-l", (char *)NULL);
    _exit(127);                         // only reached if exec failed
}
int status;
waitpid(pid, &status, 0);               // parent waits
if (WIFEXITED(status)) printf("exit %d\n", WEXITSTATUS(status));
```

CODE EXPLAINED

- → `volatile sig_atomic_t` — the only portable handler-shared type
- → `sigaction` — well-defined semantics across systems
- → Main loop checks the flag and cleans up normally
- → `fork` returns 0 in the child, child's pid in the parent
- → `WEXITSTATUS` — child's exit code

COMMON SIGNALS

SIGNAL	DEFAULT	CAUSE
`SIGINT`	terminate	Ctrl-C
`SIGTERM`	terminate	`kill pid`
`SIGSEGV`	core dump	invalid memory access
`SIGABRT`	core dump	`abort()` / failed `assert`
`SIGPIPE`	terminate	write to closed pipe/socket
`SIGCHLD`	ignore	child exited

PROCESS CALLS (POSIX)

CALL	DOES
`fork()`	clone the process
`execvp(file, argv)`	replace with a program
`waitpid(pid, &st, 0)`	wait for a child
`kill(pid, sig)`	send a signal
`getpid() / getppid()`	own / parent id

⚠ COMMON MISTAKES

- Calling `printf`, `malloc` or `exit` inside a signal handler; use a flag (or `_exit` / `write`)
- Not reaping children with `waitpid`; they linger as zombies
- Ignoring `SIGPIPE` in network code, so a closed socket kills the program

✓ TIPS & BEST PRACTICES

- Use `posix_spawn` for simple "run this program" cases
- `signal(SIGPIPE, SIG_IGN)` and handle `EPIPE` instead
- For complex signal handling, use `signalfd` (Linux) or a self-pipe

QUICK SUMMARY

- Handlers: set a `sig_atomic_t` flag only
- `sigaction` over `signal`
- `fork` + `exec` + `waitpid`
- `SIGKILL` can't be caught

Generic Programming

GENERIC CODE IN C

C has no templates, but there are several ways to write code that works for many types. `void *` plus an element size (like `qsort`) works for any type at the cost of type safety. Macros can **generate** type-specific code. `_Generic` (C11) selects an expression based on the type of its argument, giving compile-time overloading. C23 `typeof` makes type-generic macros easier to write safely.

NOTES

- `_Generic` is resolved at compile time
- `<tgmath.h>` is built on `_Generic`
- `void *` APIs need a size and callbacks
- Macro-generated code is hard to debug; keep it small

_GENERIC & MACRO TEMPLATES

```c
#define type_name(x) _Generic((x),    \
    int: "int", double: "double",     \
    char *: "char *", default: "other")
printf("%s\n", type_name(3.0));        // double

double absd(double x) { return x < 0 ? -x : x; }
long   absl_(long x)  { return x < 0 ? -x : x; }
#define ABS(x) _Generic((x), double: absd, long: absl_, default: absl_)(x)
ABS(-2.5);                             // calls absd

#define DEFINE_VEC(T)                                          \
    typedef struct { T *data; size_t len, cap; } Vec_##T;      \
    static inline int Vec_##T##_push(Vec_##T *v, T x) { /* ... */ return 0; }
DEFINE_VEC(int)                        // Vec_int, Vec_int_push
DEFINE_VEC(double)                     // Vec_double, Vec_double_push

#define MAX(a, b) ({ typeof(a) a_ = (a), b_ = (b); a_ > b_ ? a_ : b_; })  // GNU
void swap_any(void *a, void *b, size_t size);   // void * + size
```

CODE EXPLAINED

- → `_Generic(x, type: expr, ...)` — picks by `x`'s type
- → `ABS(x)` — selects the function, then calls it
- → `DEFINE_VEC(T)` — stamps out a type-specific vector
- → `typeof` + statement expression — single evaluation (GNU)
- → `void *` + `size` — runtime-generic

APPROACHES

TECHNIQUE	SAFETY	COST
`void * + size`	none	indirect calls, `memcpy`
`macro generation`	typed	code bloat, hard debugging
`_Generic dispatch`	typed	list each type
`typeof macros (C23)`	typed	macro pitfalls remain

_GENERIC RULES

RULE	DETAIL
`controlling expr`	not evaluated
`lvalue conversion`	`const int` matches `int`
`arrays`	decay: match T *
`default`	optional fallback

⚠ COMMON MISTAKES

- Forgetting a type in `_Generic`; with no `default`, it's a compile error
- Passing `float` to an `_Generic` that only lists `double`
- Huge macro templates; errors point at the macro call, not the bug

✓ TIPS & BEST PRACTICES

- Use `_Generic` for thin type-dispatch wrappers over real functions
- Keep generated code minimal and call shared `void *` helpers
- Look at stb_ds.h or klib for battle-tested generic containers

QUICK SUMMARY

- `void *` + size — runtime generic
- Macros — generate per-type code
- `_Generic` — compile-time dispatch
- `typeof` (C23) — safer macros

C23 at a Glance

WHAT C23 ADDS

C23 (ISO/IEC 9899:2024) is the biggest update since C99. It modernizes the language with `nullptr`, real `bool`/`true`/`false` keywords, `constexpr`, `typeof` and `auto` inference, standard `[[attributes]]`, binary literals and digit separators, `#embed`, and new headers for checked arithmetic and bit utilities. It also **removes** old K&R function definitions and makes `f()` mean `f(void)`.

NOTES

- `-std=c23` (GCC 14+, Clang 18+); `-std=c2x` earlier
- `__STDC_VERSION__` is `202311L`
- `constexpr` needs GCC 13+ / Clang 19+
- Library parts depend on your libc version

C23 IN ONE SCREEN

```c
#include <stdbit.h>
#include <stdckdint.h>
constexpr int N = 0b1000'0000;        // binary + separator = 128
int *p = nullptr;                     // typed null pointer
bool ok = true;                       // keywords, no header
auto total = 0L;                      // inferred: long
typeof(total) copy = total;

[[nodiscard]] int parse(const char *s);
static_assert(N == 128);              // no message needed

int sum;
if (ckd_add(&sum, INT_MAX, 1)) puts("overflow");
unsigned ones = stdc_count_ones(0xF0u);   // 4

int f();                              // means f(void) now
enum Color : unsigned char { RED, GREEN };  // fixed underlying type
static const unsigned char logo[] = {
#embed "logo.png"
};
```

CODE EXPLAINED

- → `0b...` and `'` — binary literals & separators
- → `nullptr`, `bool`, `true` — now keywords
- → `constexpr` / `auto` / `typeof` — inference & constants
- → `ckd_add`, `stdc_count_ones` — new headers
- → `#embed` — include file bytes

FEATURE CHECKLIST

FEATURE	REPLACES
`nullptr`	NULL
`bool / true / false`	<stdbool.h>
`static_assert, thread_local, alignas`	_Static_assert …
`[[nodiscard]] etc.`	__attribute__
`typeof / typeof_unqual`	GNU __typeof__
`#elifdef, #warning, __has_include`	workarounds
`= {} empty init`	= {0}
`strdup, memccpy, memset_explicit`	POSIX/extensions

REMOVED / CHANGED

CHANGE	IMPACT
`K&R definitions removed`	`int f(a) int a; {}` errors
`f() means f(void)`	calls with args now error
`realloc(p, 0)`	now UB
`two's complement required`	signed repr fixed

⚠ COMMON MISTAKES

- Using C23 features in code that must build with older compilers or MSVC
- Defining your own `bool` / `true` macros, which clash with the new keywords
- Assuming every C23 library function exists in your libc yet

✓ TIPS & BEST PRACTICES

- Guard optional use with `#if __STDC_VERSION__ >= 202311L`
- Adopt `nullptr`, `[[nodiscard]]` and `constexpr` first; they catch bugs
- Check compiler status pages (gcc.gnu.org, clang.llvm.org) per feature

QUICK SUMMARY

- `nullptr` `bool` `constexpr` `typeof` `auto`
- `[[attributes]]` standardized
- `#embed`, `0b`, `'` separators
- `f()` = `f(void)`; K&R gone

Sheet 97

Quick Reference — Language & Syntax

types & limits · printf specifiers · operator precedence · control flow · declarations (cdecl) · pointers · structs / unions / enums · preprocessor

LANGUAGE AT A GLANCE

A one-page lookup for C syntax: types, `printf` specifiers, operator precedence, control flow, declarations, pointers and the preprocessor. Each row points to the sheet with the details.

HOW TO READ

- **#** column = sheet number
- C23-only items marked `(C23)`
- `T` = any type, `p` = pointer

TYPES

SYNTAX	MEANING	#
`char short int long`	integers (min 8/16/16/32 bits)	10
`unsigned T`	non-negative, wraps	10
`int32_t uint8_t size_t`	fixed / size types	11
`float double`	floating (~7 / ~15 digits)	12
`bool true false`	<stdbool.h> or C23	14
`const volatile`	qualifiers	16
`(T)x`	cast	15

PRINTF SPECIFIERS

SPEC	TYPE	#
`%d %i`	`int`	6
`%u %x %o`	`unsigned`	7
`%ld %lld %zu`	`long` / `long long` / `size_t`	6
`%f %e %g`	`double`	6
`%c %s %p`	char / string / pointer	6
`%-8.2f %05d`	align, precision, pad	7
`PRId64`	`int64_t`	7

PRECEDENCE (HIGH → LOW)

OPS		#
`() [] -> . x++`	postfix	26
`++x ! ~ * & (T) sizeof`	unary	26
`* / % then + -`	arithmetic	22
`<< >>`	shift	24
`< <= > >= == !=`	compare	23
`& ^ | && ||`	bitwise, logical	24
`?: = += ,`	lowest	25

CONTROL FLOW

SYNTAX	#
`if (c) {} else if (c) {} else {}`	27
`c ? a : b`	28
`switch (x) { case 1: … break; default: }`	29
`for (int i = 0; i < n; i++)`	30
`while (c) {} do {} while (c);`	31
`break continue goto label`	32

DECLARATIONS & POINTERS

SYNTAX	MEANING	#
`int *p = &x;`	pointer, address-of	44
`*p p[i] p->m`	deref, index, member	44
`int a[N] int a[] = {…}`	arrays	55
`int (*fp)(int)`	function pointer	41
`const int *p / int *const p`	const data / const ptr	48
`void *`	generic pointer	48

USER TYPES & PREPROCESSOR

SYNTAX	#
`struct S { int x; }; s.x p->x`	63
`typedef struct {…} T;`	64
`union U { int i; float f; };`	66
`enum E { A, B = 5 };`	67
`#include #define #if #ifdef`	76–78
`# ## __FILE__ __LINE__`	79

⚠ COMMON MISTAKES

- `=` vs `==` in conditions (23)
- Missing `break` in `switch` (29)
- Off-by-one: `i <= n` (30)

✓ TIPS & BEST PRACTICES

- Turn on `-Wall -Wextra` (4)
- Parenthesize macros and mixed operators (26, 77)
- Initialize every variable (9)

QUICK SUMMARY

- Types 10–16 · Strings 17–21
- Operators 22–26 · Control 27–32
- Functions 36–43 · Pointers 44–49
- Structs 63–69 · Preprocessor 76–79

Sheet 98

Quick Reference — Library & Tools

string functions · memory functions · file I/O · errno codes · compiler flags · gdb / lldb commands · Make variables · standards matrix

LIBRARY & TOOLS AT A GLANCE

A one-page lookup for the standard library, compiler and debugger commands, plus a standards matrix showing which version of C introduced each major feature.

HOW TO READ

- # column = sheet number
- `(POSIX)` = not in ISO C
- ✔ = available in that standard

STRINGS & MEMORY

FUNCTION	DOES	#
`strlen strcmp strncmp`	length / compare	17, 19
`snprintf(d, n, fmt, …)`	safe build / copy	18
`strchr strstr strtok`	search / split	19
`strtol strtod`	parse numbers	20
`memcpy memmove memset`	raw bytes	52
`malloc calloc realloc free`	heap	50–51

I/O

FUNCTION	DOES	#
`fopen fclose`	open / close `FILE *`	71
`fgets getline (POSIX)`	read a line	72
`fprintf fputs`	write text	73
`fread fwrite fseek ftell`	binary	74
`perror strerror errno`	errors	33
`scanf sscanf`	parse input	70

COMPILER FLAGS

FLAG	DOES	#
`-std=c17 / -std=c23`	standard	2
`-Wall -Wextra -Wpedantic`	warnings	4
`-g -O0 / -O2`	debug / release	4
`-fsanitize=address`	runtime checks (+ `undefined`)	54
`-I -L -l -D`	paths, libs, macros	3
`-c -E -S`	stop at a stage	3

DEBUGGER & MAKE

COMMAND	DOES	#
`b / r / n / s`	break, run, next, step	86
`p x / bt / watch x`	print, stack, watch	86
`$@ $< $^`	target, first, all prereqs	81
`CC CFLAGS LDLIBS`	make variables	81
`cmake -B build`	configure	82

STANDARDS MATRIX

FEATURE	C89	C99	C11	C17	C23
`// comments, inline, VLAs`		✔	✔	✔	✔
`stdint.h, stdbool.h, designated init`		✔	✔	✔	✔
`_Static_assert, _Generic, anon structs`			✔	✔	✔
`threads.h, stdatomic.h`			✔	✔	✔
`nullptr, constexpr, typeof, #embed`					✔
`[[attributes]], bool keyword, 0b`					✔

⚠ COMMON MISTAKES

- `-lm` before sources (3, 84)
- `p = realloc(p, n)` leaks on failure (51)
- `while (!feof(f))` (72)

✓ TIPS & BEST PRACTICES

- Build with sanitizers during development (54)
- Use `snprintf`, not `strcpy` / `strcat` (18)
- Check every return value (33)

QUICK SUMMARY

- Memory 50–54 · Data structures 55–62
- Files 70–75 · Builds 80–82
- Library 83–85 · Tools 86–89
- Advanced 90–96